

Optimizing Data Layouts for Evolving Cloud Table Storage

by

Sivaprasad Sudhir

B.Tech., Indian Institute of Technology Bombay, 2017

M.S., Carnegie Mellon University, 2019

Submitted to the Department of Electrical Engineering and Computer Science
in partial fulfillment of the requirements for the degree of

DOCTOR OF PHILOSOPHY

at the

MASSACHUSETTS INSTITUTE OF TECHNOLOGY

September 2025

© 2025 Sivaprasad Sudhir. All rights reserved.

The author hereby grants to MIT a nonexclusive, worldwide, irrevocable, royalty-free license to exercise any and all rights under copyright, including to reproduce, preserve, distribute and publicly display copies of the thesis, or release the thesis under an open-access license.

Authored by:	Sivaprasad Sudhir Department of Electrical Engineering and Computer Science August 15, 2025
Certified by:	Michael J. Cafarella Principal Research Scientist, Thesis Supervisor
Certified by:	Samuel R. Madden MIT College of Computing Distinguished Professor, Thesis Supervisor
Accepted by:	Leslie A. Kolodziejski Professor of Electrical Engineering and Computer Science Chair, Department Committee on Graduate Students

THESIS COMMITTEE

THESIS SUPERVISORS

Michael J. Cafarella

Principal Research Scientist

MIT Computer Science and Artificial Intelligence Lab

Samuel R. Madden

MIT College of Computing Distinguished Professor

Department of Electrical Engineering and Computer Science

THESIS READERS

Tim Kraska

Associate Professor

Department of Electrical Engineering and Computer Science

Optimizing Data Layouts for Evolving Cloud Table Storage

by

Sivaprasad Sudhir

Submitted to the Department of Electrical Engineering and Computer Science
on August 15, 2025 in partial fulfillment of the requirements for the degree of

DOCTOR OF PHILOSOPHY

ABSTRACT

Modern data analytics platforms increasingly adopt disaggregated architectures, storing data in cost-effective cloud object stores. While this approach enables a clean separation of concerns, allowing each layer to be independently managed and scaled, it introduces significant performance bottlenecks due to expensive data movement. Effective data layouts, which organize data to minimize unnecessary data reads, are thus critical to achieving high query performance. However, existing techniques typically rely on manually specified layouts, collect limited metadata, or lack mechanisms to dynamically adapt to changing data and workloads.

This thesis investigates adaptive, metadata-rich, expressive data layouts for cloud table storage. First, we introduce **Pando**, a correlation-aware layout technique that leverages rich metadata on query predicates to significantly improve data skipping. Next, we propose **CopyRight**, a partial replication strategy that selectively replicates subsets of data and optimizes each replica differently, efficiently serving heterogeneous query patterns. Finally, we describe Self-Organizing Data Containers (SDCs), a practical table storage layer for the cloud that incrementally reorganizes complex data layouts based on changes in data and workload distributions.

Thesis supervisor: Michael J. Cafarella

Title: Principal Research Scientist

Thesis supervisor: Samuel R. Madden

Title: MIT College of Computing Distinguished Professor

Acknowledgments

I would first like to thank my advisors, Sam Madden and Mike Cafarella. From the very beginning of my PhD, they have been both encouraging and supportive, and easy to work with. They gave me the freedom to explore my ideas and choose my projects, while providing thoughtful guidance whenever I needed it. Whether it was brainstorming research directions, refining the framing of a paper, or rehearsing a talk, they consistently provided thoughtful feedback that strengthened my work. Beyond research, Sam and Mike have provided valuable personal advice and career guidance. No matter how busy they were, they always made time to meet. Their trust, patience, and support created an environment where I could explore my ideas and grow as a researcher and as a person, and I could not have asked for better mentors.

I would also like to thank Tim Kraska for serving on my thesis committee and for advising me on several projects. His advice has always been practical and specific, often giving concrete examples that helped me align my research in the right direction. He has offered valuable guidance on both research and career decisions, and I have greatly appreciated his feedback throughout my PhD. I have also enjoyed our many informal conversations by the coffee machine, which often sparked new ideas and perspectives.

I would also like to thank my mentors and collaborators from my internships for their guidance and support. At Google, I was fortunate to work with Jagan Sankaranarayanan and Vijayshankar Raman, and at Meta, with Wenbo Tao. Jagan provided excellent high-level guidance on my projects and helped me see the broader perspective. Shankar took the time to walk me through low-level technical details with rigor and depth, and I learned an enormous amount of technical knowledge from those conversations. Wenbo was both a friend and a technical mentor, whose insights and encouragement made the work even more enjoyable.

I am grateful to my labmates for making my time at MIT both productive and enjoyable. I have learned a great deal from our discussions, collaborations, and casual conversations. I would like to thank Anil, Yi, Wenbo, Favyen, Oscar, Joana, Matt Perron, Jialin, Kapil, Zeyuan, Tianyu, Anna, Markos, Geoffrey, Amadou, Eugenie, Ferdi, Ziniu, Xinjing, Ani, Darryl, Peter, Matt Russo, Sylvia, Sushrut, Wenjia, Yuze, Pascal, Dominik, Fabi, Emmanuel, Lei, Andreas, Ibrahim, Elkinid, Brit, Laurent, Chunwei, and Gerardo for their friendship, advice, and support over the years.

I am thankful to my friends for their companionship and encouragement throughout these years. I am especially grateful to Parth and Khyati for always being there for me. Finally, I would like to thank my partner, my sister, my parents, and my grandmother for their unwavering support.

Contents

<i>List of Figures</i>	11
<i>List of Tables</i>	15
1 Introduction	17
1.1 Correlation-Aware Multi-Dimensional Data Layouts	20
1.2 Partial Replication for Heterogeneous Access Patterns	21
1.3 Self-Organizing Data Containers	21
2 Pando: Enhanced Data Skipping with Logical Data Partitioning	23
2.1 Introduction	23
2.2 Motivation	25
2.3 Overview	29
2.4 Data Structure	29
2.5 Query Execution	31
2.6 Layout Optimization	31
2.6.1 Top-Down Tree Construction	32
2.6.2 Bottom-Up Block Assignment	35
2.7 Data Changes and Workload Shifts	37
2.8 Evaluation	39
2.8.1 Datasets and Workloads	39
2.8.2 Experimental Setup	40
2.8.3 Results	42
2.9 Related Work	47
2.10 Conclusion	48
3 CopyRight: Instance-Optimized Replicated Data Layouts	49
3.1 Introduction	49
3.2 Motivation	51
3.3 Overview	54
3.4 Layout	55
3.5 Layout Optimizer	56
3.5.1 Clustering Queries	59
3.5.2 Replication Configuration	59
3.5.3 Candidate Regions	60
3.5.4 Finding the Pareto Optimal Frontier	61

3.5.5	Space Allocation	65
3.6	Query Execution	66
3.7	Evaluation	66
3.7.1	Experimental Setup	66
3.7.2	Datasets and Workloads	69
3.7.3	Overall Results	71
3.7.4	Scalability	72
3.8	Related Work	74
3.9	Conclusion	75
4	Self-Organizing Data Containers	77
4.1	Introduction	77
4.2	Background	80
4.3	SDC Overview	81
4.4	Self-Organization	82
4.4.1	Static Data and Workload	82
4.4.2	Dynamic Data	82
4.4.3	Dynamic Workloads	84
4.4.4	Parallelism	85
4.5	Evaluation	85
4.5.1	Experimental Setup	85
4.5.2	Dynamic Data	86
4.5.3	Dynamic Workloads	88
4.6	Related Work	88
4.7	Conclusion	89
5	Conclusion and Future Work	91
5.1	Future Work	91

List of Figures

1.1	We propose three techniques for optimizing cloud table storage through adaptive, metadata-driven data layouts that evolve in response to real-world data and workload changes. First, we introduce Pando , a correlation-aware data layout technique that exploits rich query predicate metadata to significantly accelerate data skipping compared to prior methods. Second, we develop CopyRight , a partial replication mechanism that selectively replicates subsets of data, independently optimizing each replica to efficiently support heterogeneous query access patterns. Finally, we present Self-Organizing Data Containers (SDCs), a practical, cloud-native table storage layer that builds expressive data layouts and incrementally reorganizes them as the underlying data and workload evolve.	18
2.1	An example Pando layout for NYC Taxi dataset. B_0, B_1 , etc. are physical data blocks. There are two logical partitioning trees, one with predicates on <code>DISTANCE(start_point, end_point)</code> , another on <code>cost</code> . Leaf nodes of the tree map to the physical blocks that intersect the logical data ranges.	25
2.2	Figure 2.2a shows a 2-dimensional dataset with correlated columns X and Y. Consider a workload with 2 kinds of queries. Half the queries have a range predicate on column X and the other half have a range predicate on Y. Figure 2.2b shows the layout that qd-tree and MTO will find. Nodes traversed during execution of queries are marked along the edges of the tree. Each query scans the blocks in the leaf nodes that intersect the query. Figure 2.2c shows the Pando layout with two logical partitioning trees and data blocks that align well with the logical trees and query access patterns.	26
2.3	Overview of Pando architecture.	29
2.4	Query execution in Pando for the layout in Figure 2.2	30
2.5	The trees constructed during the top-down phase of Pando 's layout optimizer. The grid visualizes the correlation between the leaf nodes of the trees.	32
2.6	End-to-end query execution time on Spark.	39
2.7	Average number of blocks accessed during query execution. Data-induced predicates (diPs) is not relevant to Meta1 as it does not involve any joins. . .	40
2.8	Reduction in query times achieved by Pando compared to MTO for each query template.	41
2.9	Size of the partition tree indexes.	42
2.10	Time overhead incurred by Pando in offline optimization.	43

2.11	Pando's performance across varying dataset sizes, and workload sizes.	43
2.12	Pando's performance with data inserts. Although inserted data is available for querying quickly in the baselines, Pando makes up for it with faster execution of queries.	45
2.13	Pando's performance across varying minimum block sizes, and optimizer sample rates.	46
3.1	As the number of query templates (different access patterns) in the workload increases, the scan overhead of non-replicated layouts increases, degrading performance.	51
3.2	Figure 3.2a shows a multi-dimensional dataset and workload with 2 columns X and Y. The grey dots represent tuples and the rectangles represent queries. Figure 3.2b shows the data range-partitioned on column Y. Figure 3.2c shows the best layout that Tsunami is able to obtain for the workload. Figure 3.2d shows CopyRight's layout with sufficient budget to replicate a quarter of the table. The upper right quarter of the table is replicated. Layout of one of the replicas is optimized for the short and wide red queries. Other one is optimized for the tall and narrow green queries. With enough budget to replicate half the table, CopyRight also replicates the bottom right quarter with one replica optimized for red queries and other for blue queries.	52
3.3	Overview of CopyRight architecture.	54
3.4	An example Grid Tree.	56
3.5	An example Augmented Grid layout.	57
3.6	Overview of the layout optimizer with 0.125X replication budget.	58
3.7	Each dot in Figure 3.7a represents replicating a candidate region under a replication configuration. Shaded region represents partially replicated layouts. Blue lines form the Pareto frontier. In Figure 3.7b, the point $\times$ on the frontier corresponds to replication configuration RC_i . The point on an edge corresponds to splitting the candidate region and replicating fraction f under RC_j and $1 - f$ under RC_k	62
3.8	Pareto Frontier estimation.	62
3.9	Performance of layouts at various replication budgets on full datasets.	67
3.10	Performance of layouts at various replication budgets on modified datasets.	68
3.11	Performance of layouts at replication budget 1X.	69
3.12	Index creation time across workloads.	70
3.13	CopyRight's performance across workloads with varying dataset and workload parameters.	72
3.14	CopyRight's performance across workloads with varying dataset and workload parameters.	73
3.15	Index creation time with workload complexity.	74

4.1	SDC organizes data and metadata as immutable files on cloud object stores. Clients access data through the SDC library, which is integrated into their application code and exposes a standard table API. This API supports scans with column projections and filter pushdowns, inserts, updates, and deletes. SDC library guarantees ACID semantics, enabling multiple clients to safely interact with the same data concurrently.	79
4.2	An illustrative example of how SDC handles dynamic data and incremental re-optimization.	83
4.3	LST-Bench workload extensions to evaluate SDC. There are four different types corresponding to loading data, running TPC-DS queries, running TPC-DS data maintenance, and running OPTIMIZE. The sequence of operations for evaluating SDC under dynamic data and dynamic workload is shown above.	86
4.4	Performance of SDC with dynamic data.	87
4.5	Performance of SDC with dynamic workloads.	87

List of Tables

3.1	Characteristics of the datasets and workloads. Properties of the modified workload in parenthesis.	70
-----	--	----

Chapter 1

Introduction

Modern enterprises collect and analyze vast amounts of data to gain insights and make informed decisions. Global data volumes are now estimated to exceed 175 zettabytes, spanning everything from user clickstreams and IoT sensor readings to transaction logs and scientific datasets [35]. Businesses increasingly rely on this data to optimize operations, personalize user experiences, and train AI models. Supporting such scale and variety requires data systems that can deliver consistently high performance across diverse workloads.

Historically, database systems have adopted a monolithic architecture, with the system under control of all aspects of data management, including data placement, layout, scheduling, allocation of computation and memory to queries, as well as query optimization and execution. They typically coupled compute and storage into hardware that ran on-premises. This forced enterprises to provision and pay for the peak of user load and data under management, which becomes very costly as datasets grew, especially when query workloads are variable.

In contrast, in the cloud, data services are increasingly disaggregated, separating storage from compute engines. Data analytics platforms store raw data into cloud storage services, or “data lakes”, such as Amazon S3 [4], Azure Data Lake Storage[7], and Google Cloud Storage[8], and data is often stored in open file formats like Apache Parquet [6] and ORC [5]. Using open file formats makes data lake content directly accessible to various analytics engines, including machine learning systems. Execution is handled by compute engines, which run on separate clusters and load data directly from the data lake storage. This design enables a separation of concerns, allowing each layer to be managed and scaled independently, unlike the “shared nothing” designs of conventional DBMSes, where physical nodes that store data are created for each processing node that operates on this data. Additionally, these systems benefit from cloud object stores, which offer extremely low storage costs, high availability, strong durability guarantees (often exceeding 10 nines), and geo-replication. The rise in popularity of data processing systems like Spark [96] and Presto [78] is driven by their successful adoption of this new style of cloud architecture.

While the success of these new systems demonstrates the advantages of this new way of architecting data-driven applications, disaggregation leaves a great deal of performance on the table. For example, when running TPC-H, Amazon Redshift’s disaggregated variant, Redshift Spectrum, is at least $8\times$ slower than the tightly integrated Redshift system [88]. This inefficiency is partly due to the layered design of Spectrum, which requires it to load data from underlying S3 files, preventing it from sharing information about data structure

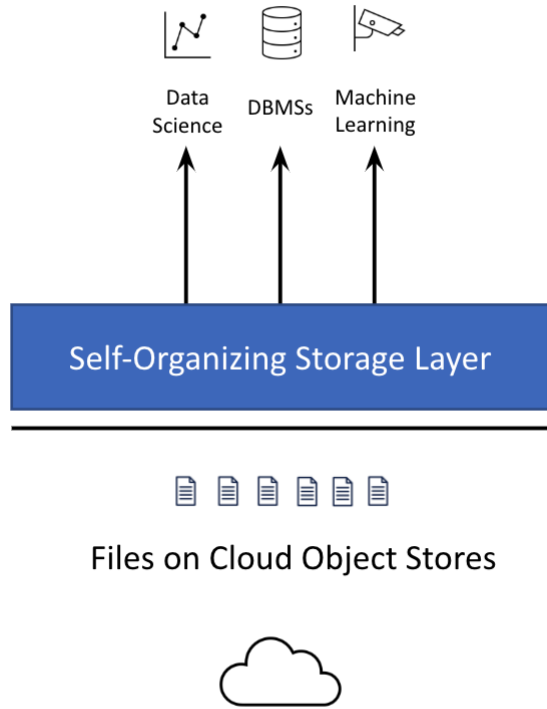


Figure 1.1: We propose three techniques for optimizing cloud table storage through adaptive, metadata-driven data layouts that evolve in response to real-world data and workload changes. First, we introduce **Pando**, a correlation-aware data layout technique that exploits rich query predicate metadata to significantly accelerate data skipping compared to prior methods. Second, we develop **CopyRight**, a partial replication mechanism that selectively replicates subsets of data, independently optimizing each replica to efficiently support heterogeneous query access patterns. Finally, we present **Self-Organizing Data Containers (SDCs)**, a practical, cloud-native table storage layer that builds expressive data layouts and incrementally reorganizes them as the underlying data and workload evolve.

and representation between the storage and query execution layers, and partly due to the fact that S3 has lower bandwidth and higher latency than local storage. Consequently, reducing the amount of data read from storage becomes critically important. To reduce data movement, traditional databases employ various optimizations, such as sorting data, building indexes, and optimizing data layouts based on collected metadata like histograms and query statistics [47, 83, 25, 32, 95, 13, 40, 15, 31, 30]. However, simply storing data in data lakes typically omits capturing such performance-related metadata, preventing them from effectively optimizing data layouts and exacerbating performance challenges in the inherently slower cloud storage environment.

What is needed is a way to build efficient data systems in the cloud while retaining the advantages of disaggregation. Our key insight is that cloud data systems can achieve substantially better performance if their storage layer is rich enough to support modern

data storage optimizations that are at the heart of high performance data analytics systems, including indexing, flexible multi-dimensional partitioning, replication, and an ability to adapt to the workloads that run on them. To achieve this, the cloud table storage layer must collect detailed metadata about both the data stored and queries executed against it, and leverage this knowledge to optimize data layouts adaptively.

Systems like Databricks Delta Lake [3], Apache Iceberg [2], Apache Hudi [1], and S3 Tables [9] take a step toward this vision by providing high-performance ACID table storage over cloud object stores. These systems implement transactional capabilities by maintaining a write-ahead log directly within the object store. They also support basic data layouts, such as range partitioning and multi-dimensional Z-order sorting. They store partition metadata alongside data and transaction logs to enable efficient skipping of irrelevant data during query processing. However, users of these systems must manually specify the sort order and partitioning schemes; the systems themselves lack the ability to automatically tune these layouts based on observed data and query distributions. Furthermore, they do not collect sufficient statistics or metadata to enable more complex data layouts.

The effectiveness of data layouts depends on what metadata is stored, how the data is laid out, and how well it aligns with the query patterns. Recent studies have shown that *instance-optimized* data layouts [95, 31, 32] can achieve higher performance over basic layouts like range partitioning and Z-order by collecting richer metadata, using more expressive partitioning schemes, and leveraging data and workload-specific information. Consider dashboards that analyze New York City’s taxi trip data. Analysis patterns differ significantly between taxi types. For green taxis (`taxi_type = ‘Green’`), predicates commonly filter on the borough where the passenger was picked up (e.g., `pickup_borough = ‘Queens’`). For yellow taxis (`taxi_type = ‘Yellow’`), analysis frequently relies on predicates specifying pickup locations at major airports, such as `pickup_location = ‘LaGuardia’` or `pickup_location = ‘JFK’`. Given these distinct and repetitive query patterns, it is valuable to organize data into a complex layout. Specifically, layouts that partition data based on these predicates and maintains rich metadata, such as indexes tracking which partitions satisfy each predicate, can enable efficient data access and significantly accelerate query performance for common dashboard analyses.

While these instance-optimized layouts offer substantial performance improvements, real-world data and query patterns frequently change over time. Data is continually updated, appended, and modified, and workloads evolve as users’ interests and business requirements shift. Thus, a layout that is initially optimal may degrade in performance as conditions change. To remain effective, instance-optimized layouts must continually adapt, incrementally reorganizing their structure as new data arrives and queries evolve. Yet, none of the proposed instance-optimized layout techniques support evolving layouts over time. This lack of adaptability significantly limits their applicability in long-running, real-world cloud data systems. Consider the scenario with New York City taxi data when electric cabs are introduced. New records will include entries marked `vehicle_type = ‘electric’`, and analytical workloads shift towards queries with predicates explicitly filtering on electric cabs. Without updating the layout, these queries would scan large amounts of irrelevant data, severely impacting query performance. Thus, the data layout should be reorganized by creating partitions based on this new filter predicate. It may be expensive and unnecessary to reorganize the entire dataset. Ideally, only the newly inserted data and partitions affected

by recent queries should be incrementally reorganized, ensuring efficient adaptation without incurring prohibitive overhead.

To this end, this thesis investigates techniques to optimize data layouts for cloud table storage. First, we introduce **Pando**, a correlation-aware storage technique that exploits richer metadata about predicate correlations to enable significantly better data skipping compared to prior methods. Next, we present **CopyRight**, a partial replication framework that selectively replicates different subsets of data, optimizing each replica separately to efficiently handle heterogeneous query access patterns. Finally, we propose Self-Organizing Data Containers (SDCs), a practical, cloud-native storage layer that supports complex data layouts and incrementally adapts them in response to evolving data and query workloads. The remainder of this thesis expands upon these ideas and, in particular, makes the following three core contributions.

1.1 Correlation-Aware Multi-Dimensional Data Layouts

Recent studies on instance-optimized layouts such as Qd-trees [95] and MTO [31] have shown that more expressive partitioning schemes can achieve higher performance by leveraging data and workload-specific information. These methods tailor the data layout for a specific application by hierarchically partitioning the physical data space using filter expressions that appear in the query workload. However, even these methods fail to exploit a common workload pattern: *correlation among query predicates*. Modern applications generate queries that involve complex predicates that are often correlated with each other, i.e., the predicates are satisfied by a similar set of tuples [46, 52]. For example, queries with filters `DISTANCE(start_point, end_point) < 3 miles` and `cost < $7` on an NYC Taxi dataset will select similar tuples. Ideally, a storage system would exploit this property by creating blocks that can supply tuples for either predicate. However, existing systems create blocks that reflect each predicate independently; for example, a split in the hierarchical partitioning based on cost is not useful for queries with filters on `DISTANCE(start_point, end_point)` and thereby forces correlated queries to touch more blocks during query processing.

We present **Pando**¹ [85], a metadata-rich correlation-aware storage system for aggressively skipping data blocks. **Pando** is able to deliver substantially better block skipping than past methods by informing physical layout decisions with correlation-aware logical partitioning. **Pando** takes advantage of correlated predicates by creating multiple logical partitionings to minimize I/O for a variety of queries. In the NYC taxi dataset, we can create one logical partitioning that splits the data space based on `cost` attribute and one that splits based on `DISTANCE(start_point, end_point)`. Each logical partition maps to physical data blocks that intersect it. To ensure good performance, each logical partition should map to only a few data blocks, i.e., the physical data layout has to align well with the logical partition boundaries. **Pando** optimizes the physical data layout to maximize the utility of these logical indexes while handling outlier tuples in separate outlier partitions. For example, the high-cost / low-distance and low-cost / high-distance rides might be stored together in an outlier partition. We discuss **Pando** in more detail in Chapter 2.

¹**Pando** is a huge grove of quaking Aspen that share a single root system and are considered one organism; In Latin, **Pando** means *I Spread*.

1.2 Partial Replication for Heterogeneous Access Patterns

Complex real-world workloads contain queries with varying access patterns including queries that access different attributes, have different selectivity characteristics, and read different subsets of records. For example, dashboard-based interactive data analytics platforms allow users to create query templates with a variety of predicates by zooming in on maps, enforcing ranges on graphs, selecting from drop-down menus etc. Users analyzing a dataset of New York City taxi trips ² issue queries across a variety of dimensions to find the average fare amount of trips between two parts of New York City where the passengers paid by cash, and to find the number of trips in a particular month of the year that costed more than \$5, etc. Existing layout tuning algorithms optimize for the workload on average. However, a single layout may not be optimal for queries of different types. As a result, the benefit of these specialized data layouts decreases as queries with different access patterns scan the same data.

To address these limitations, we built **CopyRight** [84], a novel partial replicated layout engine that automatically optimizes the layout for a particular dataset and query workload. **CopyRight** replicates different parts of the data at varying degrees and organizes each replica under different layouts to maximize the overall query performance. Our layout engine supports sub-table replication where only a subset of columns are materialized in each replica. Under storage constraints, **CopyRight** achieves high performance by focusing replication efforts on parts of the data where it matters. A table-level replication scheme may waste storage by replicating areas that give minimal advantage such as cold parts of the data that is rarely accessed or parts of the data where all queries have similar data access patterns. **CopyRight** identifies sub-spaces of data that yield large benefit from replication and allocates available storage accordingly to maximize the overall gain. By materializing only a subset of columns in each replica, **CopyRight** is able to better utilize the available space and create more replicas. We discuss **CopyRight** in more detail in Chapter 3

1.3 Self-Organizing Data Containers

Existing instance-optimized layouts, despite their performance benefits, do not support automatic reorganization in response to evolving data and workload patterns. In practice, datasets frequently evolve through inserts, updates, and deletions, and workloads shift as user requirements and business objectives change. The performance of static layouts inevitably degrades over time. To address this gap, we built Self-Organizing Data Containers (SDCs) [56], a log-structured table storage layer for cloud object stores. SDCs can be used as the storage layer for applications such as database management systems, machine learning pipelines, and data science workflows. SDCs is implemented as a library that supports insert, update, and delete operations, while transparently adapting its data layout as the underlying data and workload distributions change. Layout reorganization is incremental, targeting only subsets of data impacted by changes in queries or data distribution.

We implemented SDCs on top of Delta Lake, adding support for more sophisticated

²<https://www.omnisci.com/demos/taxis>

multi-dimensional layouts, specifically **Pando**. Delta Lake exposes an **OPTIMIZE** method for compacting small files and sorting data based on user-specified columns. We extend this method to compact and organize data according to a **Pando** layout, automatically determined based on data and workload distributions. We further enhanced this to incrementally reorganize data as these distributions shift. This extended **OPTIMIZE** method can be invoked based on downstream application requirements, such as periodic invocations, invocation during downtime, or integration into frameworks for automatic compaction of log-structured tables, such as AutoComp [37]. We discuss this in detail in Chapter 4.

The rest of this thesis is organized as follows. In Chapter 2 and Chapter 3, we present two metadata-rich data layouts: **Pando**, which exploits correlations among query predicates, and **CopyRight**, which uses partial replication to improve data skipping. In Chapter 4, we introduce Self-Organizing Data Containers (SDCs), a cloud-native table storage layer that builds on **Pando** to adapt layouts based on evolving data and workloads. Finally, in Chapter 5, we summarize our contributions and outline future directions for improving and extending cloud table storage systems.

Chapter 2

Pando: Enhanced Data Skipping with Logical Data Partitioning

2.1 Introduction

With increasing volumes of data, reducing the amount of data that is accessed and processed is key to achieving high performance and reducing resource consumption in data-intensive systems. Modern data warehouses use a variety of techniques like horizontal and vertical partitioning, clustered indexes, sort orders, etc. to avoid scanning unnecessary data during query processing [73, 24, 10, 21].

Typically, cloud-based database systems organize data as compressed columnar blocks in storage, each block containing thousands or millions of tuples [50]. A block is the smallest unit of I/O, i.e., a block is either skipped or scanned entirely at query time. I/O required for accessing data blocks from storage is one of the dominant costs for query processing in these systems. To minimize the amount I/O performed, DBMSs maintain metadata such as block indexes, per-block statistics, etc. along with the data to skip accessing irrelevant blocks during query execution.

The effectiveness of data skipping depends on what metadata is stored, how the data is laid out, and how well it aligns with the query patterns. Range partitioning that is used by most production systems is useful for skipping blocks for queries that filter on the partitioned columns, but does not collect enough metadata to provide any benefits for queries that filter on other columns. Commercial systems maintain per-block metadata such as min-max values of each column to skip blocks when the range of values in it does not intersect the query filter [23, 67, 27, 58]. But their effectiveness depends on the tightness of each block’s min-max range which depends on how tuples are assigned to blocks.

Recent studies such as Qd-trees [95] and MTO [31] have shown that more expressive partitioning schemes can achieve higher performance by leveraging data and workload-specific information. These methods tailor the data layout for a specific application by hierarchically partitioning the physical data space using filter expressions that appear in the query workload. However, even these methods fail to exploit a common workload pattern: *correlation among query predicates*. Modern applications generate queries that involve complex predicates that are often correlated with each other, i.e., the predicates are satisfied by a similar set of tuples

[46, 52]. For example, queries with filters `DISTANCE(start_point, end_point) < 3 miles` and `cost < $7` on an NYC Taxi dataset will select similar tuples. Ideally, a storage system would exploit this property by creating blocks that can supply tuples for either predicate. However, existing systems create blocks that reflect each predicate independently; for example, a split in the hierarchical partitioning based on `cost` is not useful for queries with filters on `DISTANCE(start_point, end_point)` and thereby forces correlated queries to touch more blocks during query processing.

In this chapter, we present **Pando**, a metadata-rich correlation-aware storage system for aggressively skipping data blocks. **Pando** is able to deliver substantially better block skipping than past methods by informing physical layout decisions with correlation-aware logical partitioning. **Pando** takes advantage of correlated predicates by creating multiple logical partitionings to minimize I/O for a variety of queries. In the NYC taxi dataset, we can create one logical partitioning that splits the data space based on `cost` attribute and one that splits based on `DISTANCE(start_point, end_point)`. Each logical partition maps to physical data blocks that intersect it. To ensure good performance, each logical partition should map to only a few data blocks, i.e., the physical data layout has to align well with the logical partition boundaries. **Pando** optimizes the physical data layout to maximize the utility of these logical indexes while handling outlier tuples in separate outlier partitions. For example, the high-cost / low-distance and low-cost / high-distance rides might be stored together in an outlier partition. This layout is shown in Figure 2.1.

Pando co-optimizes a set of correlation-aware logical partitionings and the physical layout of the data to maximize the overall performance of a given workload on a dataset, allowing it to specialize to data and workload distributions. **Pando** maintains a collection of logical partitionings of the data, each arranged as a tree that hierarchically splits the logical data space using predicate expressions that are useful for skipping blocks during query processing. The leaves of these trees point to the physical blocks that contain the logical ranges in the tree nodes. At execution time, these expressions are intersected with the query filters to skip blocks that do not intersect with the query. **Pando** chooses the underlying physical blocks in such a way to maximize the opportunity for block skipping, exploiting correlations between expressions to find good physical designs.

Decoupling the physical data layout and logical partitioning trees creates an extremely large search space of configurations that need to be considered during optimization. Data can be partitioned into blocks in a large number of ways (the so-called *Bell number*), which grows faster than exponential in the number of tuples. For any given blocking, an exponentially large number of index trees can be created, causing a combinatorial explosion in the space of physical design alternatives. We present a layout optimizer that efficiently navigates this space by leveraging properties of the data and workload.

To summarize, this chapter makes the following contributions

1. We present **Pando**, a metadata-rich correlation-aware data layout that jointly optimizes the selection of multiple logical partitioning trees and the physical data layout to enhance query performance by reducing the amount of data accessed.
2. We provide an efficient heuristic algorithm to determine a good set of partitioning trees and the assignment of tuples to blocks for a given dataset and workload.

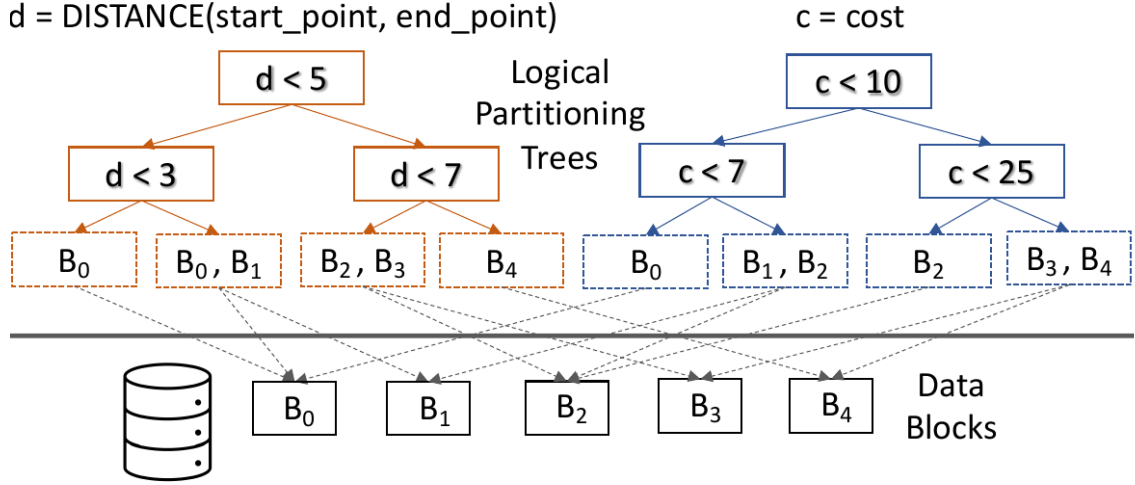


Figure 2.1: An example Pando layout for NYC Taxi dataset. B_0, B_1 , etc. are physical data blocks. There are two logical partitioning trees, one with predicates on $\text{DISTANCE}(\text{start_point}, \text{end_point})$, another on cost . Leaf nodes of the tree map to the physical blocks that intersect the logical data ranges.

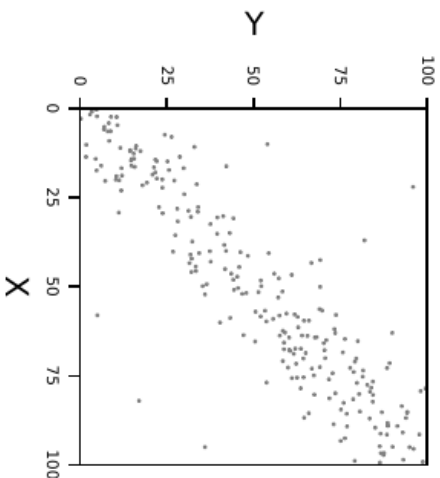
3. Finally, we demonstrate that **Pando** outperforms other layouts across a range of real-world and benchmark datasets and workloads. **Pando** is up to 2.3X faster in overall query execution time and reduces the number of blocks scanned by up to 2.8X when compared to the state-of-the-art learned data layout, MTO.

2.2 Motivation

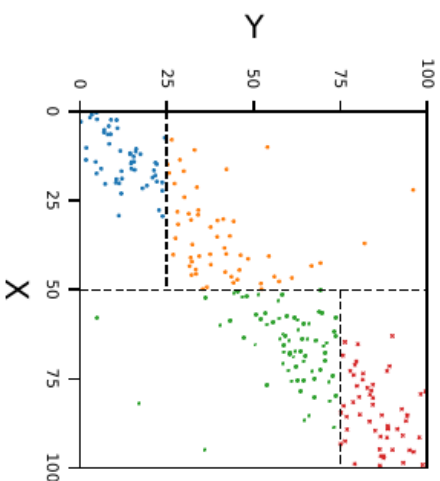
We begin with an overview of the limitations of the existing solutions and then make the case for **Pando**. Qd-tree [95] is a recently proposed learned data layout framework for single tables, that specializes its data layout to a specific dataset and the workload of queries that run on it to minimize the amount of data scanned during query processing. To optimize the layout, it extracts "simple" predicates from the workload, like $X < 50$, $X < Y$, etc. and constructs a binary decision tree from them. Data is partitioned according to this tree. Each inner node in the tree is a predicate expression, called a *cut*, that cuts the physical data space into two child nodes. One of the child nodes corresponds to the tuples that satisfy the predicate expression and the other corresponds to the tuples that do not. The leaf nodes of the tree correspond to a data block. At execution time, a query traverses the tree to find the blocks that need to be scanned. At any given node, if the query intersects only one of the cut or its negation, only the corresponding child node is traversed. Otherwise, both children are traversed.

MTO [31] extends qd-trees to optimize the layout across multiple tables. MTO takes a set of tables as the input and produces one qd-tree layout per table. In addition to single table predicates in qd-trees, MTO uses join-induced predicates to cut the nodes in the partition tree. For example, if the workload contains queries like

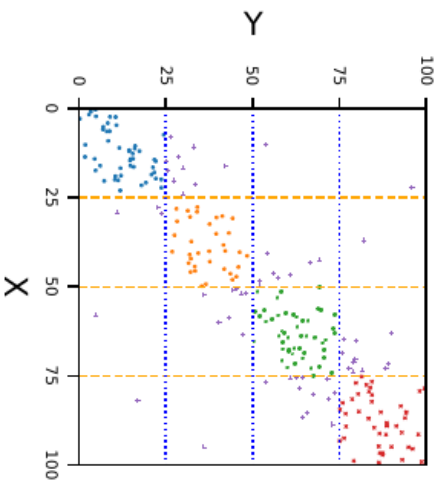
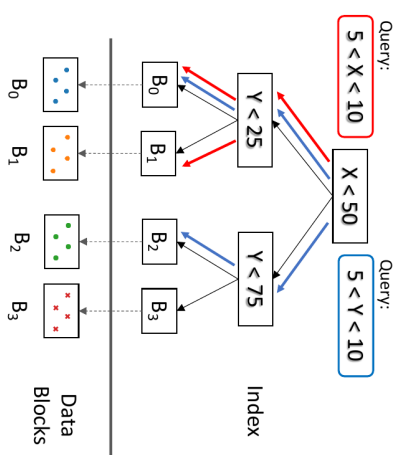
```
SELECT * FROM T1 JOIN T2 on T1.Key = T2.Key
```



(a) Dataset



(b) Qd-tree / MTO



(c) Pando

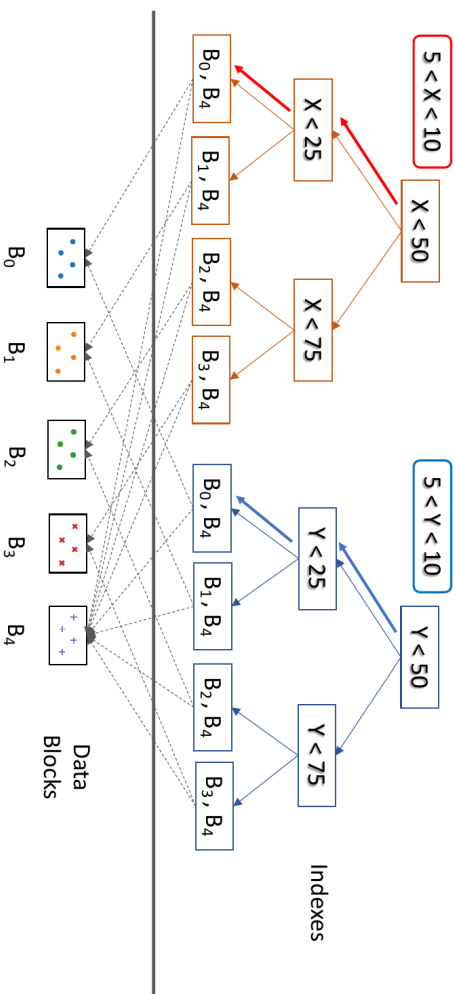


Figure 2.2: Figure 2.2a shows a 2-dimensional dataset with correlated columns X and Y . Consider a workload with 2 kinds of queries. Half the queries have a range predicate on column X and the other half have a range predicate on Y . Figure 2.2b shows the layout that qd-tree and MTO will find. Nodes traversed during execution of queries are marked along the edges of the tree. Each query scans the blocks in the leaf nodes that intersect the query. Figure 2.2c shows the Pando layout with two logical partitioning trees and data blocks that align well with the logical trees and query access patterns.

WHERE T1.X < 50 AND T2.Y > 75

the predicate `T1.Key IN (SELECT T2.Key FROM T2 WHERE T2.Y > 75)` is considered for cutting T1’s partitioning tree and `T2.Key IN (SELECT T1.Key FROM T1 WHERE T1.X < 50)` for cutting T2’s data space. These cuts are useful for pre-filtering tuples that would otherwise be dropped during the join operation.

Both MTO and qd-tree construct a single tree per table which is both a partition function and an index for skipping blocks at query time. Most cloud systems have a large minimum block size (millions of tuples) to hide the latency of accessing cloud storage and to achieve high compression ratios [50]. As each leaf node of this tree corresponds to a block with a minimum size, the height of the tree is limited, so just having a single tree of expressions limits the number of expressions that can be present in the tree. This in turn reduces the utility of the tree for skipping data for a variety of queries. This can be particularly bad when there are correlated expressions, i.e., expressions over different fields that are satisfied by substantially overlapping sets of tuples.

To illustrate where this overhead comes from, consider a toy example with a single two-dimensional table visualized in Figure 2.2a. The X and Y axes correspond to two correlated columns in the table (e.g., cost and distance). The grey dots represent the tuples in the table. Consider a workload with 2 kinds of queries uniformly distributed in the data space. Half the queries have a range predicate on column X and the other half have a range predicate on Y that select a narrow range of data. The table has 500 tuples and a minimum block size of 100 tuples.

Figure 2.2b shows the layout for the dataset that qd-tree/MTO would find for the given workload. The root node of the expression tree has the predicate $X < 50$ and the two inner nodes have predicates on Y. The tuples that satisfy $X < 50$ AND $Y < 25$ are in block B_0 , the tuples that satisfy $X < 50$ AND NOT($Y < 25$) are in B_1 , and so on. The black dotted lines in the grid represent the physical partition boundaries. When a query with filter $5 < Y < 10$ arrives, we first intersect it with the filter in the root node expression $X < 50$. As they are not comparable, the query may intersect both child nodes. The query intersects with the left side of both children, so we only scan the blocks B_0 and B_2 , scanning half the data in the table. Figure 2.2b highlights the nodes traversed during query execution using this tree. Most queries in the workload will also scan half of the table. Queries with a filter on X can use the root node to skip half the data, and those with a predicate on Y can use the child nodes with an expression on Y.

An issue with the qd-tree is that correlated expressions $X < 25$ and $Y < 25$ cannot appear on a path from the root node to a leaf. If they do, then the child node corresponding $X < 25$ AND NOT($Y < 25$) will have very few tuples as the columns are correlated. But if we can create two different trees, then we can create one tree with predicates on X and one on Y as shown in Figure 2.2c. One of the trees can be used for skipping queries with filters on X, and the other for queries that filter on Y.

A qd-tree is not only an index but also a partitioning function. Each leaf node maps to one physical data block with precisely the tuples that satisfy the leaf node’s expression. But with two index trees, it is unclear what the data blocks are. If both trees have to be partitioning functions, then they have to be perfectly correlated i.e., both trees have to partition the tuples into the exact same blocks. This restricts the space of possible trees

significantly as most datasets have outliers of some form.

If, as we do in **Pando**, we relax the precision of the leaf node expressions and allow them to point to multiple blocks containing a superset of tuples that satisfy the expressions, we can lay out the data to align well with the two trees as shown in Figure 2.2c.

Here, each tree is a logical partitioning of the data space. The orange dotted line in Figure 2.2c corresponds to the boundaries of the orange partitioning tree with predicates on X . Similarly, the blue dotted lines show the logical partition boundaries of the second partitioning tree with predicates on Y . Tuples with different colors/shapes represent the physical data blocks. The leaf nodes of the trees map to blocks that intersect its logical subspace. The leaf node of the orange tree corresponding to the expression $X < 25$ points to blocks B_0 and B_4 as the blue/purple tuples intersect the subspace of data corresponding to $X < 25$.

Each leaf node in Figure 2.2c points to $2/5^{th}$ of the data, more tuples than earlier. But with more expressions stored in the trees, queries scan fewer leaf nodes. The query $5 < Y < 10$ can use the second partitioning tree and only intersects the first leaf node that maps to B_0 and B_4 . In our workload, most queries intersect only one leaf node and scan only $2/5^{th}$ of the data compared to half the data when using a qd-tree. Intuitively, the layout in Figure 2.2c reduces the amount of data scanned by splitting correlated expressions like $X < 25$ and $Y < 25$ across two logical partitioning trees, and assigning tuples to blocks to maximize the utility of the trees.

This is a toy example and a simple correlation between two columns like this can also be addressed by existing techniques like Correlation Maps [45], zone maps [36], etc. But our method goes beyond that and captures the correlation between expressions that exist in the workload. For example, in the NYC taxi dataset, the trees may contain complex predicates such as `DISTANCE(start_point, end_point) < ?` and `cost < ?` that are correlated but where `DISTANCE()` does not exist as an attribute in the database. **Pando** builds two trees for expressions on X and Y that are correlated. The X and Y axes in the Figure 2.2c correspond to the sort order of the data dictated by the two trees, and it visualizes the correlation between the expressions in the two trees. Our method can also take advantage of correlations across tables. For example, in TPC-H, predicate `l_shipdate > ?` and join-induced predicate `l_orderkey IN (SELECT o_orderkey FROM orders WHERE o_orderdate > ?)` are correlated. **Pando** leverages this to construct multiple expression trees that work in conjunction and organizes the data to maximize their effectiveness.

Even if the expressions are not correlated, storing multiple trees can perform well and in fact, save space. A single query can combine information from multiple indexes to scan fewer blocks. For example, if columns X and Y were not correlated and uniformly distributed, the layout corresponding to the 16 grid cells in Figure 2.2c can be optimal for a minimum block size of one-sixteenth of the table size. A single tree that can index the 16 blocks will need 15 expression nodes. This single tree can be visualized by appending the second partitioning tree to every leaf node of the first tree. The two trees shown in Figure 2.2c can index the 16 blocks with equal expressive power, but with just 6 expressions. A query with a predicate on both X and Y can combine information from both trees to only access the data blocks that intersect the query. We incur additional overhead in maintaining the pointers to blocks, but it is outweighed by the space benefits from fewer nodes. So we focus on creating multiple partitioning trees instead of one large tree.

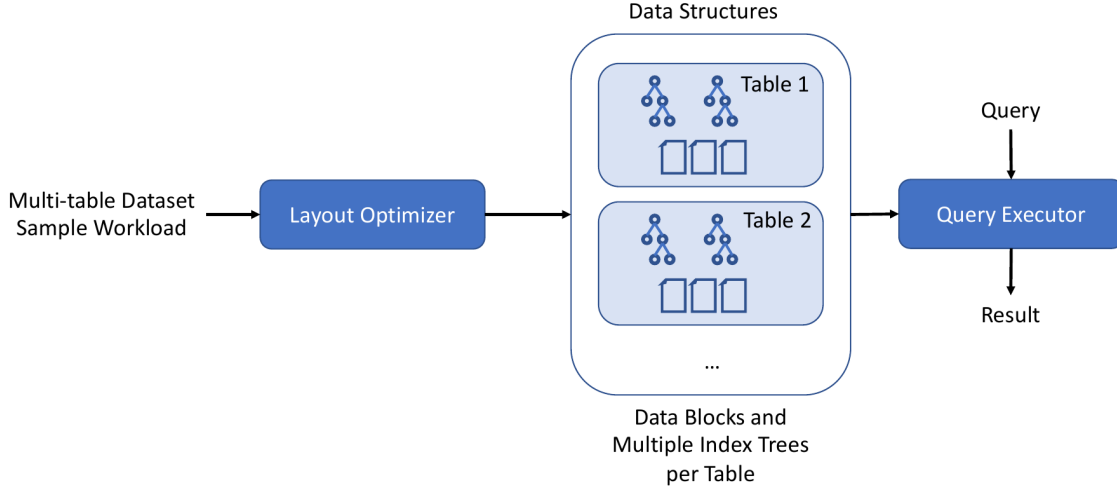


Figure 2.3: Overview of Pando architecture.

2.3 Overview

As shown in Figure 2.3, Pando is comprised of two components: a layout optimizer which determines what indexes to create and how to lay out the data, and a query executor which executes queries on this layout. Pando organizes data as blocks, disjoint sets of rows, with a minimum size in storage and builds one or more binary partitioning trees that index these blocks (Section 2.4). When a query arrives, the query executor gets the data blocks to retrieve using each partitioning tree and combine them smartly to access as few blocks as possible from storage (Section 2.5). Given a multi-table dataset, a representative workload, a minimum block size, and a number of indexes to create per table, the layout optimizer finds the data layout and associated indexes for each table to maximize overall performance (Section 2.6). Now, we describe these components in more detail.

2.4 Data Structure

Pando organizes data for each table in a database as compressed columnar blocks in storage. A block only contains tuples from one table. Blocks are mutually exclusive, i.e., tuples are not replicated across blocks. There is only one copy of the data for each table.

For each table, we create k logical partitionings. Each logical partitioning is an expression tree that indexes a table’s blocks. Each node in the tree is a predicate expression that filters that table’s tuples, called cut. For example, the cuts in table T1’s partitioning tree can contain single-table expressions like $T1.Col1 < 5$ or join-induced predicates like $T1.Key \text{ IN } (SELECT T2.Key \text{ from } T2 \text{ WHERE } T2.Z > 2)$. The former is useful for queries that have a predicate on $T1.Col1$. The latter is useful for skipping blocks for queries that join T1 with T2 and have a filter on $T2.Z$.

Unlike qd-trees that cut the physical data space into blocks, Pando’s partitioning trees cut the logical data space. The left child of the node corresponds to the logical data subspace

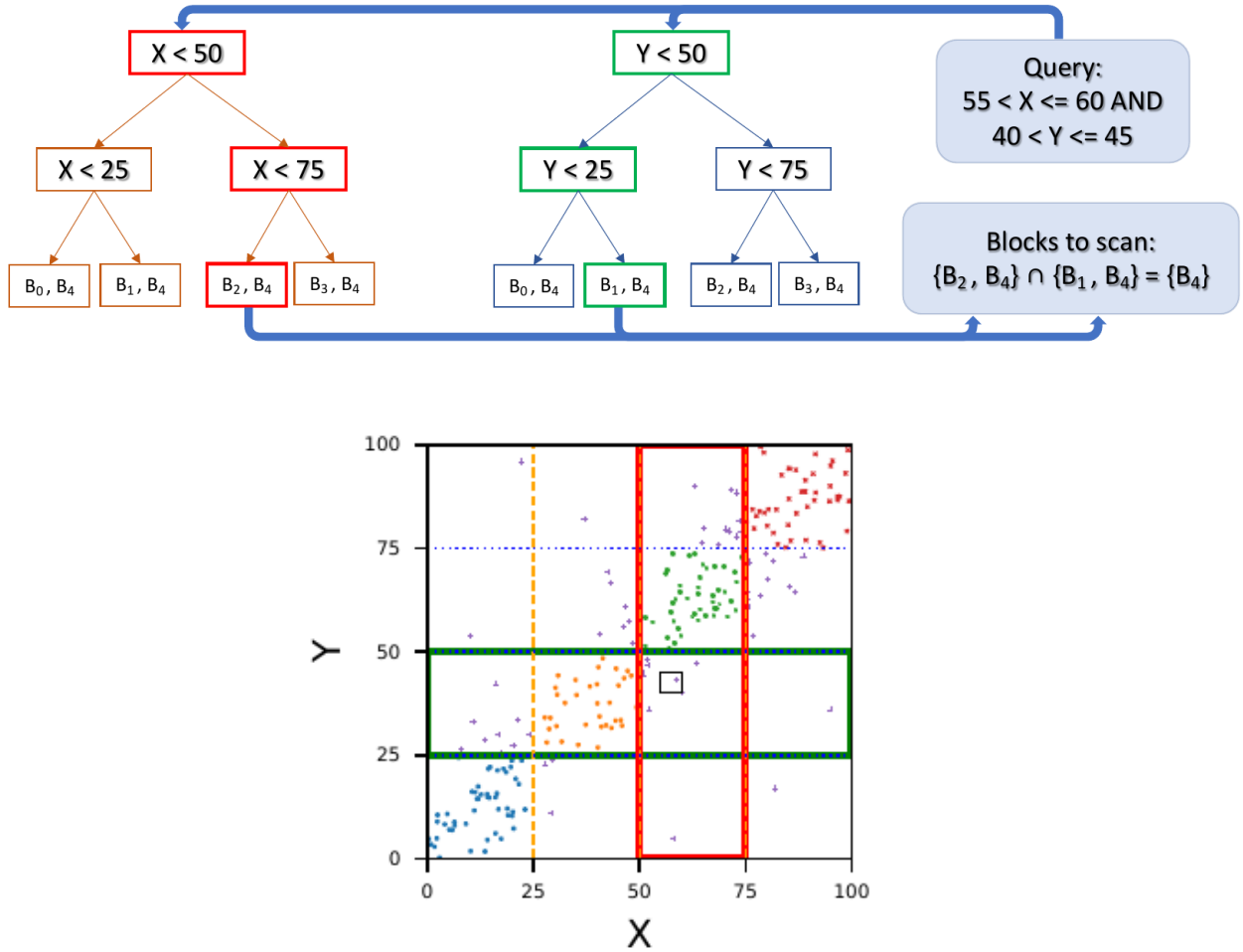


Figure 2.4: Query execution in Pando for the layout in Figure 2.2

that satisfies the expression, and the right child corresponds to the subspace that do not satisfy the predicate. For example, the predicate $T1.Key \text{ IN } (SELECT T2.Key \text{ from } T2 \text{ WHERE } T2.Z > 2)$ splits table $T1$'s data space into two subspaces: tuples that when joined with $T2$ on Key will satisfy $T2.Z > 2$ and tuples that will not find a match during join or when joined will not satisfy $T2.Z > 2$.

Each tree partitions the logical data space into mutually exclusive subspaces. Each leaf node corresponds to a predicate based on a conjunction of the expressions (or their negations) in the tree as we traverse the tree from root to leaf. For the rest of the chapter, we call this *leaf node expression*. In the first partition tree in Figure 2.2c, the first leaf node from left (the one that points blocks B_0 , and B_4) corresponds to the expression $X < 50 \text{ AND } X < 25$. Similarly, the second one has $X < 50 \text{ AND NOT}(X < 25)$.

Each leaf node contains a set of pointers to the physical blocks that intersect with this logical space. In other words, the leaf node maintains a list of blocks with at least one tuple satisfying the leaf node expression. Note that each leaf node expression is complete, i.e., it points to every block that contains a tuple that satisfies the leaf node expression. However, it is not precise, as there may be tuples in some of the pointer blocks that do not satisfy the expression.

2.5 Query Execution

When a query arrives, we leverage information from all expression trees to minimize the number of blocks scanned. Each index in **Pando** is a logical partitioning of the data space. We use each index to identify the leaf nodes of the tree that needs to be scanned to answer the query. We traverse down each expression tree to find the leaf nodes that intersect the query filters. At each node, if the query does not intersect the cut in the node, we skip the left child. If it does not satisfy the negation of the cut, we skip the right child. For e.g., consider a node with cut $X < 50$. For query $X > 70$, we skip scanning the left child. Similarly, for the query $X < 10$, we skip the right child. For queries like $X < 60$ that intersect both the cut and its negation, or for queries like $Y < 10$ where intersection cannot be determined from the expressions, we scan both child nodes.

The set of leaf nodes returned by a tree corresponds to a logical data subspace that subsumes the query i.e., all tuples that satisfy the query are contained in this space. The physical blocks pointed to by these leaf nodes are a superset of the logical space represented by the expressions. Scanning these blocks is sufficient to answer the queries. We obtain the set of blocks to scan from each tree independently and then intersect all the sets to get the set of blocks to retrieve from storage.

Figure 2.4 shows the execution of an example query with filter $55 < X \leq 60$ AND $40 < Y \leq 45$. The black rectangle in the grid visualization corresponds to the subspace of data that corresponds to the query filter. The two indexes are used to identify the set of leaf nodes that intersect the query filter. The path traversed is highlighted in each tree. The index on X finds the leaf node corresponding to $50 < X \leq 75$, and the index on Y finds the leaf node corresponding to $25 < Y \leq 50$. The logical subspace of data that satisfies these leaf node expressions is highlighted by the red/green rectangle in the grid that subsumes the black query rectangle. The leaf node $50 < X \leq 75$ maps to physical data blocks B_2 and B_4 which correspond to all green and purple tuples. These blocks subsume the red rectangle in the grid which subsumes the query. Similarly, blocks B_1 and B_4 subsume the query. As physical data blocks are mutually exclusive, only the blocks that are in the intersection of the two sets need to be retrieved. So **Pando** only retrieves block B_4 to answer the query.

2.6 Layout Optimization

The input to our layout optimizer is a multi-table dataset D , a representative workload W , a minimum block size b , and a number of expression trees to create per tree k . The output is a blocked layout for each table such that each block has greater than b tuples and k expression trees per table to index the blocks in the table. We note that our algorithm works for different values of k for each table. For simplicity of presentation, we use one k for all tables in the database. Our algorithm works in two phases.

Top-Down tree construction: We first extract relevant single-table and join-induced predicates from the representative workload and use them to construct k trees per table in a top-down fashion. Trees are constructed to capture different expressions that are useful for accelerating queries in the workload. As we will describe below, correlated expressions that exist in the workload are captured in different trees if they are useful for skipping queries.

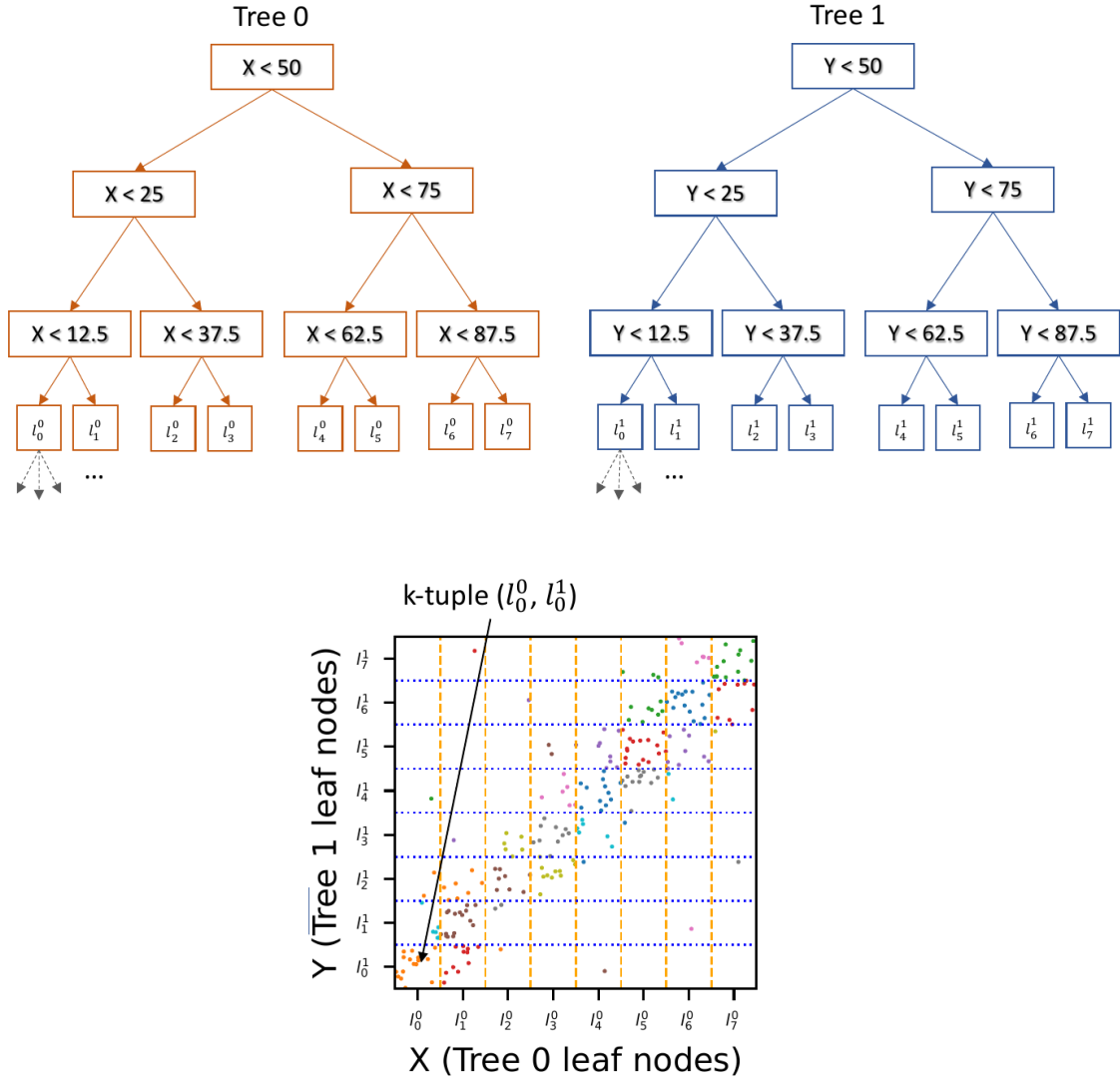


Figure 2.5: The trees constructed during the top-down phase of Pando's layout optimizer. The grid visualizes the correlation between the leaf nodes of the trees.

Bottom-Up blocking assignment: We then group the tuples in a bottom-up fashion to maximize the utility of these indexes until each block has at least b tuples. This bottom-up clustering of tuples is done to minimize the I/O cost of the queries when executed using the indexes constructed in the previous phase. The expression trees created in phase 1 are updated if required.

We now discuss each step in detail.

2.6.1 Top-Down Tree Construction

The goal of this stage is to construct k partitioning trees per table, each with predicates that are useful for skipping blocks for queries in the workload. The top-down stage works as follows:

1. Extract useful predicates from the workload and create a set of *candidate cuts* for each table.

2. For each table independently: construct k partitioning trees using the cuts from its candidate set to minimize the number of tuples scanned from the table.

First, we extract candidate expressions from the workload for cutting the data space in the tree nodes. For each query in the workload, we extract predicates on each table and decompose them into simple expressions without any conjunctions or disjunctions and add them to a set of candidate cuts for that table. For example, consider the query: `SELECT ... FROM T1 JOIN T2 on T1.Key = T2.Key WHERE T1.X < 100 AND T1.Y > 72 AND T2.Z > 2`. Single-table candidate cuts extracted for table T1 are `T1.X < 100`, and `T1.Y > 72`. Similarly, `T2.Z > 2` is added to the candidate cuts set for table T2. These single table predicates are then propagated through the join graph to create join-induced candidate cuts. The predicate on T2, `T2.Z > 2` is propagated to table T1 through the join `T1.Key = T2.Key` to create the join-induced candidate `T1.Key IN (SELECT T2.Key from T2 WHERE T2.Z > 2)`. This is added to T1's set of candidate cuts. Similarly, `T2.Key IN (SELECT T1.Key from T1 WHERE T1.X < 100)` and `T2.Key IN (SELECT T1.Key from T1 WHERE T1.Y > 72)` are added to T2's candidate cuts. We then construct the k trees independently for each table using expressions from its candidate set.

We give a top-down greedy algorithm, which extends the algorithm for the construction of trees from qd-tree and MTO, to construct k trees, one cut at a time. Throughout the first stage of the algorithm, we assume that each tuple is in a separate block. The iterative algorithm works as follows:

1. Initialize k trees, each with exactly one node that does not have any cuts and points to every block in the table.
2. For each tree, for each leaf node, for each candidate cut, find the cost of executing the workload if the leaf node is cut using the candidate.
3. Let (t, l, c) be the tree, leaf node, and candidate cut triple with the lowest cost in step 2. Cut the leaf node l of the tree t using the cut c .
4. Repeat steps 2 and 3 until one of the following conditions is satisfied:
 - (a) Cutting leaf nodes no longer yield any more benefit.
 - (b) Cutting leaf nodes result in child nodes that point to fewer than a threshold number of tuples.

We start with k trees with no cuts in any of the trees, i.e., the root node of each tree is also a leaf node. Each leaf node corresponds to the entire logical dataspace and maps to every block in the table (each block is a tuple).

In each iteration of the algorithm, we pick a leaf node of one of the k trees and a cut from the candidate cuts to split the leaf node into two child nodes. This choice is made greedily by picking the tree/leaf node/cut combination that provides the maximum I/O benefit for the workload. We consider a cut only if the child nodes after the node is cut have greater than a threshold number of tuples. The threshold number of tuples can be varied to trade off optimization time for higher-quality layouts (discussed in Section 2.6.2). The child

nodes become new leaf nodes that are considered for cutting in later iterations. We continue splitting leaf nodes until no more nodes can be cut or cutting nodes yields no benefit.

The cost model in step 2 is the I/O cost which is simply the total number of blocks scanned (the same as the number of tuples as each tuple is in a separate block) for all queries in the workload. The I/O benefit is estimated using the query execution described in Section 2.5 i.e., for each query we only scan the blocks that are returned from all the trees. A cut c is useful for cutting a leaf node l in tree t only if queries can skip blocks using c that they could not already skip using predicates in other trees. This means that the benefit of a cut in one tree depends on the cuts in other trees. Our cost estimation accounts for using multiple trees to skip blocks for each query so that we do not end up with k identical trees. Using these trees can reduce the cardinality of the scan output (eg. join-induced cuts pre-filter tuples that would otherwise be dropped during join), thus can have an impact on query plan (eg. join order). Our cost model does not account for that and only models the I/O cost.

This algorithm picks expressions that are useful for minimizing the I/O needed for the workload when the indexes are used in conjunction. The expressions that yield a higher reduction in the amount of data scanned for most queries appear at the higher levels of the tree. For example in our running example, the predicates $X < 50$ and $Y < 50$ can reduce the number of records scanned by half for queries that filter in X and Y respectively, so will be picked first.

We note that this algorithm tends to pick correlated expressions in different trees, and not in the same tree. To understand why correlated expressions tend to be picked in different trees, consider $Y < 25$ being considered as a candidate to cut the left child node $X < 25$. The benefit of this cut for a query that does not intersect the negation of the predicate is bounded by the number of tuples that satisfy $X < 25$ AND NOT($Y < 25$) which is a very small number. Whereas in another tree, where the path to the leaf node has not been cut by $X < 25$, the benefit is likely higher. Our algorithm will pick such a leaf node if it is beneficial for the overall cost of the optimization process.

Now we discuss, how to scale this phase of the optimization algorithm for large datasets. Our cost model is the total amount of data that is scanned by the queries in the workload when k trees are used in conjunction. This requires finding the tuples that satisfy the leaf node expression for each candidate cut considered which can be very expensive. To reduce the runtime of the optimization algorithm, we run this stage using a sample of the dataset for estimating the cost, similar to qd-tree and MTO. This is done for all the tree, leaf node, and candidate cut triplets considered in step 2 in parallel.

At the end of this stage, we have constructed k trees for each table that cut the logical data space using a variety of expressions that are useful for avoiding unnecessary data access during query execution. The cuts that yield higher benefits or are useful for a large number of queries tend to be in the higher levels of the trees and correlated expressions if useful for block skipping tend to be in different trees. Figure 2.5 shows the trees that are constructed by the algorithm with a threshold size of one-eighth of the table. We use l_j^i to denote the j^{th} leaf node of the i^{th} tree. We now discuss how we find the block assignment to maximize the benefit of these trees.

2.6.2 Bottom-Up Block Assignment

The goal of the second stage of the algorithm is to assign tuples to blocks that maximize the utility of the indexes in skipping blocks for the given workload. Each tree partitions the logical data space into mutually exclusive subspaces, so each tuple satisfies the leaf node expression for exactly one leaf node in a given tree. Logically, each tuple has an associated k-tuple whose i^{th} entry is the id of the leaf node of the i^{th} tree that the tuple belongs to. The grid in Figure 2.5 illustrates the mappings between tuples and leaf nodes for the two trees constructed in the top-down phase of our algorithm. The two axes in the visualization correspond to the leaf nodes of the two trees, i.e., each partition along each axis corresponds to a leaf node in a tree. Each cell in the grid formed by these partitions corresponds to a k-tuple. In our running example, consider the tuple $(X = 5, Y = 5)$ in our running example. When routed down the first partitioning tree with predicates on X, this lands in the leaf node l_0^0 . Similarly, it lands in the leaf node l_0^1 in the second tree. So this tuple has an associated k-tuple (l_0^0, l_0^1) .

The goal of our bottom-up clustering algorithm is to merge the tuples into blocks such that queries when executed using the indexes created in the first phase scan as few blocks as possible. To minimize the number of blocks, we need to group tuples that are accessed together in the same block. In other words, tuples should be assigned to blocks such that queries that need to scan one tuple in the block also need most other tuples in the block. Outliers are those tuples that hinder this, i.e., they are needed for answering queries that scan very different sets of tuples. So however you assign these outlier tuples to blocks, it is likely that many queries that need to scan the outlier tuples do not require other tuples in the block. As trees in the first phase were constructed to align the leaf nodes well with the queries assuming each tuple is in a separate block, it is likely these outlier tuples are also outliers in the leaf-node expression space that is visualized in the grid Figure 2.5.

In our example, nearby tuples along the diagonal $X=Y$ are good candidates to be in the same block as most queries of both templates (the ones with a predicate on X, and ones with a predicate on Y) that intersect them scan many of the nearby tuples. In contrast, a tuple far away from the diagonal, such as $X=15, Y=100$, is an outlier. Queries with a predicate on X that need this tuple mostly scan the bottom left corner of the grid, whereas queries with a predicate on Y that need this tuple scan the top right corner of the grid. So if you group the outlier tuple with ones in the bottom left, then queries on Y will scan an unnecessary amount of data. Similarly, if you group it with tuples in the top right corner, then queries on X will scan irrelevant tuples. So to maximize skipping, we should minimize the number of blocks with outliers by assigning the outlier tuples to separate outlier blocks.

We now describe our bottom-up algorithm that clusters tuples to minimize the total amount of data scanned using these trees.

1. Route the tuples down each partitioning tree to find the associated k-tuple.
2. Merge all tuples with the same k-tuple into a block.
3. For each pair of blocks, find the cost of merging them.
4. Merge the pair with the lowest cost from step 3.

5. Repeat steps 3 and 4 until each block has greater than the required minimum block size b .

All tuples that have the same k-tuple cannot be differentiated by the trees that we have constructed, so there is no benefit in spreading the tuples across blocks. So all the tuples that satisfy the same k-tuple are merged into the same block. In Figure 2.5, each k-tuple corresponds to a cell in the grid. Thus, all tuples that fall in the same grid cell are merged into a block.

We then iteratively merge these cells until each block has more tuples than the minimum block size b . We start with each block having just one k-tuple, corresponding to one cell in the above grid. In each iteration, we pick two blocks to be merged into one. We make this choice greedily by merging two blocks that incur the least cost for executing the queries. We continue merging blocks until each block has more tuples than the minimum number of tuples in a block, b .

To compute the cost of merging two blocks, we use a cost model that is the total number of records that are scanned according to query execution described in Section 2.5. For each query, we get the set of blocks to scan from each partitioning tree and scan the ones that are returned by all trees. When we merge two blocks B_i and B_j , all queries that scanned block B_i or B_j will now scan the data that is in both blocks. It is also possible that some queries that did not access either of the blocks will now scan the merged block. To see this, suppose that, before merging, the blocks to be scanned returned for a query Q in tree 1 are (B_i, B_k) , and in tree 2 are (B_j, B_k) . The query would only scan B_k as that is the only block that is in the intersection of sets returned by the two trees. After merging, both trees now return (B_{ij}, B_k) where B_{ij} is the merged block, so both blocks will be scanned during query execution.

Intuitively, two blocks are merged if most queries that scan one block also scan the other. In the running example, this is true for blocks along the diagonal of the data space. The outlier tuples are scanned by queries that scan a variety of other blocks. Outliers k-tuples that are fewer in number get merged into their own block. This outlier block is unlikely to be merged with the diagonal block as that will incur high costs for most queries that scan either of the blocks. This handling of outliers is essential for finding a block assignment that is good for the indexes. We eventually end up with a block assignment shown in Figure 2.2c.

At this point, many sibling leaf nodes in the partitioning trees will point to the same set of blocks. A cut in the tree is not useful if both child nodes point to the same set of blocks. We truncate the trees by merging nodes of the tree in a bottom-up fashion, eliminating cuts if they provide no benefit. For example, both child nodes of $X < 12.5$ in Figure 2.5 will point to blocks B_0 and B_4 . The cut can be eliminated and the node corresponding to $X < 12.5$ can be made a leaf node that points to B_0 and B_4 . This is done until all sibling nodes have different block mappings, thus yielding a tree shown in Figure 2.2c.

We now discuss how to scale this phase of the algorithm. Step 3 can be done in parallel and the cost model depends on the number of tuples in each block, which can be easily maintained while merging blocks. If a table has NT tuples and each tree has NL_i leaf nodes, the number of possible values of k-tuples, NK , is $\min(NT, \prod NL_i)$ which can be huge. The complexity of this phase is quadratic in NK . Capturing correlated expressions in separate trees in the top-down phase results in many of these cells being empty, making this bottom-up

algorithm tractable. The threshold parameter we have in the first phase can be tuned to trade-off time spent in optimization for a higher quality layout. The smaller the threshold, the more the number of leaf nodes, the more the number of distinct k-tuples, and the longer it takes for merging. But smaller thresholds can capture finer-grained outliers improving the layout quality.

We note that **Pando** with $k = 1$ is the same as MTO. We can use b as the threshold for the top-down algorithm and we will create the same tree as MTO. For values of threshold lower than b , the top-down phase will pick additional predicates in the deeper levels of the tree, which will be removed during the bottom-up phase. Increasing k will improve the performance of the layout with diminishing returns. However, this comes at the cost of an increasingly larger offline optimization time. In theory, the optimal value of k can be arbitrary as workloads could contain an arbitrary number of distinct expressions that are correlated. We did not observe many such expressions in the workloads at Meta and expect small values of k would be sufficient for most workloads. Based on our experiments in Section 2.8.3, we expect 3 or 4 trees to be sufficient for many realistic datasets. To pick k , we can evaluate the query performance for increasing values of k on a dataset sample until the query performance flattens.

2.7 Data Changes and Workload Shifts

In this section, we describe how **Pando** adapts to evolving data and workload distributions. **Pando** supports both bulk and regular data changes. Most analytic data systems handle regular inserts, updates, and deletes by absorbing them in a delta store which is periodically merged with the main data store, effectively turning point modifications into batch updates [50]. Any data layout that maintains data in some sorted order, including simple schemes like single-column range partitioning will impose some overhead as a result of merging to maintain its performance advantages. **Pando** faces two types of performance challenges related to updates.

First, mapping from leaf nodes of the partitioning tree to physical blocks can become stale with data changes because of join-induced predicates. Consider a join-induced predicate `T1.Key IN (SELECT T2.Key FROM T2 WHERE T2.X > 10)` that is present in one of the partitioning trees of T1. When the tree was constructed, suppose this predicate was equivalent to the *literal cut* `T1.Key IN (1, 2, 7)`, i.e., `SELECT T2.Key FROM T2 WHERE T2.X > 10` evaluated to `(1, 2, 7)`. When a new tuple, say `(Key:10, X:15)`, is added to T2, the join-induced predicate is now equivalent to `T1.Key IN (1, 2, 7, 10)`, but the mappings in the leaf nodes still correspond to the old literal cut. When data is inserted to T2, the mapping from logical blocks to physical blocks in T1 might be stale and requires remapping. This can be expensive.

We use the same approach as MTO for handling inserts and deletes. Specifically, for the case of induced join predicates, we restrict the joins to be foreign key-primary key joins and only induce predicates from the primary key table to the foreign key table. Continuing the example, we use the cut `T1.Key IN (SELECT T2.Key FROM T2 WHERE T2.X > 10)` in T1's partitioning tree only if T2.Key is the primary key of T2 and T1.Key is a foreign key that references it. More generally, for star schemas, we induce predicates from a dimension table

to a fact table, but not from a fact table to a dimension table. Intuitively, this has minimal impact on performance as dimension tables are typically smaller than fact tables. Assuming referential integrity, data inserts or deletes do not affect the logical to physical block mapping. In our example, none of the tuples in T1 will have `Key=10`, the newly inserted unique value in T2, so the mappings in T1’s trees are still correct.

To route tuples down the partitioning tree during insertion, we need to evaluate the predicate in each node. To do this efficiently for join-induced predicates, we store the literal version of the cut in each node, as in MTO. This can potentially be very large, so we store them as compressed bitmaps. This literal cut needs to be maintained in the presence of inserts and deletes. This can be performed by evaluating the relevant join-induced predicate on only the inserted or deleted records, not the entire table. In our example, the join-induced predicate is evaluated on the newly inserted tuple in T2, (`Key:10, X:15`), to update the literal cut from `T1.Key IN (1, 2, 7)` to `T1.Key IN (1, 2, 7, 10)`. Updates can be implemented as a delete followed by an insert but may cause existing tuples to shift between blocks.

The second update-related challenge is that **Pando**’s logical partitioning trees do not give a unique block to insert the data into. To merge deleted records from the delta store, we need to locate the blocks with the deleted records; **Pando**’s logical partitioning trees can be used to find those blocks. However, if we route a tuple down k partitioning trees in **Pando**, we will get a set of possible blocks the tuple could be in. In **Pando**, each block is a collection of k -tuples. To support point (trickle) inserts, we can maintain the non-truncated trees and the mapping from k -tuples to blocks. This can be used to route tuples to blocks during insertion.

While this can work well for small in-distribution inserts, most analytics systems ingest data in large batches and contain out-of-distribution data. For example, a day’s worth of data is loaded into the table by nightly ETL jobs, and the data contains date/timestamp columns, serial ids, etc. with increasing values that are out-of-distribution (unseen during layout optimization done in the past). Each such new batch is written into a new partition, naturally resulting in a partitioning on the data/timestamp column as data is usually inserted in the order of these columns. Most queries that run on this data filter on these columns. **Pando**’s layout optimizer can be the last step of these ETL jobs, which optimizes the layout for each bulk insertion batch independently and writes the data blocks according to this layout. Intuitively, this is a two-level partitioning. At the first level, data is partitioned based on the date column, and each such partition is organized into blocks using **Pando** at the second level. Tuples that arrive out-of-order can be buffered and periodically merged with data that is already partitioned according to **Pando** as described above. This is similar to how systems like Amazon Redshift maintain data with sort keys [14]. The staleness of the layout from shifts in workload and data distribution can be monitored for each partition. Reorganization of data can be triggered at the granularity of each partition if required. We will discuss a practical solution for evolving data layouts in response to changes in the dataset and workload in Chapter 4 in detail.

Decoupling logical and physical partitioning gives more degrees of freedom for evolving the index structures and the data layout. Logical partitioning trees can be adapted in a lightweight manner without reorganizing the physical data blocks. Physical layout of the data can be reorganized based on how well the logical structures align with the physical block boundaries. We defer exploring these strategies to future work.

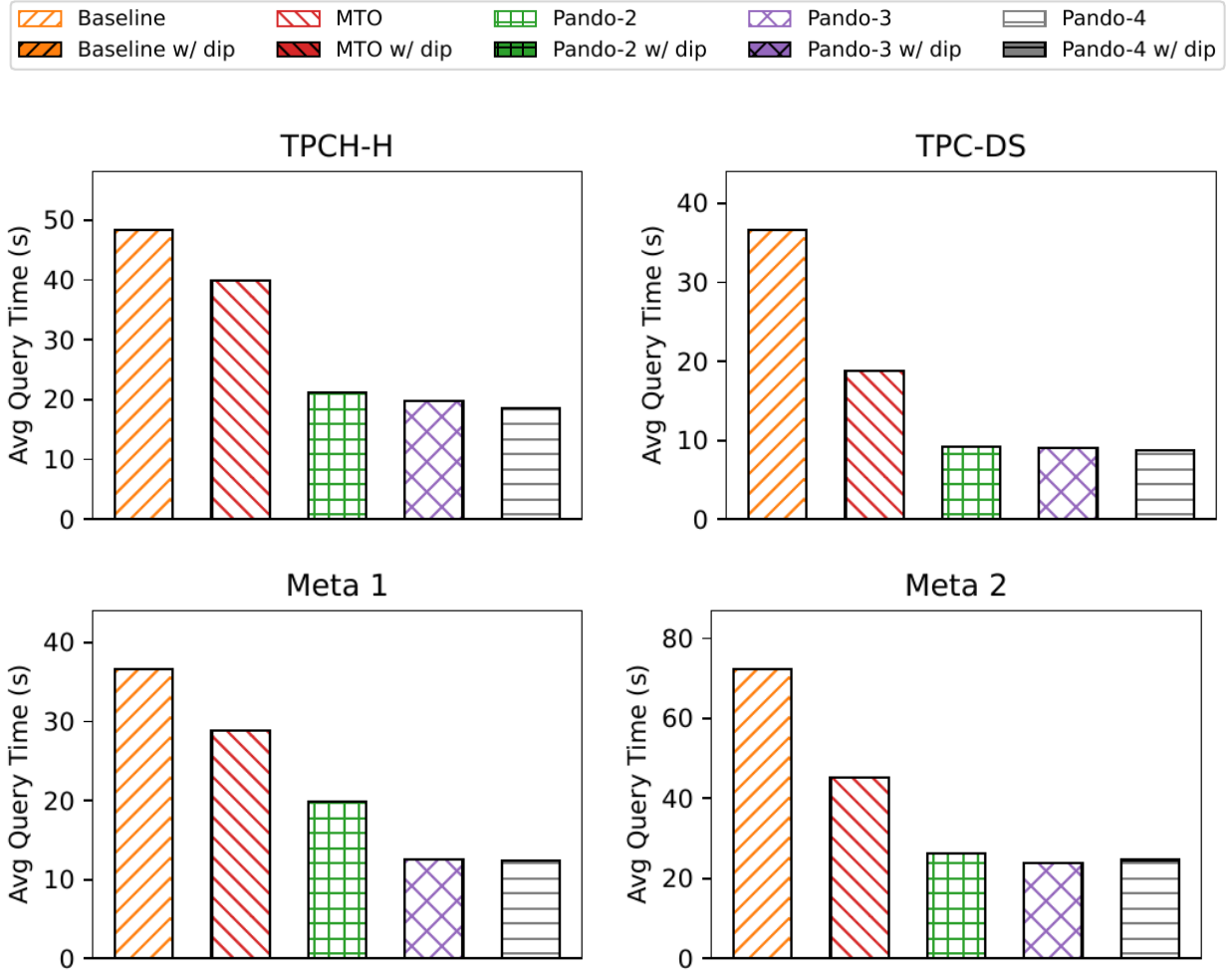


Figure 2.6: End-to-end query execution time on Spark.

2.8 Evaluation

In this section, we present an experimental evaluation of Pando. In our experiments, we seek to answer the following questions.

1. How does Pando compare against other baselines on the amount of I/O performed and overall query execution time?
2. What is the computational and storage overhead in finding the optimized layouts, organizing data according to it, and storing the associated metadata?
3. How does the performance advantage of Pando scale with dataset and workload complexity?
4. How does dynamic data affect Pando’s performance?

2.8.1 Datasets and Workloads

We evaluate Pando on TPC-H [91], TPC-DS [90], and two production workload traces from Meta. We use a scale factor of 100 for the synthetic datasets which corresponds to 100GB

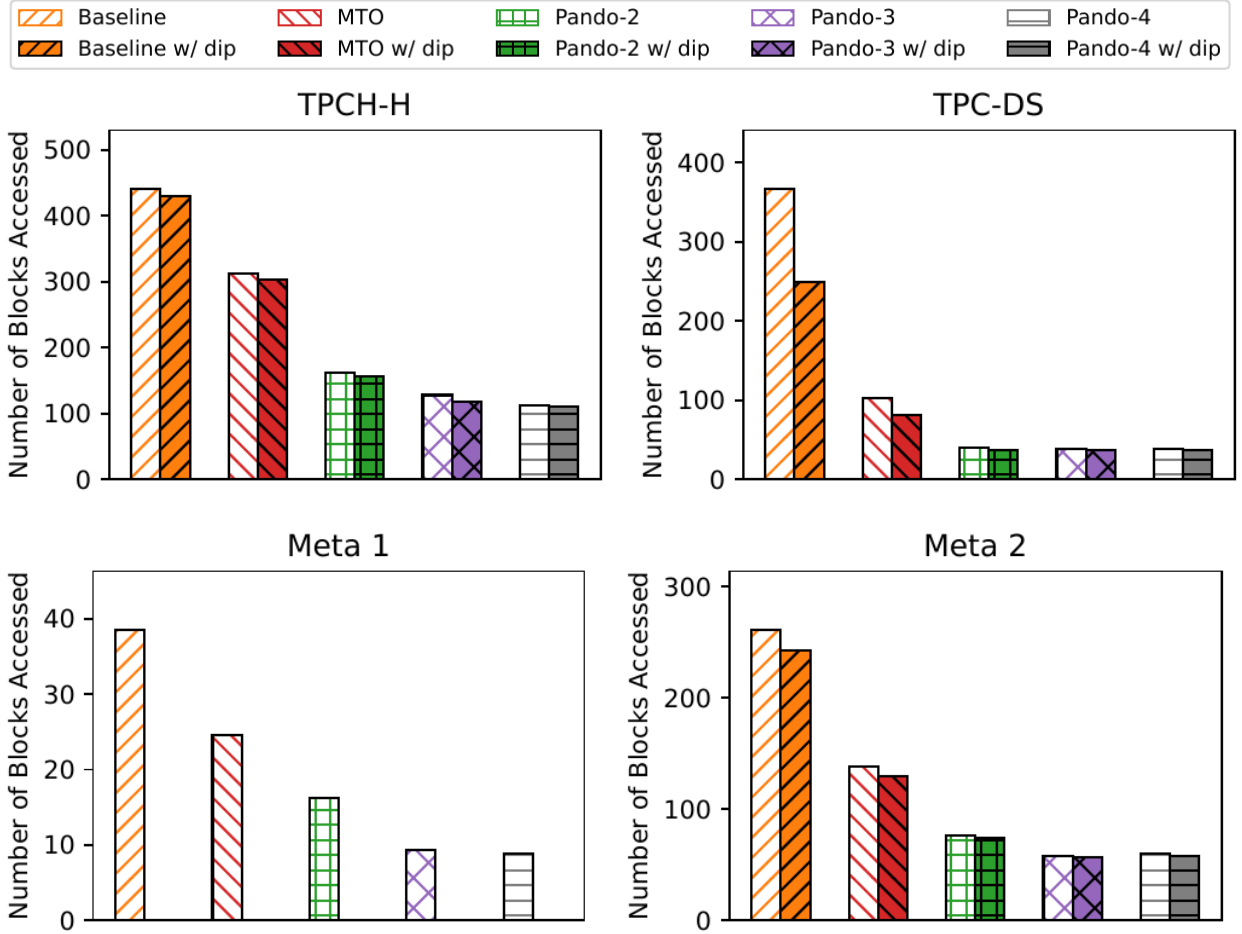


Figure 2.7: Average number of blocks accessed during query execution. Data-induced predicates (diPs) is not relevant to Meta1 as it does not involve any joins.

of data. For TPC-H, we support all 22 templates and use a workload with 176 queries, 8 queries per template. For TPC-DS, we use 184 queries, 4 queries for each of templates 1-50, except for 14, 23, 24, and 39, which contain multiple queries. We use two real-world datasets from Meta. Meta 1 is a single table dataset. We use a sample from one day’s data amounting to around 40 GB of data. Queries are generated from an interactive query engine and involve many correlated `LIKE` predicates on different attributes. The second dataset, Meta 2, contains 4 tables amounting to 120GB of data that powers an interactive dashboard. Queries involve joins over these tables with a variety of predicates on each table. We use a sample of 100 queries from the production traces that were generated on one day in our experiments.

2.8.2 Experimental Setup

We implemented Pando’s layout optimizer and a simulator for the number of blocks accessed during query execution in Python. We performed a shallow integration with Meta’s internal version of Spark. We use Hive [89] for storing the data in our experiments. We augment each table in the dataset with an additional partition column `block_id`. This field is the block

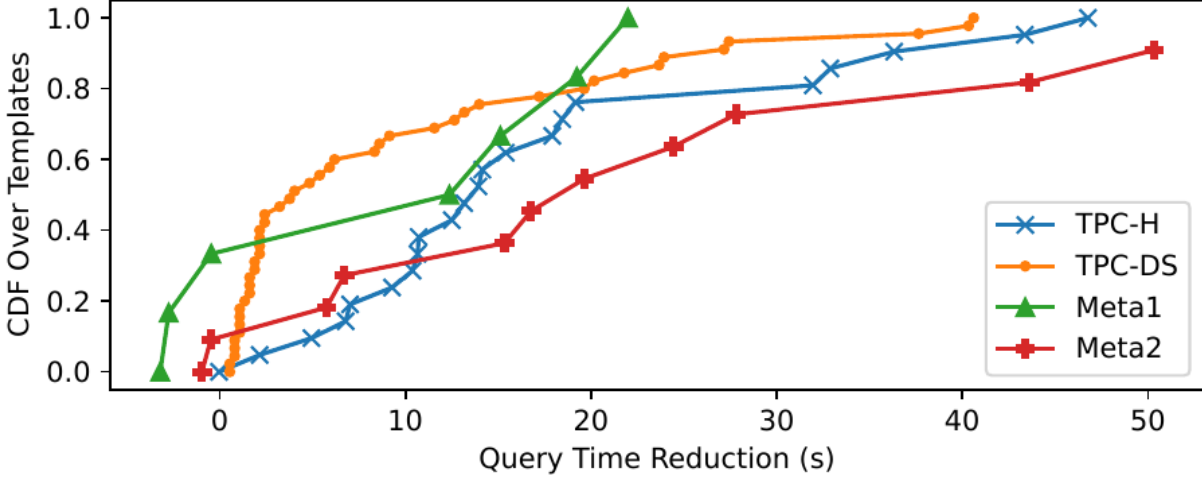


Figure 2.8: Reduction in query times achieved by Pando compared to MTO for each query template.

id for each tuple obtained after running Pando’s layout optimizer. Each table is partitioned on `block_id` in the Hive cluster. Each block is stored as an ORC file. Integer columns are run-length encoded and strings are dictionary encoded which are in turn compressed using Zstandard algorithm. We augment each query with an additional filter for each table of the form `block_id IN ()` with the block ids obtained from Pando index data structures. For example, if the index on T1 return blocks 1 and 7, and the index on T2 returns blocks 8 and 14 for the query `SELECT * FROM T1 JOIN T2 on Key WHERE T1.X < 17 AND T2.Y >= 45`, we execute the query `SELECT * FROM T1 JOIN T2 on Key WHERE T1.X < 17 AND T2.Y >= 45 AND T1.block_id IN (1, 7) AND T2.block_id IN (8, 14)`. Spark uses this filter to skip unnecessary blocks. The partition column `block_id` is encoded in the directory path and not materialized as a part of the table’s data in Hive. So the overhead of this integration on query performance is minimal. We report the end-to-end query execution time for a single-node spark environment.

We compare Pando against two other layouts. First, a baseline, that range partitions each table on a single user-tuned column. For TPC-H, we sort lineitem by shipdate, orders by orderdate, and all other tables by primary key for the baseline. For TPC-DS, we sort all fact tables by date (`sold_date` for sales tables and `returned_date` for returns tables) and all dimension tables by primary key. For Meta datasets, we use the same single/multi-column range partitioning as in the production instance. Second, we use MTO as the instance-optimized layout that organizes data according to a single hierarchical physical partitioning tree.

Data-induced predicates (diPs) [42] is a recently proposed technique to propagate predicates that are pushed down on one table through joins to benefit from block skipping on other tables at query execution time, similar to join-induced predicates in Pando. We use diPs to show that baseline layouts along with runtime optimizations cannot achieve the same speed up as a layout that is aware of correlated expressions. We use this in our simulator to show the reduction in I/O. Spark does not support diPs and it needs a deeper integration into Spark’s query optimizer and execution engine. So we could not show end-end runtime

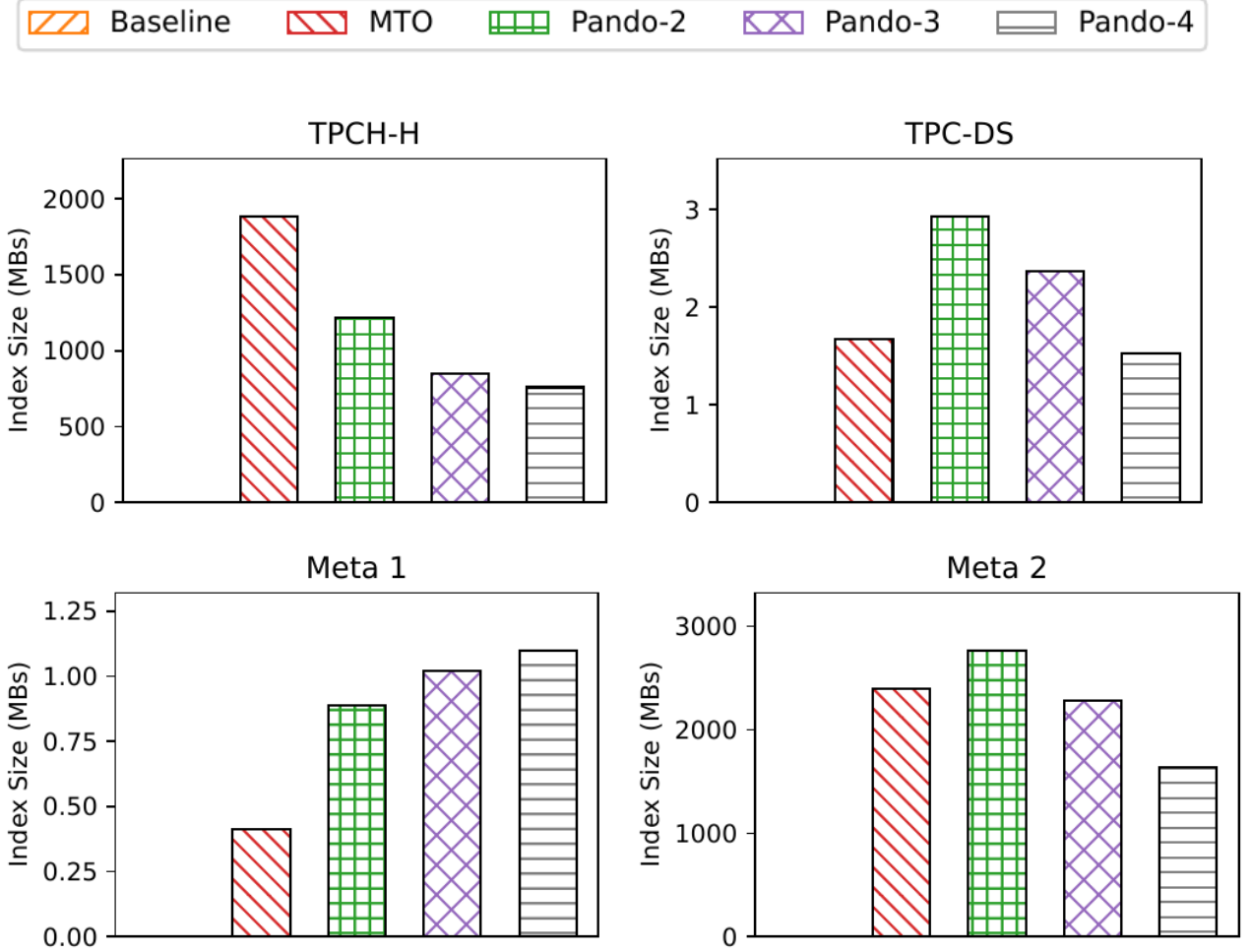


Figure 2.9: Size of the partition tree indexes.

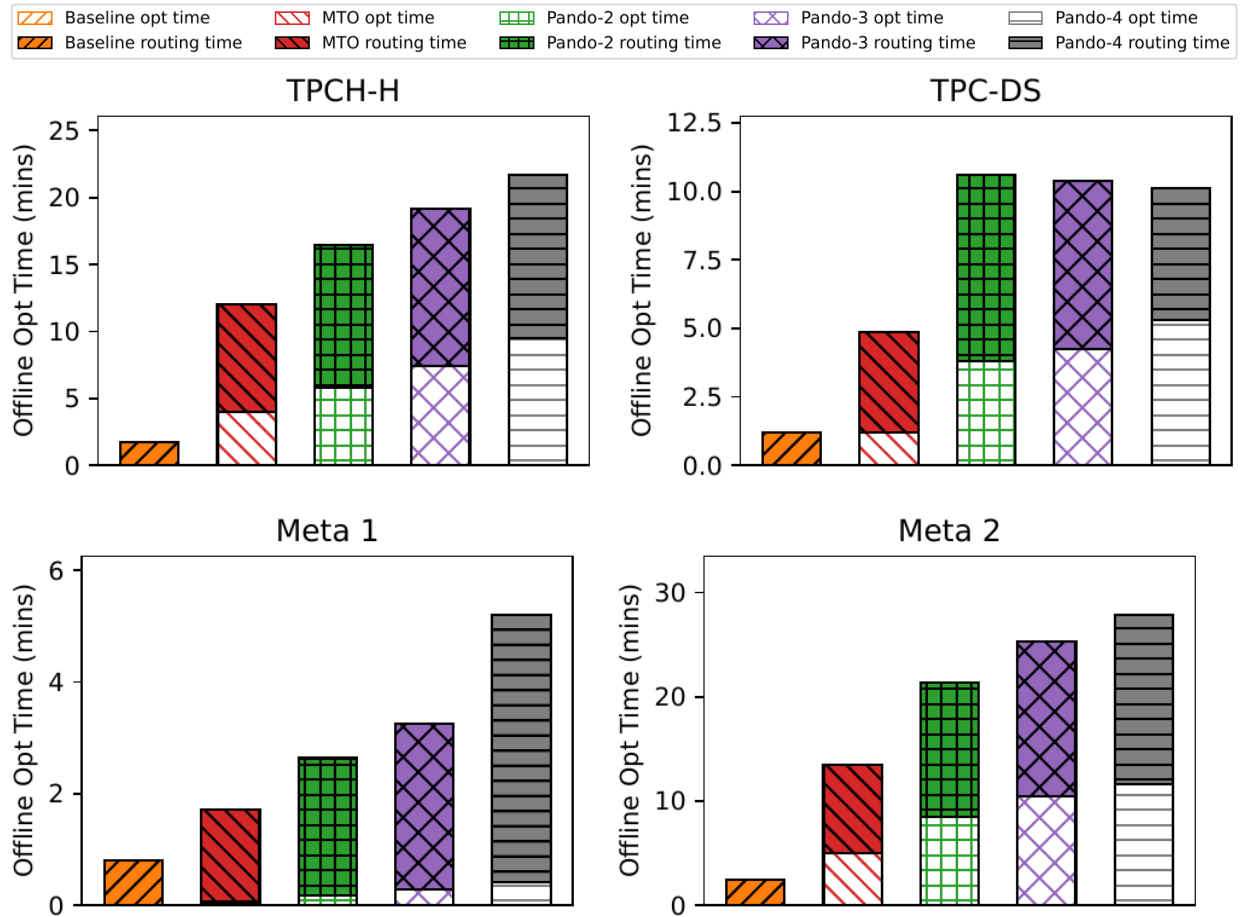
numbers with diP.

We performed all our experiments on a machine running CentOS Stream 8 equipped with a 72core Intel Core Processor (Broadwell) with 224GB RAM and a 2TB Toshiba XD5 local SSD. We use a block size of 1M tuples for all our experiments which is the default for many systems like Microsoft SQL Server [50]. We optimized the layout using 50% of the workload and evaluated using the remaining half. We used a 1% sample of the data for optimization. The threshold number of tuples for the top-down stage is set such that there are under 10000 k-tuples for the bottom-up stage. This means the more the number of trees, the larger the leaf nodes of each tree, and the smaller each tree. We report the end-to-end query execution time, the number of blocks accessed during query processing, the space overhead in storing the logical partitions, and the time overhead in optimizing the layout and reorganizing data.

2.8.3 Results

Figure 2.6 shows the overall performance on various layouts. Pando- k means, we created k trees for each table in the dataset. Figure 2.7 plots the number of blocks that are accessed during query execution.

Overall Performance: Pando is consistently better than MTO and range partitioning (the baseline) across datasets and workloads. Pando achieves up to 4.2X faster query



t

Figure 2.10: Time overhead incurred by Pando in offline optimization.

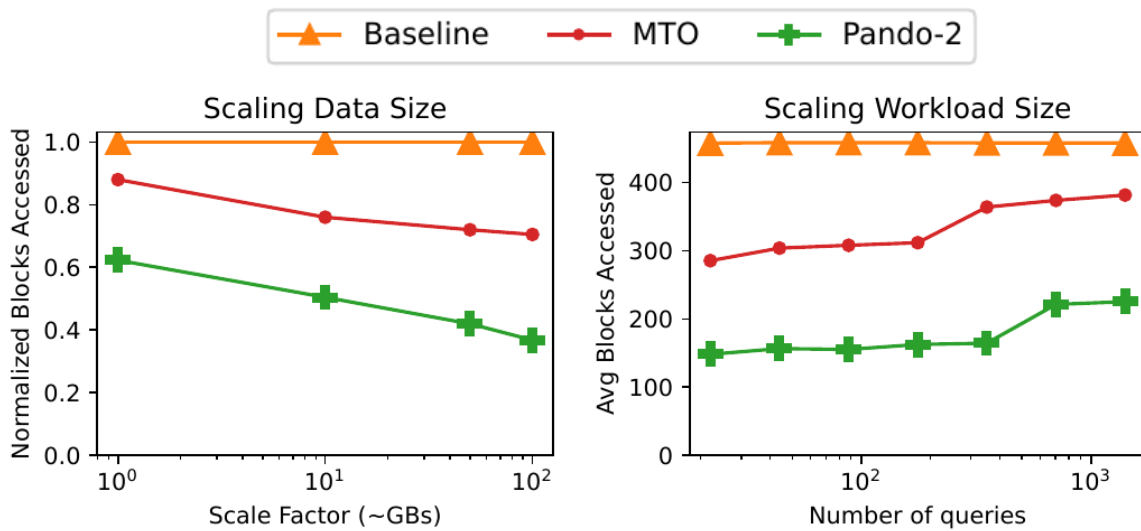


Figure 2.11: Pando's performance across varying dataset sizes, and workload sizes.

performance than range partitioning and up to 2.3X faster performance than MTO. **Pando** attains up to 9.5X reduction in I/O compared to the baseline and up to 2.8X reduction compared to MTO. By capturing correlated expressions in the workloads, **Pando** is up to 2X faster than MTO by creating just one additional logical partitioning tree. As the number of indexes increases, the performance of the layout also increases, but with diminishing returns. For all the workloads we tested on, adding a fourth tree resulted in minimal performance advantages. The trends in the amount of data that is scanned are similar to the overall performance as shown in Figure 2.7.

Impact of diPs: diPs compute join-induced predicates at query optimization time to improve skipping blocks. Its effectiveness depends on how the data is blocked and is sensitive to outliers. While this gives some benefit over all the layouts, it does not match the performance of **Pando** that co-optimizes its indexes and the data layout. diPs decrease the I/O done by MTO as well as **Pando**. But **Pando** still achieves a better reduction in I/O than MTO with diPs. Meta 1 is a single table dataset and the workload does not involve any joins, so diPs are not relevant.

Where does the advantage come from?: Meta 1 is a single table dataset with many LIKE predicates on different columns in the table. Several of these predicates are correlated and **Pando** leverages these in creating multiple partitioning trees to improve query performance. **Pando** accelerates query performance in the other three datasets by capturing correlated expressions across tables. Let us look at TPC-H as an example. Clearly, partitioning the `lineitem` using predicates on `l_shipdate` is useful for TPC-H queries. Attributes `l_shipdate` and `o_orderdate` are correlated, so are the predicates `l_shipdate > ?` and `l_orderkey in (SELECT o_orderkey FROM orders WHERE o_orderdate > ?)`. A single physical partitioning does not fully capture these predicates in the same tree, thus limiting the skipping opportunities. In contrast, **Pando** effectively distributes these predicates over multiple trees and optimizes the data layout to maximize its effectiveness. This improves skipping for queries that predicate on `l_shipdate` as well as that join `lineitem` and `orders` with a predicate on `o_orderdate`. Figure 2.8 shows the reduction in query time **Pando** achieves compared to MTO for different query types. As **Pando** optimizes for the overall workload, performance may regress for some query types.

Index Size: Figure 2.9 plots the space required in storing the partition trees. This includes the space required for the literal cuts, non-truncated trees, and the mapping from k-tuples to block id that is required for supporting data changes. Without including these, all schemes take under 1MB for all their indexes for each of the datasets. The size of the index with joins-induced cuts can be high, depending on the number of cuts in the tree, selectivity of joins, number of unique keys, etc. As discussed earlier, storing multiple trees has the same expressive power as a much larger single tree. So storing multiple trees can also save space. The takeaway from Figure 2.9 is that the partitioning trees that index data blocks are not prohibitively large like secondary indexes. The size of the index does not increase linearly with the number of partitioning trees. We are able to achieve better performance without a major increase in the space requirements for the index.

Offline Optimization Time: Instance-optimized data layouts are only useful when they can be efficiently found and organized. The offline optimization time required for **Pando** is shown in Figure 2.10. `opt time` (optimization time) is the time spent in finding the optimal layout. In **Pando**, this includes the time spent in the top-down and bottom-up stages of the

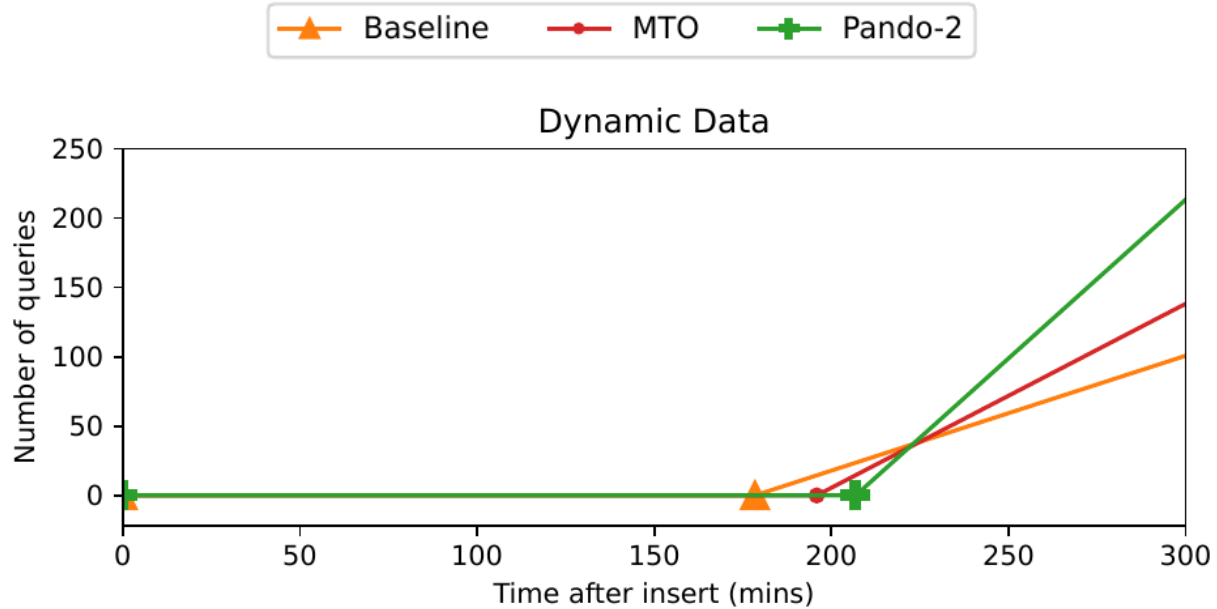


Figure 2.12: *Pando*’s performance with data inserts. Although inserted data is available for querying quickly in the baselines, *Pando* makes up for it with faster execution of queries.

algorithm. **routing time** is the time spent in locally partitioning the full dataset using the tree indexes, i.e., finding the tuple assignment for each block. This combined is the overhead that *Pando* adds over other methods. After local partitioning, each partition is uploaded to the Hive cluster. All partitioning schemes with the same minimum block size take roughly the same time to upload the partitions to Hive, so we do not include it here. In general, the optimization time and data routing time increase with more trees. Figure 2.10 shows that overhead is in ~ 10 s of minutes for datasets with ~ 100 GBs of data. For the workloads that we tested on, *Pando-2* offsets this overhead in as low as 7 queries for Meta1 and as high as 37 queries for TPC-H. The minimum number of queries is 7 and the maximum is 36 for *Pando-2* to offset the overhead over MTO. A C++ implementation of the layout optimizer may be faster, and the absolute overhead incurred by *Pando* may be smaller. *Pando* can make up for the overhead in fewer queries.

Scalability: Figure 2.11 shows that *Pando* maintains its performance advantage over the baseline across varying dataset and workload sizes. To study the impact of increasing dataset sizes, we vary the scale factor on TPC-H from 1 to 100, resulting in 1 to 100GBs of data. We use a workload with 176 queries, 8 per template, and set the minimum block size to 1 million tuples. Figure 2.11 plots the number of blocks scanned normalized against the number of blocks scanned by the baseline as the total number of blocks varies with data size. *Pando* (and MTO) performs better at higher dataset sizes. This is because larger datasets have a larger number of blocks and the partitioning tree can store more expressions for more fine-grained skipping. To study the impact of scaling the workload size, we vary the number of queries per template in the workload from 1 to 64. With 22 templates, this results in workloads ranging from 22 to 1408 queries. We use scale factor 100 for dataset size. Figure 2.11 shows that the number of blocks scanned by *Pando* (and MTO) increases as the workload size increases.

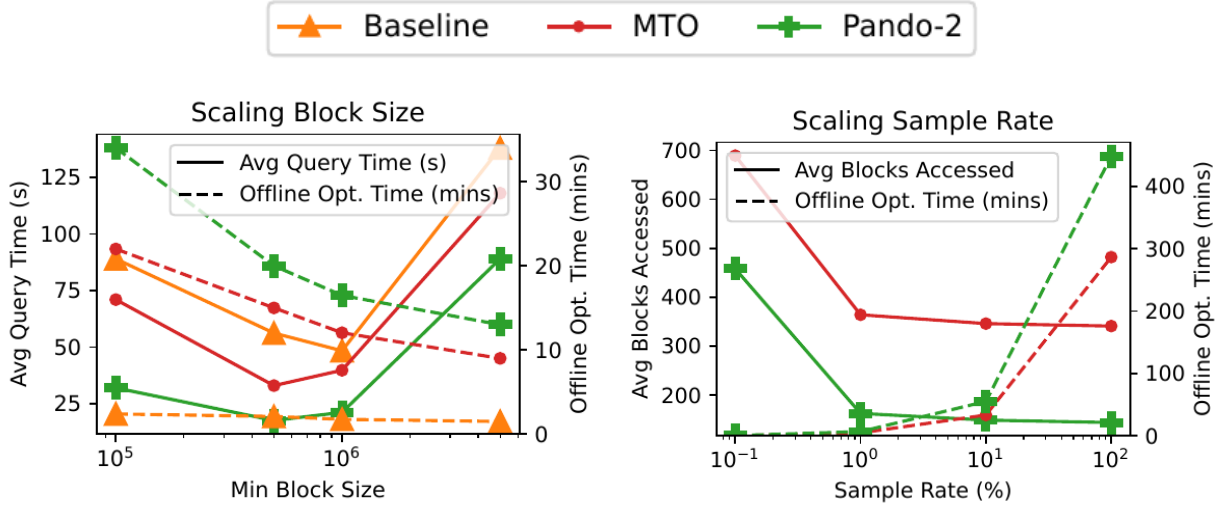


Figure 2.13: Pando’s performance across varying minimum block sizes, and optimizer sample rates.

This is because, with more templates, we need to store more expressions to achieve better skipping. This affects Pando slower than MTO. Pando continues to perform better than other baselines across varying workload sizes.

Dynamic Data: To show how Pando adapts to data changes, we use the Meta2 dataset. We start with one day’s worth of data that is already optimized according to Pando in the database. We then bulk-insert another day’s worth of data into this database as described in Section 2.7. Pando spends around 9.6 mins in repairing the join-induced predicates in the previous day’s partitioning trees and MTO spends around 6.4 mins. Optimization and local data partitioning take 21.3 mins for Pando, 13.4 mins for MTO, and 2.4 mins for the baseline. Meta’s implementation of Hive does not support repartitioning the data based on a new column. So we have to repartition the data separately and insert each partition separately into the Hive cluster by explicitly mentioning the value of the partition column (aka the directory path). Uploading the data to Hive takes around 3 hrs in our setup. As shown in Figure 2.12, Pando is able to quickly recover the additional time spent in optimization and repair and surpass the baselines in the number of executed queries.

Sensitivity: Figure 2.13 shows the impact of varying minimum block sizes (100k, 500k, 1M, and 5M tuples) on query performance and offline optimization time using TPC-H with scale factor 100 using 8 queries per template. The offline optimization time decreases with increasing block size. The query latency initially decreases as queries scan fewer data, but eventually increases due to too many small block accesses. We study the impact of varying sample rates during layout optimization (0.1%, 1%, 10%, and 100%) on Pando’s query performance and optimization time. We use TPC-H with a scale factor 10, 8 queries per template, and a minimum block size of 100k. Pando is able to significantly reduce the optimization time by sampling without significantly affecting the layout quality.

2.9 Related Work

Data Partitioning: Carefully organizing data on storage for accelerating the performance of queries is a well-studied area. Most production systems often partition the data either based on ingestion time, or using range, hash, or round-robin distribution schemes [50]. Several techniques have been proposed to tune the data layout to reduce the amount of data that is scanned using horizontal, vertical, and hybrid partitioning [73, 24, 13, 83, 16, 33]. To prune the set of the blocks that are scanned, these systems maintain additional statistics along with these blocks such as minimum and maximum values for each attributes called Small Materialized Aggregates (SMAs) [59], and zone maps [36]. Slalom [64] logically partitions the data and build lightweight partition-specific indexes, but they do not consider optimizing the physical layout of the data. [34] combines physical and virtual partitioning to fragment and dynamically tune partition sizes for efficient intra-query parallelism, but not data skipping. Distributed database systems employ various strategies to partition data between nodes to minimize the number of cross-partition queries [41, 55, 26]. Pando can be used to optimize the layout within each node.

Learned Layouts: Recent work has shown that significant gains in query performance can be achieved by more expressive partitioning schemes that collect rich metadata and can adapt to datasets and workloads [56, 47, 32, 95, 31, 84]. Qd-tree [95] and MTO [31] are learned layouts that maximize data skipping in cloud-based DBMSs (discussed in this chapter). Feature-based data skipping [86, 87] maintains a bit vector per block where each bit indicates whether a predicate corresponding to the bit is satisfied by any tuple in the block. These methods and Pando use bottom-up clustering to find the data layout, but their method only scales to a small number of predicates and we store much richer metadata in the form of multiple expression trees. Qd-trees [95] already showed better performance than feature-based skipping with just one partitioning tree. Tsunami [32] is a learned partitioning scheme that is aware of correlations, but they only capture simple correlations in data and only work for multi-dimensional range queries on in-memory databases. CopyRight [84] co-optimizes multiple expressive partitionings, but replicates data and creates one partitioning per replica.

Automated Physical Database Design: There is a rich corpus of work on automatically tuning a database and its physical design that focuses on selecting structures like indexes and materialized views for optimizing query performance [10, 22, 69, 12, 21]. Each partitioning tree in Pando is a coarse-grained index that indexes data blocks. Many index selection works consider co-optimizing the layout along with indexes [13, 22, 10]. However, they only work for simple schemes like single or multi-column range and hash partitioning, b-tree indexes, etc. Materialized view selection algorithms explore the space of syntactically relevant views, focusing on creating logical views that cover queries, whereas we focus on optimizing the physical layout of the table [12, 99, 20]. Pando can be used to partition materialized views.

Correlation Awareness: Prior work has explored several methods to take advantage of column correlations such as automatically discovering algebraic constraints [18], soft functional dependencies [18], and approximate dependencies between columns [49] in the data. Our method goes beyond these methods and captures correlations using predicates that appear in the workload. Various techniques such as Hermit [93], Correlation Maps [45],

Cortex [61] exploit correlation among columns to improve the performance and reduce the size of the secondary indexes. Hermit [93] captures monotonic functional dependencies between attributes, while Correlation Map [45] maintains a mapping between correlated columns. Cortex and Pando look at outliers from a similar lens of query performance, but Cortex focuses on capturing correlation between a user-provided set of columns. None of these techniques consider optimizing the physical layout of the table. CORRADD [46] is a correlation-aware physical database designer but they explore a smaller search space that consists of only single-column sort orders for physical layouts and single-column b-tree indexes.

2.10 Conclusion

I/O for accessing data from storage is a major bottleneck for cloud-based data analytics systems. Modern cloud-based database systems employ a variety of strategies to partition the data into blocks and skip accessing irrelevant blocks during query execution. The effectiveness of these systems depends on how well the data layout and the associated metadata align with the queries that are run on it. Existing methods miss substantial opportunities for block skipping blocks because they do not collect enough metadata or do not exploit correlated predicates in the workloads. In this chapter, we propose **Pando** a metadata-rich data layout framework that achieves a significant reduction in the amount of I/O performed by jointly optimizing the physical layout of the data and multiple correlation-aware logical partitionings. Our experiments show that **Pando** achieves up to 2.3X speedup in end-to-end query execution time and up to 2.8X reduction in the number of blocks scanned compared to the state-of-the-art data layouts.

Chapter 3

CopyRight: Instance-Optimized Replicated Data Layouts

3.1 Introduction

Carefully organizing data on storage is key to achieving high query performance in modern analytical DBMSs. Modern data warehouses use a variety of techniques including horizontal and vertical partitioning, clustered indexes, sort orders, etc. to accelerate scan performance by reducing the cost of accessing and processing data.

Several algorithms, often based on historical properties of a workload of queries run over the data, have recently been proposed to tune the physical layout of tables to optimize performance of the workload [47, 83, 25, 32, 95, 13, 40, 15]. Complex real-world workloads contain queries with varying access patterns including queries that access different attributes, have different selectivity characteristics, and read different subsets of records. For example, dashboard-based interactive data analytics platforms allow users to create query templates with a variety of predicates by zooming in on maps, enforcing ranges on graphs, selecting from drop-down menus etc. Users analyzing a dataset of New York City taxi trips¹ issue queries across a variety of dimensions to find the average fare amount of trips between two parts of New York City where the passengers paid by cash, and to find the number of trips in a particular month of the year that costed more than \$5, etc. Existing layout tuning algorithms optimize for the workload on average. However, a single layout may not be optimal for queries of different types. As a result, the benefit of these specialized data layouts decreases as queries with different access patterns scan the same data.

To address these limitations, we built **CopyRight**, a novel in-memory read-optimized partial replicated layout engine that automatically optimizes the layout for a particular dataset and query workload. **CopyRight** replicates different parts of the data at varying degrees and organizes each replica under different layouts to maximize the overall query performance. Our layout engine supports sub-table replication where only a subset of columns are materialized in each replica.

Prior work has explored several approaches for automated physical database design in both single node and distributed systems [12, 13, 22, 21]. To the best of our knowledge, **CopyRight**

¹<https://www.omnisci.com/demos/taxis>

is the first layout-aware partial replication engine for performance in an in-memory single node setup. Joint optimization of partitioning and replication has primarily been studied in the context of distributed databases [26, 57, 41, 55, 51]. The focus of these approaches is on minimizing the number of cross-partition queries rather than the exact layout of data in each partition.

Under storage constraints, **CopyRight** achieves high performance by focusing replication efforts on parts of the data where it matters. A table-level replication scheme may waste storage by replicating areas that give minimal advantage such as cold parts of the data that is rarely accessed or parts of the data where all queries have similar data access patterns. **CopyRight** identifies sub-spaces of data that yield large benefit from replication and allocates available storage accordingly to maximize the overall gain. By materializing only a subset of columns in each replica, **CopyRight** is able to better utilize the available space and create more replicas.

Incorporating partial replication makes the layout design problem much harder as the space of layouts that need to be considered significantly increases. A table can be horizontally partitioned in a large number of ways (the so-called *Bell number*), which grows faster than exponential in the number of tuples. With replication, any subset of these partitionings that fits in the storage bound is a feasible layout, causing a combinatorial explosion in the space of physical design alternatives. With partial replication, a replica may materialize any subset of the tuples and any subset of the columns, further exacerbating the combinatorial nature of the problem.

CopyRight organizes data as a two-level structure: the data space is split into regions and each region is replicated differently. Our layout optimizer employs a number of novel techniques to intelligently prune the space of layouts. Instead of directly searching in the space of layouts, we approach replication as a workload partitioning problem. We partition the workload into clusters and create a replica that is optimal for each cluster. We present a scalable algorithm that jointly optimizes partial replication and layout of each replica by leveraging properties of the data and workload.

In summary, this paper makes the following contributions

1. We present **CopyRight**, a novel layout-aware partial replication engine that concentrates replication efforts on parts of the data where it matters.
2. We provide an efficient heuristic algorithm to determine good partially replicated layouts and estimate the cost-space Pareto optimal frontier for a given dataset and workload.
3. Finally, we demonstrate that **CopyRight** can outperform other layouts for a variety of real-world datasets and query workloads. **CopyRight** is 1.1X to 7.9X faster than the best non-replicated layout with 0.25X overhead. In 25% space overhead **CopyRight**'s partial replicated layout is able to achieve the same or up to 5.2X better performance than a table-level replicated layout that incurs 100% overhead .

The paper is organized as follows: we make the case for partial replication in Section 3.2, provide an overview of our approach in Section 3.3, present the layout structure in Section 3.4, describe the optimization algorithm in Section 3.5, describe the query workflow in Section 3.6, show our experimental evaluation in Section 3.7, compare to related work in Section 3.8, and conclude in Section 3.9.

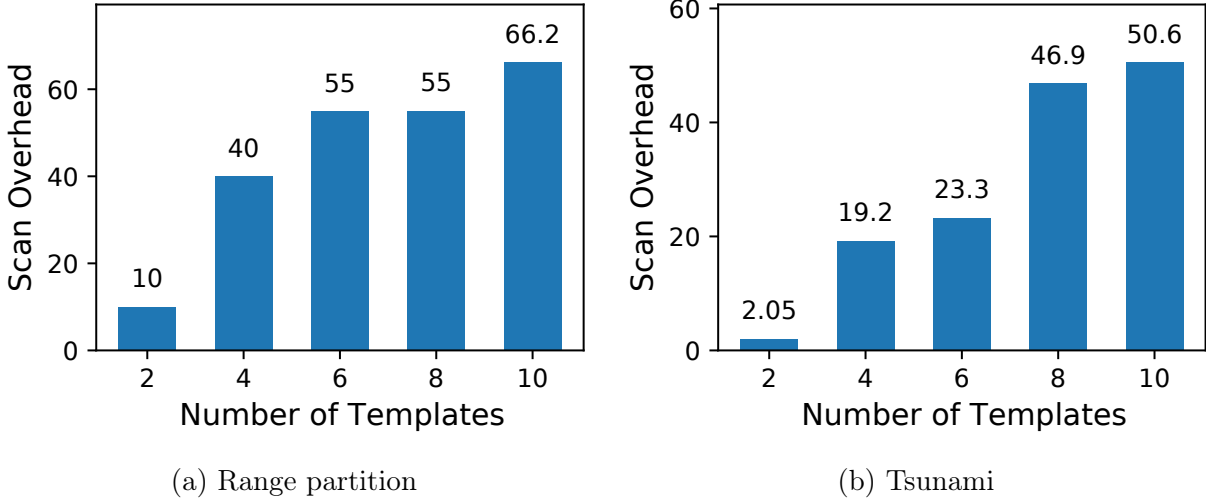


Figure 3.1: As the number of query templates (different access patterns) in the workload increases, the scan overhead of non-replicated layouts increases, degrading performance.

3.2 Motivation

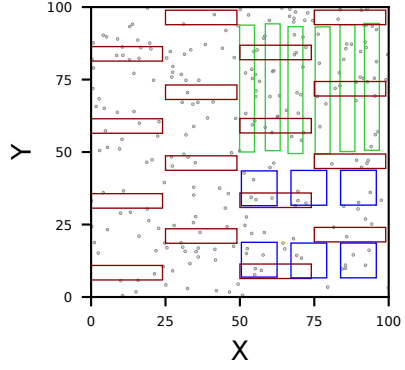
We begin with an overview of the impact of heterogeneous query access patterns on the performance of database systems for analytical workloads. We then make the case for layout-aware replication of data focusing the efforts on parts where it matters.

Background: In this work, we focus on multi-dimensional analytical workloads. A table with d numeric columns can be thought of as a d -dimensional space and tuples as points in the d -dimensional space. Queries are equivalent to hyper-rectangles in the space.

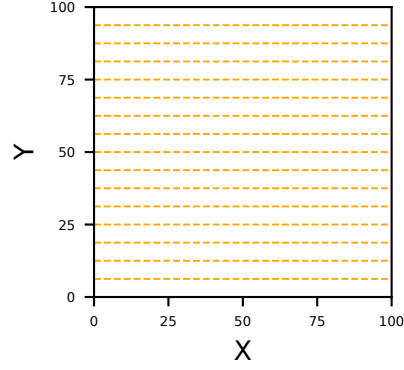
With queries that filter data on multiple attributes, DBMSs can use secondary indexes to improve scan performance. However, a secondary index scan works better than a full sequential scan only when the predicate on the indexed attribute has very low selectivity [77, 44]. An alternative approach is to partition data, organize partitions in some sorted order on one-dimensional storage, and build auxiliary multi-dimensional indexes to skip partitions that don’t match query filters. Examples include range partitioning on single or multiple columns, Grid Files [62], k-d tree [17], R-tree [39], Z-order [81], Flood [60], and Tsunami [32]. The partitions and sort order are adapted to reduce the *scan overhead*, the ratio of number of records scanned by the query to the number of records that satisfy the query filter, without incurring large index lookup costs.

Tsunami [32], is a recently proposed learned multi-dimensional layout that has been shown to outperform a variety of other traditional and learned variants. It horizontally partitions the data into non-overlapping regions and organizes each region as a grid. The two-level structure of Tsunami accounts for query skew (i.e, query frequency varies in different parts of data space) by laying out each region differently. The boundaries of the grid and regions are learned based on the data and workload distribution.

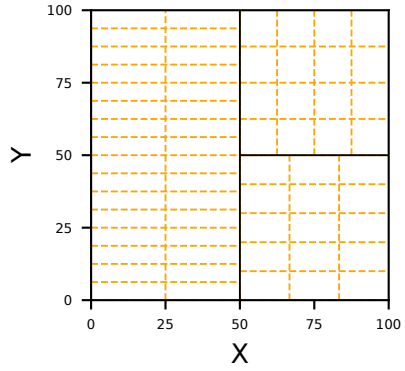
Performance of non-replicated layouts: Non-replicated layouts incur high scan overheads when queries with different access patterns scan the same parts of the data. Figure 3.1 shows the scan overhead of the workload with increasing number of query templates in the workload for non-replicated layouts. Data is range-partitioned on the most frequently



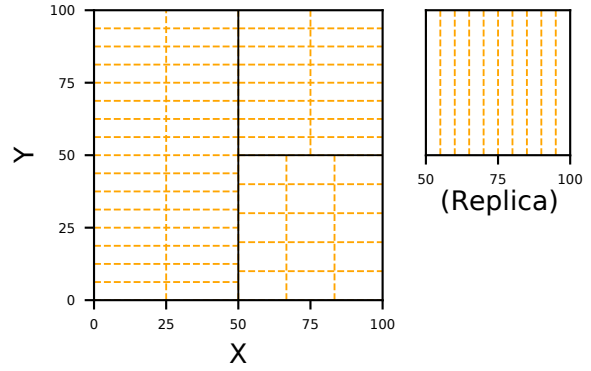
(a) Dataset and workload



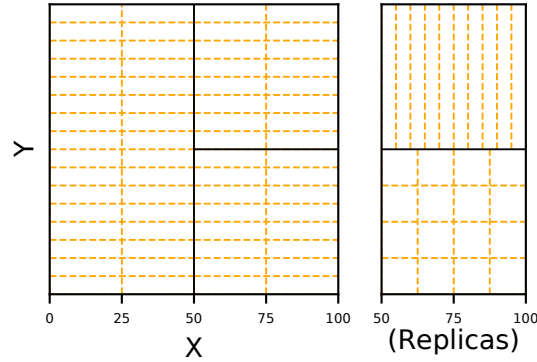
(b) Non-replicated layout (range partition)



(c) Non-replicated layout (Tsunami)



(d) CopyRight with 0.25X replication budget



(e) CopyRight with 0.5X replication budget

Figure 3.2: Figure 3.2a shows a multi-dimensional dataset and workload with 2 columns X and Y . The grey dots represent tuples and the rectangles represent queries. Figure 3.2b shows the data range-partitioned on column Y . Figure 3.2c shows the best layout that Tsunami is able to obtain for the workload. Figure 3.2d shows CopyRight's layout with sufficient budget to replicate a quarter of the table. The upper right quarter of the table is replicated. Layout of one of the replicas is optimized for the short and wide red queries. Other one is optimized for the tall and narrow green queries. With enough budget to replicate half the table, CopyRight also replicates the bottom right quarter with one replica optimized for red queries and other for blue queries.

accessed column in Figure 3.1a and organized using Tsunami in Figure 3.1b. The table has 8 dimensions with uniformly randomly distributed data. The workload contains equally represented query templates that filter on two randomly selected columns. Queries are uniformly distributed in the data space. Even with a few templates accessing the same data, the scan overhead of the workload is significantly high and queries waste a lot of time scanning unnecessary data. This is a consequence of the *curse of dimensionality* and affects non-replicated layouts across the board.

To illustrate where this overhead comes from in more detail, consider an example of a 2-dimensional dataset and workload shown in Figure 3.2a. Here, X and Y axes represent the 2 columns in the table. Data is uniformly randomly distributed, as shown by grey dots. The workload contains 3 types of queries with range predicates on both columns. Dark red queries that are highly selective along Y are uniformly distributed throughout the entire data space, light green queries that are very selective along X in the upper right quarter of the data space, and blue queries with moderate selectivity along both X and Y in the bottom right quarter.

Figure 3.2b shows the data range-partitioned on column Y. The orange lines indicate the partition boundaries. While red queries are well-aligned with the partitioning, green ones are not and executing them will require processing many partitions and thereby yield a poor scan overhead. A single layout is efficient for some queries but inefficient for others. This is also applicable to more expressive layouts like Tsunami.

Figure 3.2c shows the layout Tsunami finds for the workload. Here, the data space is divided into 3 regions having different query distributions. The black lines indicate the region boundaries. Within each region, the data is further partitioned into cells by constructing a grid. The orange dotted lines indicate the cell boundaries. Tuples within a cell is laid out contiguously on storage. At query time, we find all orange cells that intersect the query and scan all tuples from each intersecting cell. Grid dimensions are partitioned based on the “shape” of the queries, i.e, the selectivity of queries along each dimension. As the right half of the data space has queries with multiple access patterns, Tsunami optimizes its layout for the average query. Data is laid out as a grid that is optimal for the overall workload, but suboptimal for each query type individually. This is shown in the two quadrants in the right half of Figure 3.2c.

To reduce the scan overhead of both query types with a single layout, we need to create more fine-grained cells. However, this incurs additional cost for looking up the intersecting cells to scan. So a fine-grained grid is not a scalable solution for handling heterogeneous access patterns. We can see from the shape of the grid cells, that queries of both types incur a scan overhead. Such effects are exacerbated by workloads with a large number of access patterns, very different access patterns, queries scanning different columns, etc as suggested by Figure 3.1.

The case for a partial replicated layout: If we had additional space, we could replicate parts of the data and layout replicas differently to reduce the overall query latency. With sufficient space to replicate a quarter of the data, we can create a replica for the upper right region $X \geq 50$ and $Y \geq 50$. One of the replicas can be optimized for answering red queries. The other one can be optimized for the green queries. This is shown in Figure 3.2d. We choose to replicate the upper right quarter over the bottom right as red and green queries have very different access patterns. Allocating the replication budget smartly is important as

splitting red and green queries into two replicas gives more advantage than splitting red and blue. If we had sufficient space to replicate half the table, the bottom right quarter can also be replicated and each replica can be optimized for its query type as shown in Figure 3.2e. This is exactly what **CopyRight**, our novel replicated layout engine, does.

Real-world workloads include queries with a variety of templates and access patterns. The distribution of these access patterns varies across the data space and so does the benefit from replication. For four real-world datasets, our algorithms found 12 - 20 query templates and 19 - 62 sub-spaces of data with different distributions of access patterns. We take a partial replication approach that concentrates replication efforts in parts of the data that provides maximum reduction in the execution cost. As workloads show patterns in columns being accessed together, we allow sub-table replication, materializing only a subset of columns in a replica, to efficiently utilize the available space by creating multiple smaller replicas.

3.3 Overview

As shown in Figure 3.3, **CopyRight** consists of two components: a layout optimizer, for determining how to layout data, and a query executor, responsible for executing queries on this layout. We briefly describe these components below, starting with a description of the structure of data in **CopyRight** and then describing how our optimization engine automatically selects good layouts for a given workload.

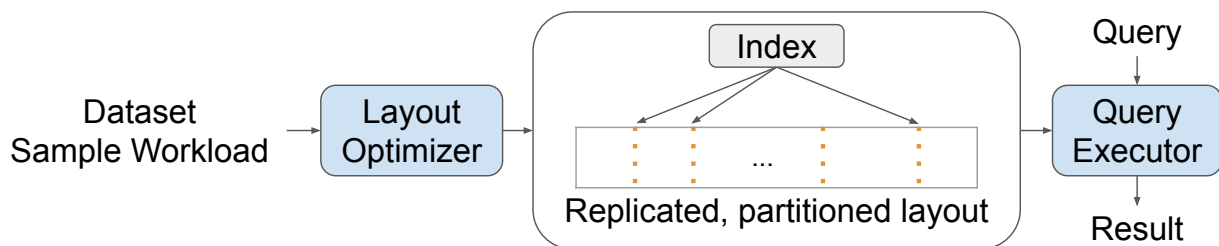


Figure 3.3: Overview of **CopyRight** architecture.

Layout: (Section 3.4) **CopyRight** organizes data as a hierarchy of grids. The data space is hierarchically partitioned into non-overlapping *regions*. In Figure 3.2d the black lines indicate the region boundaries. Each region can be replicated any number of times. Each replica is organized as a grid-based layout on storage. A replica may contain only a subset of columns but contains all tuples in the region. In Figure 3.2d the orange dotted lines indicate grid boundaries. For the rest of the paper, we use ‘region’ to denote the unit of replication and ‘grid’ to denote the layout of each replica of a region. This two-level structure reduces the size of the search space and allows us to efficiently navigate the search space in a top-down fashion by restricting the granularity of replication to horizontal partitions of the table. Throughout this paper, we use horizontal partitioning to denote splitting data into disjoint sets of rows. We use a grid-based layout as it is simple enough to tune efficiently. We use a space partitioning tree to index the regions. The tree index combined with the grid layout is expressive enough to achieve high performance for complex workloads.

Layout Optimizer: (Section 3.5) Under storage constraints, we need to concentrate the available replication budget in regions where it matters to get maximum performance from

partial replication. Identifying these regions and allocating the replication budget among them is non-trivial as the benefit obtained from replication depends on a large number of factors such as the density of queries, distribution of data, similarity of access patterns, etc. For example, in Figure 3.2, with sufficient space to replicate a quarter of the table, we chose to replicate upper right quarter over the bottom right quarter as red and green queries have vastly different access patterns. If the density of blue queries was much higher than green ones, it would have been better to replicate the bottom right quarter. Allocating available replication budget gets harder as the distribution of access patterns becomes more complex.

Given a dataset D , a representative workload W , and a space budget for replication B , CopyRight’s layout optimizer finds the data layout that minimizes overall query execution time. The optimizer has to find the regions, allocate available storage for replication amongst them, decide how many replicas to create for each region, what columns to materialize in each replica, and how to layout each replica. All these sub-problems are tightly coupled, making the search process hard. We discuss our heuristic optimization algorithm that addresses these challenges in detail in Section 3.5.

Query Executor: (Section 3.6) When a query arrives, we find the regions that intersect the query predicates using the tree index. Within each region, we pick the best replica, the one that minimizes the execution time, to execute the query. Within each replica, we use find the grid cells that intersect the query filter, scan the tuples in the cells and emit the tuples that satisfy the filter.

Now, we describe these components of CopyRight in more detail.

3.4 Layout

In this section we describe CopyRight’s layout structure. CopyRight organizes data as a hierarchy of grids using a two-level structure. The top level structure is *Grid Tree*, a hierarchical space partitioning tree. Leaf nodes of the tree correspond to a region and are replicated. Each replica is laid out using *Augmented Grid*. These layouts were introduced in [32]. We extend them to support partial replication.

Grid Tree: CopyRight horizontally partitions the data space into non-overlapping regions. We use a hierarchical space partitioning tree, called Grid Tree to index these regions. The root node of the Grid Tree corresponds to the entire data space. Each node splits the data space into multiple child nodes based on a dimension. The dimension to split and the boundary values are picked based on the workload and dataset. Leaf nodes of the Grid Tree are the regions that are replicated. Figure 3.4 shows the Grid Tree index for the replicated layout in Figure 3.2e. The root node, the entire data space, is split into 2 child nodes based on dimension X . The node $50 \leq X$ is further split based on dimension Y . Leaf nodes are the regions that are replicated and organized as one or more grids represented by dashed circles. Region R_0 has only a primary replica ($AG_0^{(0)}$) whereas regions R_1 and R_2 are replicated twice ($AG_i^{(0)}, AG_i^{(1)}$).

Augmented Grid: An Augmented Grid is a generalization of a grid that takes advantage of the correlation in data. An Augmented Grid splits the data into p_X partitions along each dimension X of the table. It may split a dimension, say X , into equally sized partitions using $CDF(X)$, cumulative distribution function of X , independent of other dimensions.

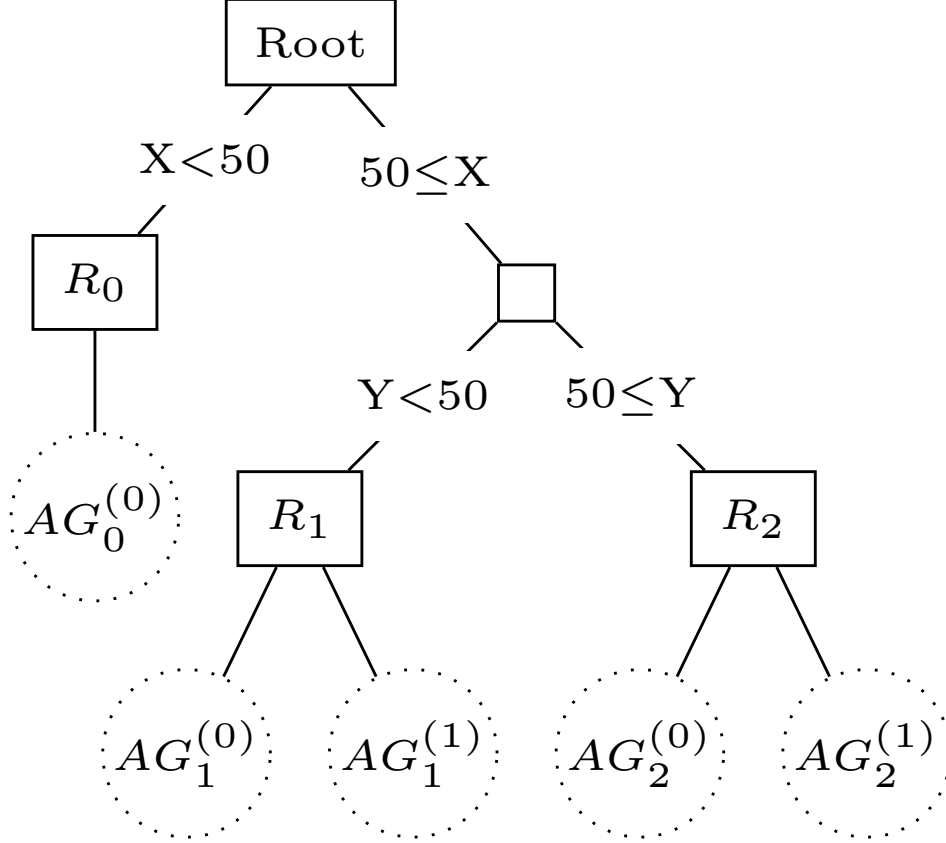


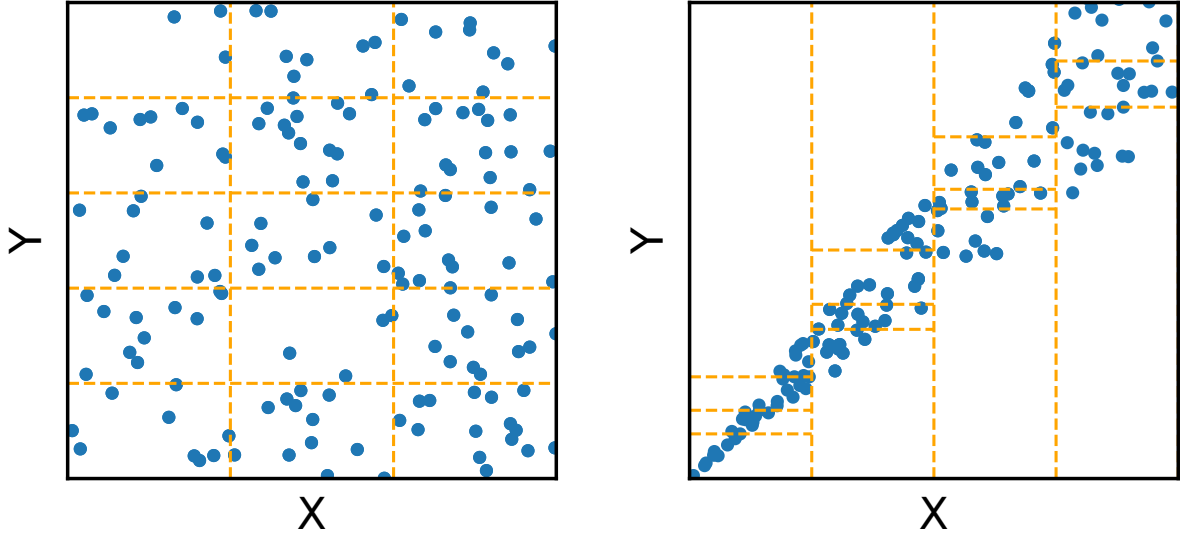
Figure 3.4: An example Grid Tree.

In Figure 3.5a the data is partitioned into 3 columns based on X such that each column has approximately the same number of records. Independently, it is partitioned into 5 rows along Y . Augmented Grid may also choose to partition a dimension Y dependent on another dimension X , uniformly in $CDF(Y|X)$. For example, in Figure 3.5b, each partition along X is further split into cells with equal number of tuples using $CDF(Y|X)$. When combined, these partitions form a grid with $\prod_{X \in Attrs} p_X$ cells, which are ordered. The tuples within each cell are stored contiguously on storage. The number of partitions along each dimension is decided based on the workload. Soft functional mappings between columns are stored for rewriting query filters during execution to take full advantage of data correlation.

3.5 Layout Optimizer

The input to our optimization algorithm is a dataset D , a representative workload W , and a replication budget B . The output is the set of regions and an associated Grid Tree index, how each region (leaf node of the Grid Tree) is replicated, and the Augmented Grid layout for each replica. We first give an overview of our heuristic optimization algorithm and then delve into details.

We approach replicating a region as a workload partitioning problem. Queries in the sample workload W that intersect the region are partitioned into clusters. A replica is created for each cluster and is organized optimally for queries in the cluster. To scale this problem, we



(a) Dimensions X and Y are partitioned independently

(b) Dimension Y is partitioned based on dimension X

Figure 3.5: An example Augmented Grid layout.

first group queries with similar access patterns into *types*, each of which becomes a potential target for an optimized replica, and then solve replication as a partitioning of the set of all query types in the workload (Section 3.5.1).

Associated with each leaf node of the Grid Tree (region) is a *replication configuration* which describes how the data is replicated. As we explore the space of layouts in a top-down fashion, a block (horizontal partition) may be replicated according to a replication configuration or may further be divided into smaller blocks that are replicated under different replication configurations (Section 3.5.2).

Ideally, we want to partition the data space into as few regions as possible such that further dividing these regions into smaller blocks and replicating them under different configurations does not yield any additional benefit. Then, distribute the space budget among these regions and find the best replication configuration for each such region to maximize the overall performance. Finally, create replicas as specified by the configuration and optimize each replica's layout. The tightly coupled nature of the sub-problems along with huge search space make the optimization problem challenging. As shown in Figure 3.6, the layout optimization algorithm works in 3 stages.

1. First, we horizontally partition the data into candidate regions such that they are good candidates for replicating differently and the cost of executing the workload under **CopyRight** layout can be efficiently estimated for any space budget. Note that a candidate region may be split into regions that are replicated under different replication configurations in the final layout. (Section 3.5.3)
2. For each candidate region, we then estimate the cost-space Pareto optimal frontier. A point (s, c) on this curve corresponds to the **CopyRight** layout of the candidate region

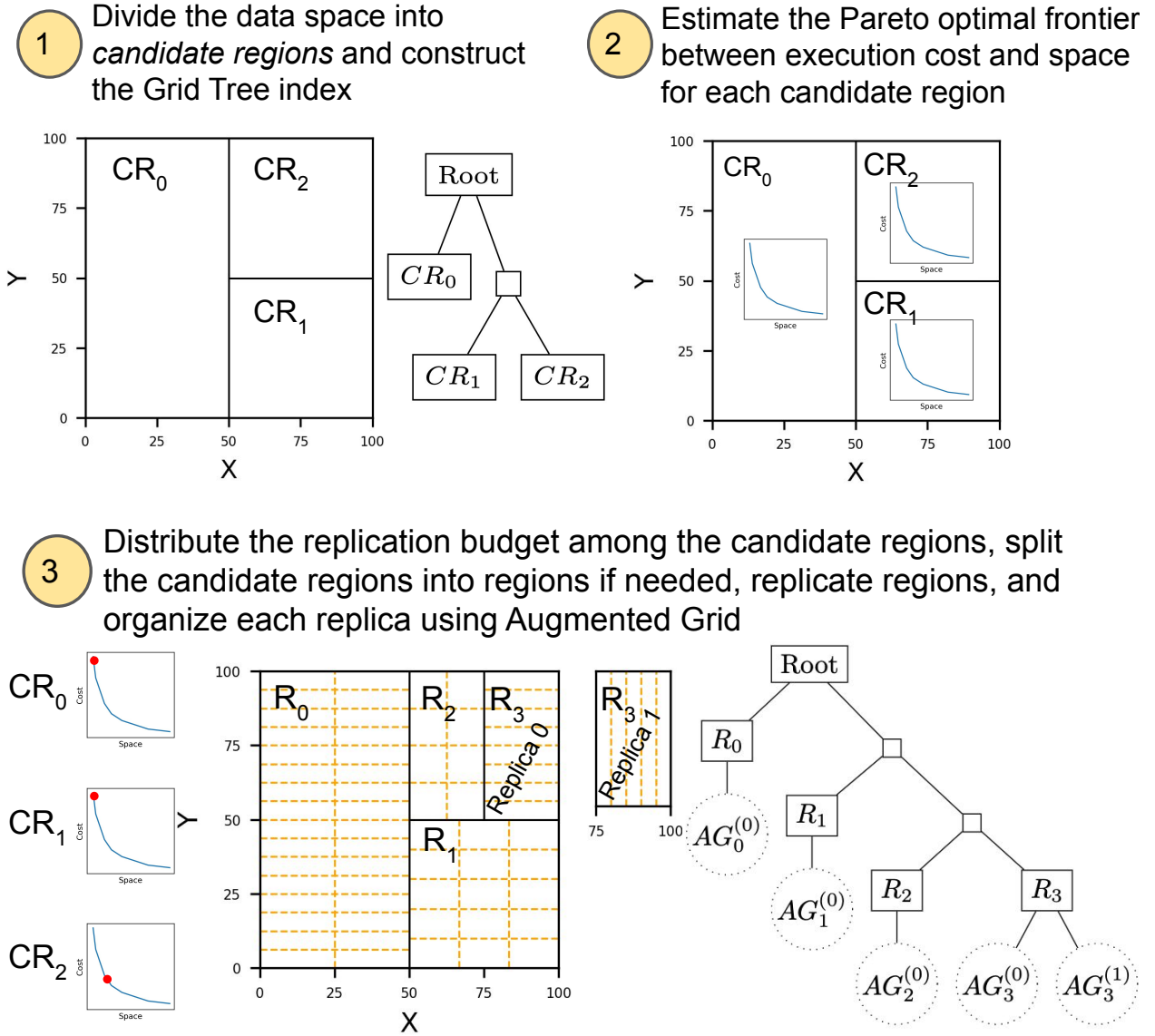


Figure 3.6: Overview of the layout optimizer with $0.125X$ replication budget.

that fits in the space budget s , where c is the cost of executing the workload on that layout. (Section 3.5.4)

3. We use each candidate region’s frontier to distribute the replication budget and obtain the layout of each candidate region. The layout of each candidate region combined with the Grid Tree that indexes the candidate regions forms the final CopyRight layout. (Section 3.5.5)

3.5.1 Clustering Queries

We approach replication of a region as a workload partitioning problem. Specifically, each replica is associated with a disjoint subset of the representative workload W that intersects the region. Each replica’s layout is optimized for its share of the workload and is used for executing queries that are similar to the ones in its subset. Enumerating all possible subsets of the workload is not scalable. To address this, as a pre-processing step, we cluster the queries into *types* and assign all queries of a type to the same replica. To have minimal impact on the quality of the final layout, queries with access patterns that are likely to benefit from the same optimal layout should be placed in the same cluster.

Augmented Grid partitions the data space into cells having equal number of points along each dimension. So queries having similar selectivity along each dimension are likely to benefit from similar layouts. For example, in Figure 3.2, all red queries are likely to benefit from similar Augmented Grid layouts whereas red and green queries have different access patterns and are likely to benefit from different layouts.

Queries with different templates are placed into different clusters. Queries from the same template with different selectivity characteristics should be placed in different clusters. For a d -dimensional table, we featurize a query as d dimensional vector where i^{th} entry in the embedding is its selectivity along dimension i . For each template, we group the queries into types by running a clustering algorithm on this query embedding. We use DBSCAN to cluster the queries into types. The epsilon parameter of DBSCAN can be tuned (0.2 worked well for our experiments). The overall performance was not sensitive to small changes in epsilon. The choice of clustering algorithm is orthogonal to our work. This algorithm is also used in [32] for computing workload skew. For the workload in Figure 3.2, if X is dimension 0, and Y is dimension 1, the red queries have embeddings close to $[1/4, 1/16]$, the green queries have embeddings close to $[1/12, 1/2]$, and the blue queries have embeddings close to $[1/8, 1/8]$. As a result, the clustering algorithm will group all queries of the same color into one type.

3.5.2 Replication Configuration

A replication configuration describes how any block (horizontal partition) of data is replicated. If a block is organized under a replication configuration, this means that the block is not further horizontally partitioned into smaller partitions that are replicated differently. The final goal of CopyRight’s layout optimizer is to find the optimal replication configuration for each *region*. In the search process, it will explore organizing a variety of blocks of data under a variety of replication configurations.

Formally, a replication configuration is a partitioning of query types. $RC = \{\{t_1, t_2\}, \{t_3\}, \{t_4\}\}$ is an example replication configuration for a workload W with 4 types, t_1, t_2, t_3 , and t_4 . Given a block of data, the replication configuration fully describes how the block is replicated. If the replication configuration has k clusters of query types, then the block has k replicas, one for each cluster. Each replica’s layout is optimized for queries of types in this cluster that intersect the block. For example, under RC , the block is replicated thrice (including the primary replica). The first replica optimizes its layout for queries of type t_1 and t_2 in W that intersect the block. The second and third replicas optimize their layout for queries of types t_3 and t_4 respectively. Each replica contains all tuples in the block. Each replica only materializes the columns that are required for answering the queries in its cluster. One primary replica always exists and materializes all columns.

The size of a block under a replication configuration is the total size required for materializing all replicas of the block specified in the configuration. The cost of executing queries over a block with a replication configuration is the cost of executing each query Q with type t_Q that intersects the block using the replica the configuration specifies for t_Q .

3.5.3 Candidate Regions

In this section, we describe how we partition the data space into candidate regions that are good candidates for replicating differently. We want candidate regions such that the cost-space Pareto optimal frontier of organizing the candidate region using **CopyRight** can be efficiently estimated.

The intuition that drives our approach is that two blocks of data are good candidates for replicating differently if they have different distributions of access patterns. Consider three different sub-spaces of data from the example in Figure 3.2. The left half $X < 50$ (block 0) has only one type of query. The bottom right quarter $50 \leq X$ and $Y < 50$ (block 1) has two types of queries: red and blue. The upper right quarter $50 \leq X$ and $50 \leq Y$ (block 2) has two types of queries: red and green. As these blocks have different distributions of access patterns, the benefit that can be obtained from replicating vary across these blocks. It makes them good candidates for replicating differently under different storage constraints.

The data space can be split into two blocks with different distributions of access patterns if a query type has different distributions in the two blocks, i.e., if the distribution of a query type is skewed. Our goal is to partition the data space into candidate regions to minimize the overall query skew, i.e, to horizontally split the data space such that the distribution of queries is as close as possible to uniform distribution for each type. We take a top-down approach hierarchically partitioning the data space so as to construct a tree index to efficiently index the candidate regions. We use the skew-tree algorithm from [32] to divide the space into “uniform” candidate regions and construct an associated Grid Tree. At a high level, the algorithm recursively divides a Grid Tree node into child nodes by picking a dimension and values to split the node that minimizes the total query skew. For the workload in Figure 3.2, the algorithm will split the data space into 3 candidate regions indicated by the black partitions in Figure 3.2c.

It is possible that two such candidate regions are replicated identically under same storage budget. It’s also possible that a candidate region needs to be further split in the final layout. For example, in Figure 3.2, if the storage budget for replication was an eighth of the size

of the table, then the upper right quarter region could be further split into two so that one of the children could be replicated twice. How to split such a candidate region depends on the space available for replicating that candidate region in the final layout. This cannot be solved independently for each candidate region.

The benefit of uniform regions: To estimate the cost-space Pareto frontier, we need to consider splitting a candidate region in all possible ways and replicating them under different replication configurations. We make an observation about uniform candidate regions and leverage that to efficiently estimate the cost of dividing the candidate region further and replicating them differently.

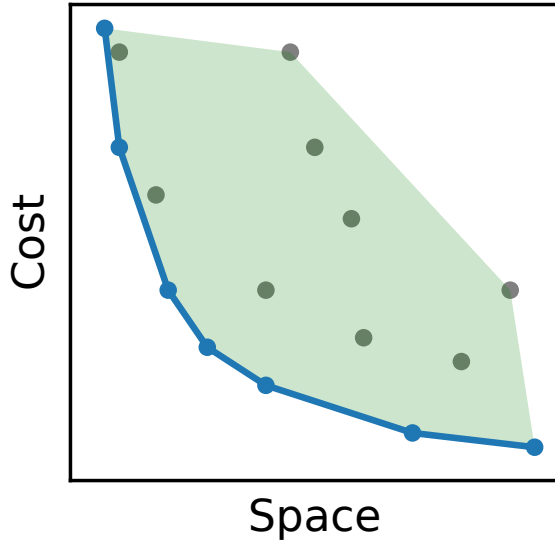
Consider splitting a uniform candidate region into two blocks along some dimension, one containing a fraction f of the data and the other containing the remaining $1 - f$. Let the block with fraction f of the data be replicated under configuration RC_1 . Let c_1 be the cost and s_1 be the space requirement for replicating the entire candidate region under RC_1 . Let the block with fraction $1 - f$ of the data be replicated under configuration RC_2 that has cost c_2 and space requirement s_2 for the entire candidate region. If the distribution of each query type in the candidate region was truly uniform, by the symmetry of the uniform distribution, the cost of the block with fraction f under RC_1 is $f * c_1$. Similarly, the cost of the block with fraction $1 - f$ under RC_2 is $(1 - f) * c_1$. Due to uniformity, the cost of the split configuration is: $f * c_1 + (1 - f) * c_2$. Similarly, the space requirement for the setup is: $f * s_1 + (1 - f) * s_2$. This can be extended to any number of splits, with the cost of further dividing a uniform candidate region into any number of blocks and replicating them under different configurations approximated by an affine combination of the cost of the corresponding configurations for the entire candidate region. We now describe how this can be used to efficiently estimate the cost-space Pareto optimal frontier.

3.5.4 Finding the Pareto Optimal Frontier

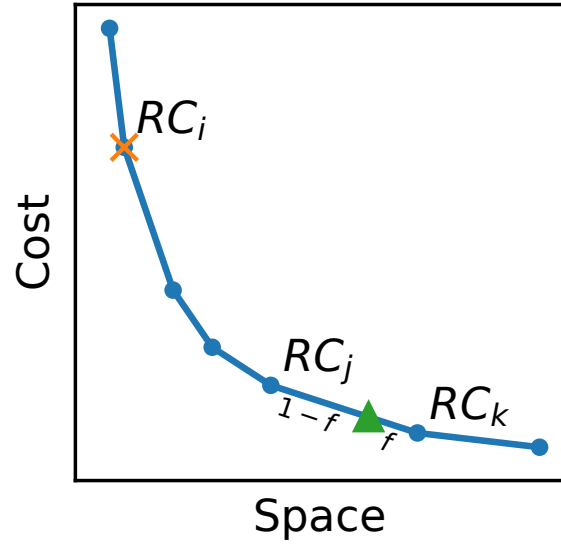
Now we describe how we estimate the cost-space Pareto optimal curve of laying out a candidate region under **CopyRight**. This is done independently for each candidate region. We use this curve to find the benefit from allocating a certain amount of space to replicating a candidate region while distributing the available space among candidate regions (Section 3.5.5). For the rest of the paper, we assume that the space available for replication is on the X axis and the cost of execution is on the Y axis. If (s, c) is a point on this curve, c is the cost of executing the workload on the candidate region organized using the optimal **CopyRight** layout that fits in the space budget s .

A candidate region may be replicated under a replication configuration. The dots in Figure 3.7a represent the cost-space pair for all possible replication configurations. A candidate region may also be split into smaller blocks that are replicated under different configurations. As discussed earlier, its cost/space can be approximated by an affine combination of the cost/space of replicating the entire candidate region under the replication configurations. The feasible space of all possible cost-space tuples is a convex polygonal hull as shown by the shaded region in Figure 3.7a. The blue edges of the polygon form the convex piece-wise linear Pareto optimal frontier.

Each vertex on the frontier corresponds to replicating the entire candidate region under a replication configuration. A point on an edge of the curve corresponds to dividing the

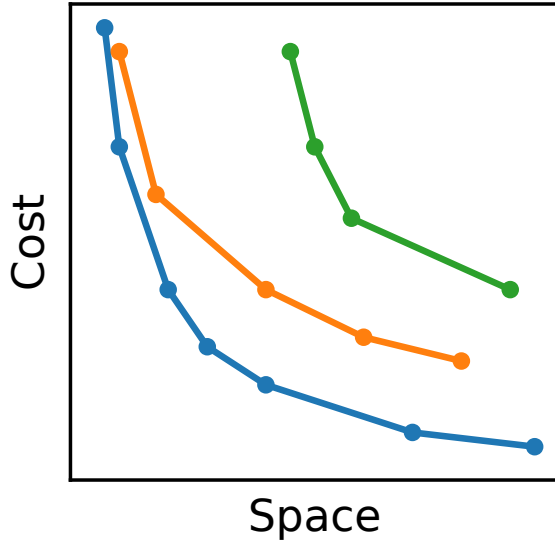


(a) Feasible space

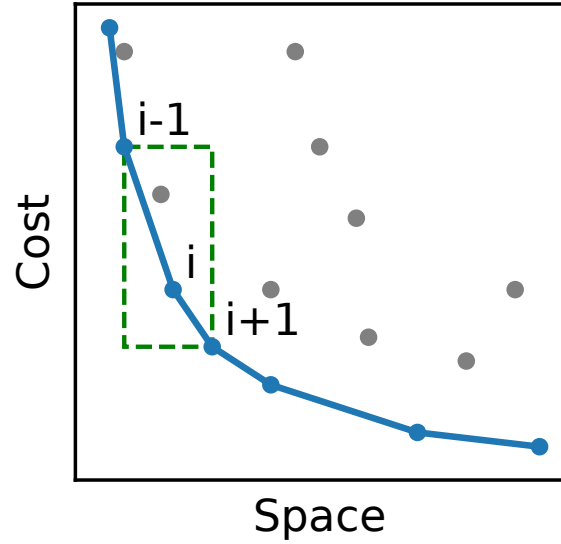


(b) Pareto optimal curve

Figure 3.7: Each dot in Figure 3.7a represents replicating a candidate region under a replication configuration. Shaded region represents partially replicated layouts. Blue lines form the Pareto frontier. In Figure 3.7b, the point $\times$ on the frontier corresponds to replication configuration RC_i . The point $\triangle$ on an edge corresponds to splitting the candidate region and replicating fraction f under RC_i and $1 - f$ under RC_k .



(a) Non-dominated sorting



(b) Crowding distance

Figure 3.8: Pareto Frontier estimation.

candidate region into two blocks and replicating them under configurations of the endpoints of the edge. For example, in Figure 3.7b the orange cross corresponds to replicating the candidate region under RC_i and the green triangle corresponds to splitting the space and replicating fraction f under RC_j and $1 - f$ under RC_k . This reduces the complexity of our

algorithm as we can search in the space of replication configurations to find ones on the Pareto optimal front instead of searching in the space of all possible partial replicated layouts.

Algorithm: We propose a greedy multi-objective optimization algorithm to find the configurations on the Pareto optimal frontier. We start at the rightmost point on the Pareto curve, the fully replicated configuration, i.e., the configuration with a separate replica for each query type. For a workload W with 3 types, this corresponds to $RC = \{\{t_1\}, \{t_2\}, \{t_3\}\}$. We iteratively generate and explore other replication configurations. While exploring the space, we maintain the Pareto optimal frontier of all configurations seen so far. This can be found by the finding configurations that are not dominated by any other partial replicated layout. Note that dominance is different from traditional multi-objective optimization problems in our setup. As any point on the line segment connecting 2 configurations is a feasible layout, they need to be considered while checking dominance. A point is non-dominated if there is no line segment connecting two configurations that dominates it. The iterative algorithm works as follows.

1. Generate offspring configurations from the current generation of configurations.
2. Evaluate the cost and space of all offspring configurations.
3. Select k configurations from the offspring as parents for the next generation.
4. Update the Pareto optimal frontier.
5. Repeat steps 1-4 until the termination condition is met.

In step 1, we generate an offspring configuration by merging 2 replicas in a parent configuration. A parent $RC = \{\{t_1\}, \{t_2\}, \{t_3\}\}$ will generate $\{\{t_1, t_2\}, \{t_3\}\}$, $\{\{t_1\}, \{t_2, t_3\}\}$, and $\{\{t_1, t_3\}, \{t_2\}\}$ as offspring. All offspring configurations have one less replica than the parent configuration.

The offspring can have varying sizes and costs which is computed in step 2. The space requirement of a merged configuration can be easily computed as the space required to materialize the relevant columns. Computing the cost of a merged replication configuration is time-intensive as it involves optimizing the layout of each replica in the configuration. We use an approximation as described later.

In step 3, we select k configurations from the offspring as parents for the next iteration. k is a tunable parameter. Picking configurations with the lowest cost or space can be suboptimal. In this multi-objective setting, we pick ones that push the Pareto frontier forward while maintaining diversity on the frontier by sorting them based on non-domination rank and crowding distance, as in [29].

We first compute the non-domination rank of all offspring configurations by organizing them into a series of non-intersecting non-dominated frontiers. For example, in Figure 3.8a the configurations are organized into three frontiers. The non-dominated blue frontier is assigned rank 0. The orange frontier is dominated by the one other frontier (blue), so it gets rank 1. Green is assigned rank 2. Smaller ranks correspond to better configurations.

To have a more uniformly spread-out Pareto frontier, we prefer configurations from less crowded parts. We calculate the crowding distance of a point as the average distance of two points on either side of this point along each of the objectives on its non-domination frontier as shown in Figure 3.8b. This can be thought of as an estimate of the density of configurations surrounding a particular point.

We first sort the configurations in the increasing order of the non-domination rank. Within the same non-domination rank, we sort the configurations in the decreasing order of crowding

distance. We then pick the top- k configurations as parents for the next iteration. We continue iterating until the only remaining configuration has a single replica that is responsible for queries of all types.

We do not discard all dominated configurations as they can eventually lead to better solutions. Consider the following configurations for a workload with four types. $RC_1 = \{\{t_1, t_2\}, \{t_3\}, \{t_4\}\}$, $RC_2 = \{\{t_1\}, \{t_2, t_3\}, \{t_4\}\}$, and $RC_3 = \{\{t_2, t_3\}, \{t_1, t_4\}\}$. Types t_3 and t_4 are more popular in the workload. RC_1 will likely have lower cost than RC_2 as the popular query types get their own replicas. So RC_1 can dominate RC_2 . RC_3 can be the optimal configuration under some storage budget but will not be considered if we discard RC_2 .

Cost Evaluation: Each replica is optimized for a disjoint subset of the workload. The cost of a replica is the cost of executing these queries on the grid layout optimized for them. The cost of a configuration is the sum of costs of each replica. Computing this involves optimizing the layout of each replica. We use a gradient descent style iterative algorithm from [32] to optimize the Augmented Grid layout parameters. Only the relevant columns of data and its share of the workload are used for optimizing the layout of a replica. The cost of executing a query is estimated using a cost model

$$cost = w_0(\#cell\ ranges) + w_1(\#scanned\ pts)(\#scanned\ dims)$$

A cell range is a set of adjacent cells in physical storage. $\#cell\ ranges$ is the number of such ranges that are scanned. $\#scanned\ pts$ is the number of tuples that are scanned and $\#scanned\ dims$ is the number of columns that are accessed by the query. w_0 and w_1 are weights that are learned. The first term in the cost model accounts for random accesses and the second term for sequential accesses. The features of this cost model can be efficiently estimated from the grid parameters and the number of tuples per grid cell. The latter is estimated from a sample of the data.

To avoid too many re-optimizations of the grid layout while merging replicas, we use a heuristic to approximate the cost of the merged replica. Let $c_{i,j}$ be the cost of executing the queries of replica i on the layout of replica j . The cost of the replication configuration is $\sum_{i \in replicas} c_{i,i}$. The cost of merging replicas in R into a new replica, m , is approximated as

$$c_{i,m} = \frac{\sum_{j \in RC_{j,j}} c_{i,j}}{\sum_{j \in RC_{j,j}}} \quad \forall i \neq m \wedge i \notin R \quad (3.1)$$

$$c_{m,i} = \sum_{j \in RC_{j,i}} \quad \forall i \neq m \wedge i \notin R \quad (3.2)$$

$$c_{m,m} = \frac{\sum_{i,j \in RC_{j,j}} c_{i,j}}{\sum_{j \in RC_{j,j}}} \quad (3.3)$$

Equation (3.1) estimates the cost of evaluating the workload of replica i on the merged replica m as a weighted average of the cost of executing queries of i on the layout of each replica in R . The intuition is that when we merge two workloads containing queries of different types the layout is optimized to be good on average. The layout will be biased in favor of the expensive set of queries. We use $c_{j,j}$ as the weight to mimic that. Equation (3.2) is the cost of evaluating the workload of m on the layout of i . As the workload of m is the union of the workloads of all replicas in R , the cost is the sum of cost of individual workloads. Equation (3.3) is a combination of Equation (3.1) and Equation (3.2).

We found that the above approximation worked well in our experiments. We tried other heuristics such as estimating the merged layout (instead of the cost) by taking a weighted average of the layout parameters (number of partitions along each dimension), merging the queries with the most similar access patterns, etc., but found they did not work as well for the datasets that we tested.

For the starting configuration with a replica for each query type, we train Augmented Grid for each replica and compute $c_{i,j}$ for all i, j . For other configurations explored, we approximate the cost as described above. The cost estimate gets stale as more nodes are merged into the same replica. The costs are recomputed by optimizing the layout when the number of merges exceeds a threshold.

3.5.5 Space Allocation

We now discuss how to allocate the space budget available for replication among the candidate regions and determine the final layout. Our objective is to find the operating point on each candidate region's cost-space Pareto frontier that minimizes the total cost of execution with the total size under the available budget.

The total cost of execution is the sum of costs and the total size of the layout is the sum of sizes of each candidate region. Each Pareto frontier is piece-wise linear and convex. So we can start at the left-most point on the Pareto curve in each candidate region and descend down the steepest edge of the curve until the replication budget is exhausted to find the optimal space allocation.

We index the points on the Pareto optimal frontier from left to right for all candidate regions. (s_0, c_0) is the leftmost point corresponding to the replication configuration with one replica for all query types. On each candidate region's Pareto frontier, we may move along the curve by taking a *full step* from (s_i, c_i) to (s_{i+1}, c_{i+1}) . This consumes $(s_{i+1} - s_i)$ space from the available budget. The benefit per unit of space this step is $(c_i - c_{i+1}) / (s_{i+1} - s_i)$. If there is not enough space to take a full step, then we may take a *partial step* from (s_i, c_i) to $(f * s_i + (1 - f) * s_{i+1}, f * c_i + (1 - f) * c_{i+1})$ where $0 < f < 1$, exhausting the budget. The benefit per unit of space is the same as a full step.

Our iterative algorithm works as follows: We initialize the operating point to (s_0, c_0) for each candidate region.

1. Pick the candidate region with the highest benefit per unit of space for the next step.
2. Take a full step on the candidate region's Pareto curve if there is sufficient remaining budget. Otherwise, take a partial step.
3. Iterate until space is exhausted or every candidate region is fully replicated.

After finding the optimal space allocation, if the operating point on the Pareto curve is on a vertex, then the candidate region is replicated using the configuration of that vertex. It's possible that the operating point is on an edge, in which case the candidate region is split into two and replicated under the configurations of the end points of the edge. A fraction of the data is replicated using the configuration of one end point and remaining fraction using the configuration of the other end point to fully utilize the available space budget. The Grid Tree node corresponding to the candidate region is split into two. As the candidates regions are nearly uniform as defined in section Section 3.5.3, we can choose an arbitrary dimension to split the node. For example, with enough storage budget to replicate an eighth of the

table, we will split the Grid Tree node corresponding to the upper right quarter into two and replicate one of the child nodes as in Figure 3.6.

At this point, we have the final Grid Tree and the replication configuration for each leaf node. If the layout for any replica in the final configuration was not already computed during the Pareto frontier estimation (because we approximated its cost or split the Grid Tree node after space allocation), we train the layout for each replica independently using its subset of the workload.

3.6 Query Execution

When a query arrives, we first identify the type of the query. We featurize the query into the same embedding used at training time for clustering queries. We classify the query by finding its nearest neighbor among the embeddings of the centroid of the clusters. We find the regions that intersect the query predicates using the Grid Tree index. At each leaf node we pick the best replica for executing the query. If the replica corresponding to the query type has all columns required to answer the query, we pick that. It’s possible that the replica does not have all columns required for executing the query because of misclassification of query type or if the query template was unseen during training. In that case, the query is answered from the primary replica. For each replica, we find all cells in the Augmented Grid that intersect the query filter. For each intersecting cell, we identify the corresponding range in physical storage using a lookup table. Finally, we scan all records within the range and emit the tuples that satisfy all query filters. The overhead of picking the replica to execute the query is minimal and does not offset the benefit from replication.

3.7 Evaluation

In this section, we present an experimental evaluation of **CopyRight**. Our experiments show that

1. Layout-aware replication improves query performance.
2. Partial and sub-table replication gives significant speedup over full table replication.
3. **CopyRight** is consistently superior to simpler alternatives.
4. The performance advantage of **CopyRight** scales with dataset and workload complexity.

3.7.1 Experimental Setup

We implemented **CopyRight** as a custom in-memory column store in C++. All attributes are stored as 64-bit integers. Any string and categorical values are dictionary encoded. We limit floating point values to a fixed number of decimal points and scale them by the smallest power of 10 that converts them to integers, as in [60]. Our implementation has a scan-time optimization: if the range of data being scanned is exact, i.e, all elements in the range satisfy the query filter, we skip applying predicates and scan only the columns that are required by downstream aggregation operators.

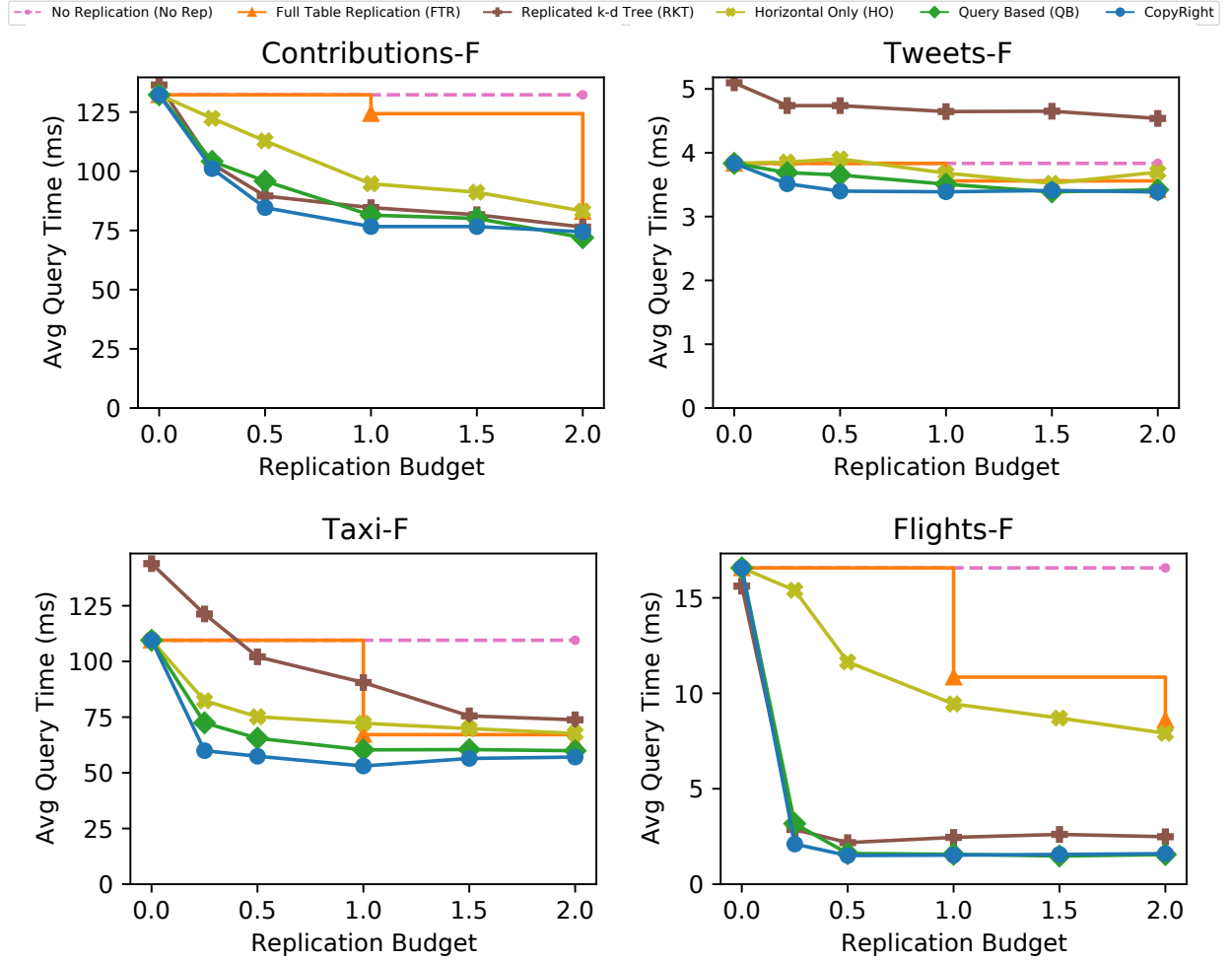


Figure 3.9: Performance of layouts at various replication budgets on full datasets.

We performed all experiments on an Ubuntu Linux machine with Intel(R) Xeon(R) Platinum 8275CL 3.00GHz CPU with 192 GB RAM. All experiments are single-threaded. We compare CopyRight to the following baselines implemented on the same column store.

No Replication (No Rep): We use Tsunami [32] as a layout-aware baseline that does not replicate any data.

Full Table Replication (FTR): We implemented a simple layout-aware replication scheme that replicates data at the granularity of tables. For a d -dimensional table, we embed the queries as a $2d$ -dimensional feature vector. The first d entries encode the selectivity of the query along each dimension. The remaining d entries are binary variables corresponding to the columns in the table to encode the query template. Variables corresponding to the columns accessed in a query are set to 1. If there is sufficient budget to replicate the entire table k times, we split the workload into k clusters using k-means clustering of the query embeddings. Each replica is responsible for queries in one of these clusters and is laid out using Tsunami, trained independently for each replica using data in the entire table and queries in its cluster. At execution time, we find the replica to run the query on by identifying the cluster with the most similar embedding.

Replicated k-d Tree (RKT): We split the space using a k-d tree [17] for identifying candidate regions for replication instead of constructing a Grid Tree using the algorithm in

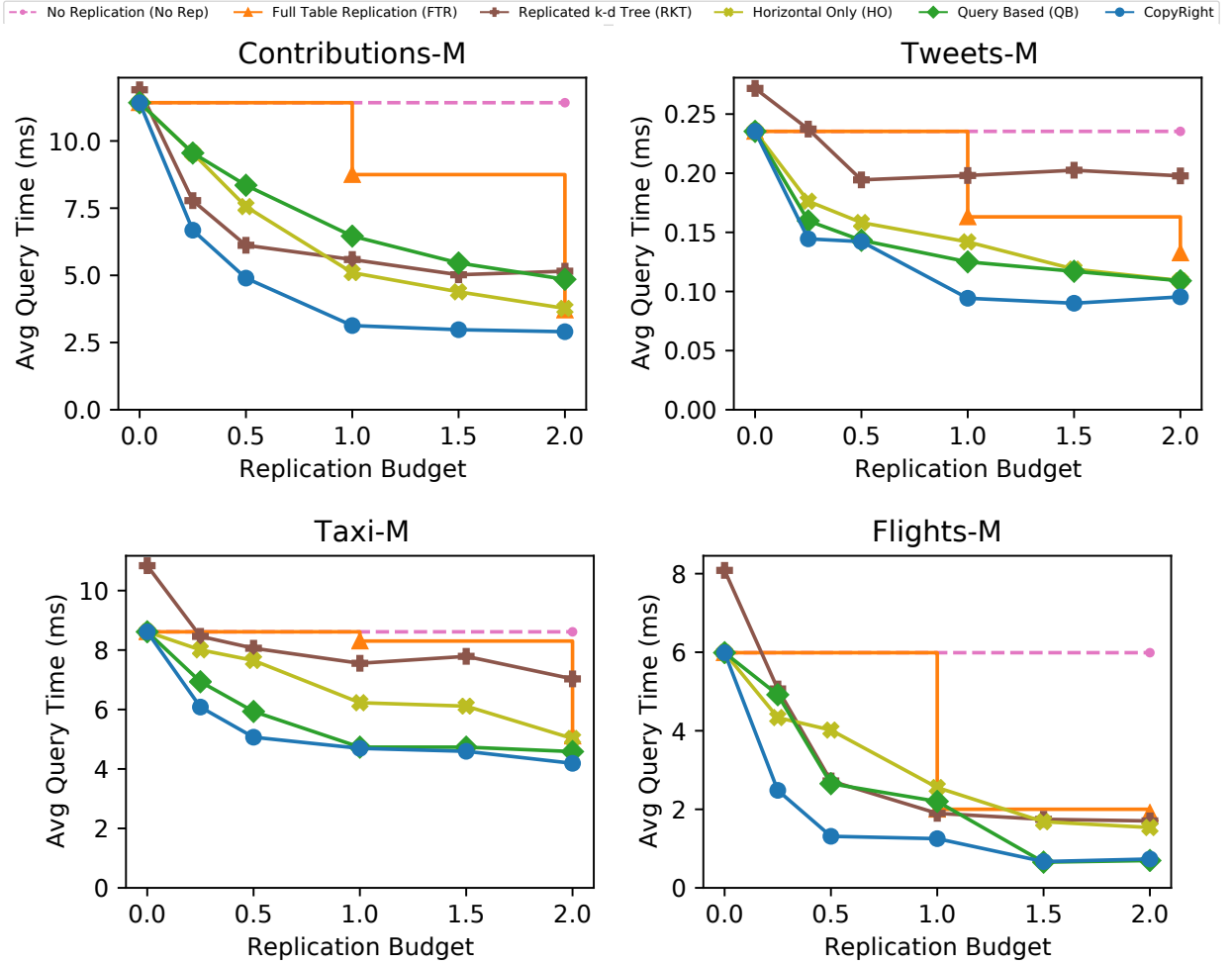


Figure 3.10: Performance of layouts at various replication budgets on modified datasets.

Section 3.5.3. The k-d tree recursively partitions space using the median value along each dimension, until the number of tuples in each leaf falls below a threshold. A threshold of 1% of the number tuples in the table worked well for the four real-world datasets that we tested on. The dimensions are selected in a round robin fashion, in the order of selectivity. Each leaf node of the k-d tree is replicated as in **CopyRight**. We use the same algorithm to solve for the replication configuration and corresponding layouts.

Horizontal Only (HO): We implemented a fine-grained replicated layout without sub-table replication. HO is same as **CopyRight** except that all columns in the table are materialized in every replica.

Query Based (QB): This baseline implements layout-aware partial replication including sub-table replication similar to **CopyRight**. But, it uses similarity between query templates as a proxy cost function for merging replicas in the algorithm described in Section 3.5.4. We use the cosine similarity between $2d$ -dimensional query embedding discussed above. Prior works on layout design have used distance between the binary encoding of query templates (the last d entries of our embedding) for clustering queries [15, 76].

We note that the latter three baselines are variants of **CopyRight** and use various aspects of the system. We discuss them in detail in the ablation study in Section 3.7.3. We do not compare against other non-learned layouts as Tsunami was shown to be superior over them

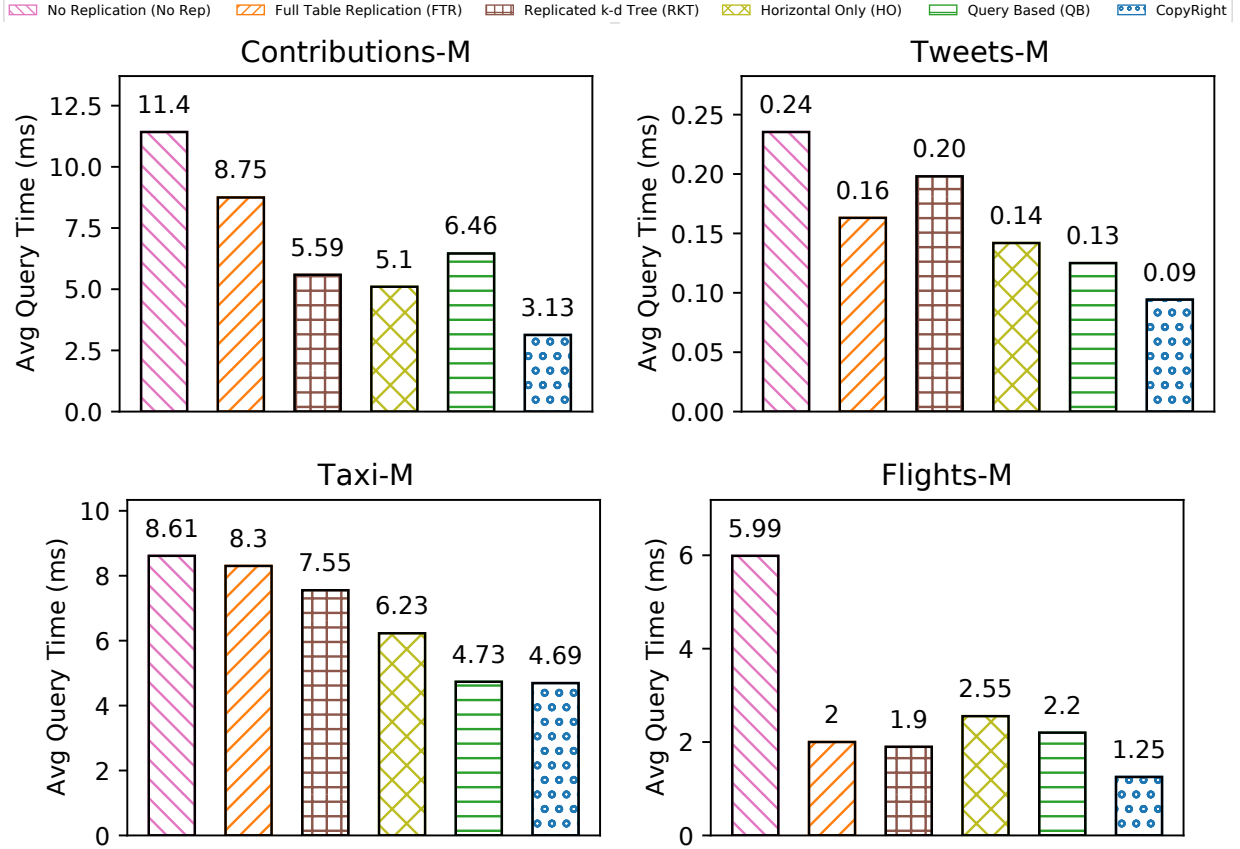


Figure 3.11: Performance of layouts at replication budget 1X.

[32]. We also don't compare against other learned multi-dimensional layouts as they do not adapt based on query distribution [28, 71, 92] or are optimized for disk [53, 95].

3.7.2 Datasets and Workloads

We evaluate the layout schemes using four real-world datasets and query workload traces from a dashboard-based interactive data analytics platform at OmniSci [65]. The dashboard has a variety of interactive knobs to change the templates and parameters of a query. This includes zooming in on maps, enforcing ranges on graphs, selecting from drop-down menus etc. These knobs vary across the datasets. The workload contains scan-oriented queries with multidimensional range and equality predicates. We dropped columns that are not used in any queries from all datasets.

For each dataset D , we use 2 versions of the workload, D -F and D -M, in our experiments. D -F contains the full workload. D -M is a modified workload where we dropped queries with selectivity over 10%. When queries scan a large fraction of the data, non-replicated layouts can achieve low scan overheads without incurring high look-up costs. High selectivity can thus delay the curse of dimensionality and the benefit from replication is sometimes lower. The workload traces we used are from OmniSci, a GPU based DBMS that is heavily optimized for scans, and many of their workloads include a number of large scans that we believe are not particularly representative of other analytics workloads. Prior work on analytics

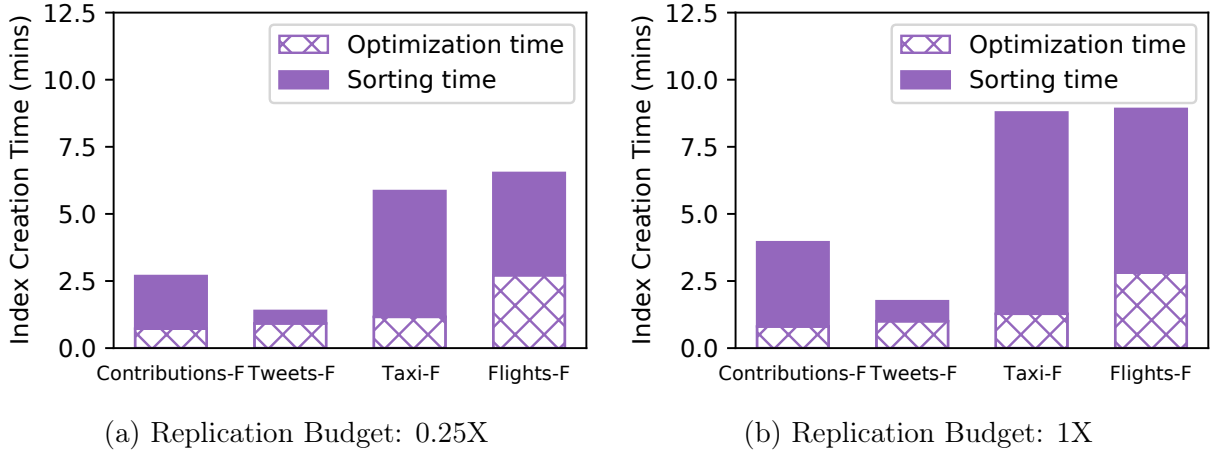


Figure 3.12: Index creation time across workloads.

Table 3.1: Characteristics of the datasets and workloads. Properties of the modified workload in parenthesis.

Dataset	Contributions	Tweets	Taxi	Flights
#tuples	86M	15M	175M	120M
#dimensions	9	16	13	21
#templates	13(13)	14(6)	20(15)	12(9)
#types	34(27)	16(8)	31(25)	18(13)
#regions	19(31)	46(28)	62(65)	46(57)
selectivity(%)	25.89(1.45)	14.36(0.08)	26.05(0.23)	15.97(0.38)

and partitioning from researchers with access to corporate workloads confirm that many real-world workload traces have low overall scan selectivity, e.g., 0.0005% and 0.0697% from Microsoft [95] and 0.3% from Conviva [87]. We use the modified workload to demonstrate the results on workloads with moderate to low selectivity. The characteristics of the datasets and workloads is summarized in Table 3.1. We now describe them in detail.

The **Contributions** dataset contains data about 25 years of political donations in the US. Columns include contribution amount, coordinates of contributor’s address, recipient details, etc.

The **Tweets** dataset contains details about tweets from November 2014. Columns in the table include date and time of tweets, latitude and longitude of the location, language, country, etc. We use one month’s worth of data from November 2014.

The **Taxi** dataset has records of yellow taxi trips in New York City in 2014. Columns in the table include pickup and drop-off times, pickup and drop-off coordinates, trip amount, tip, etc.

The **Flights** dataset has historic flight data in the US for 3 decades. Columns include date of the trip, day of the month, day of the week, source and destination coordinates, etc. The full Flights workload has a large fraction of `COUNT(*)` queries that benefit significantly from partitions that are subsumed by the query filters.

To study the behavior of algorithms by varying dataset parameters, we use synthetic datasets that are discussed in Section 3.7.4.

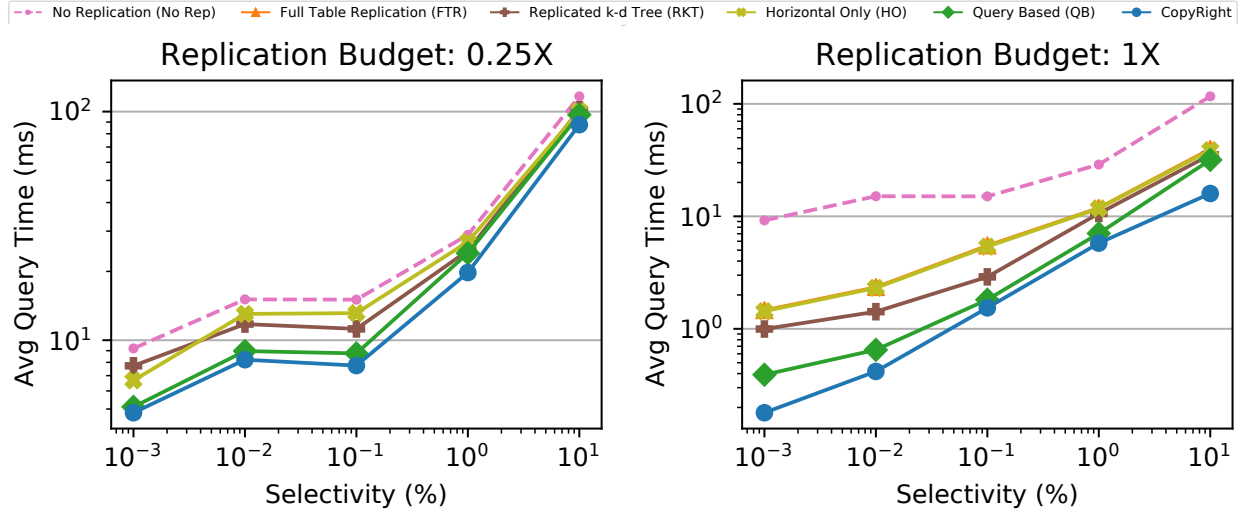
3.7.3 Overall Results

Figure 3.9 and Figure 3.10 show the performance of various layouts at different replication budgets. The replication budget is expressed as a fraction of the table size. A replication budget of 0 means there is no space for replicating any tuple. A replication budget of 2 means that there is sufficient space to create two additional full replicas of the table. Figure 3.11 highlights the performance at space budget = 1X.

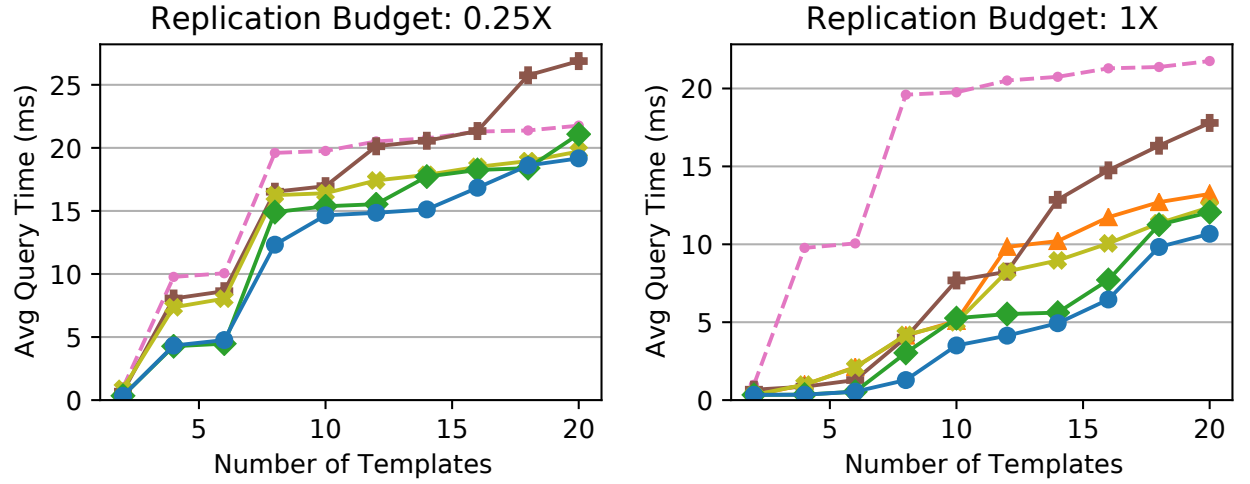
CopyRight is consistently better than all baselines at all budget points across datasets and workloads. At 0.25X space overhead, **CopyRight** attains 1.1X to 7.9X faster performance than non-replicated layout and the same or up to 5.2X faster performance than full table replication on the raw workloads. On low selectivity workloads, with 0.25X replication budget, **CopyRight** is 1.4X to 2.4X faster than No Rep. Compared to FTR with 100% overhead, **CopyRight** achieves 80% of the performance to 1.4X faster performance with just 25% space overhead. With 1X overhead, our replication engine attains 1.8X to 4.8X speedup over No Rep and 1.6X to 2.8X speedup over FTR for low selectivity workloads. At very high or very low replication budgets, the relative advantage of **CopyRight** over other baselines is low. At high budgets, with sufficient space to create a separate replica for frequent or expensive query types, finding a good layout is easy and at low budgets, there is limited opportunity to improve. While **CopyRight** achieves the same or up to 2X faster performance than the next nearest baseline, no single baseline dominates others across all datasets. FTR, HO, RKT, QB are at least 7.1X, 7.8X, 2.1X, and 2.1X slower than **CopyRight** at some replication budget in at least one of the eight workloads.

Ablation Study: Layout-aware replication boosts query performance compared to non-replicated layouts as all replicated layouts achieve significant speedup over No Rep. FTR is up to 3X faster than No Rep with 1X additional space requirements. **CopyRight**'s performance advantage comes from focusing replication efforts in parts of the data that gives maximum benefit. By focusing replication efforts in sub-spaces of the data, HO, **CopyRight** without sub-table replication, attains up to 1.7X better performance than FTR at 1X replication budget. With sub-table replication, **CopyRight** is able to better utilize the replication budget by materializing a subset of columns in each replica, achieving up to 7.7X faster performance than HO. By being layout-aware while grouping query types to replicas, our engine achieves up to 2.1X better speedup than QB which groups queries based on similarity of query templates. **CopyRight** performs up to 2.6X better than RKT, by identifying the correct granularity of replication and focusing efforts there.

Optimization Costs: **CopyRight**'s layout optimizer took under 3 minutes to find the layout for all four datasets. Data reorganization completed in less than 7.5 minutes with 1X overhead. Figure 3.12 shows the layout optimization and data reorganization costs. The layout optimizer has a peak memory usage of 225.08MB for Contributions, 76.86MB for Tweets, 436.45MB for Taxi, and 332.78MB for Flights. This includes the 1% sample of the data that the optimizer uses.



(a) Selectivity



(b) Number of templates

Figure 3.13: CopyRight’s performance across workloads with varying dataset and workload parameters.

3.7.4 Scalability

To demonstrate the scalability of our approach we compare **CopyRight** against all baselines by varying different parameters of the workload. We use a synthetic dataset with 8 columns. Each column is independently uniformly distributed. All queries use 2 columns for predicates and perform a **COUNT** aggregation. The filter columns in each template are randomly generated. Unless specified otherwise, we use a dataset with 100M tuples and a workload with 8 equally represented query templates uniformly distributed in the data space. In each experiment, we re-optimize **CopyRight** and baselines for each dataset/workload configuration.

Selectivity: We vary the selectivity of the workload from 0.001% to 10%. Figure 3.13a shows that performance of **CopyRight** scales with selectivity. The relative advantage of **CopyRight** over other layouts decreases at higher selectivities as all layouts incur low scan overhead with large query rectangles.

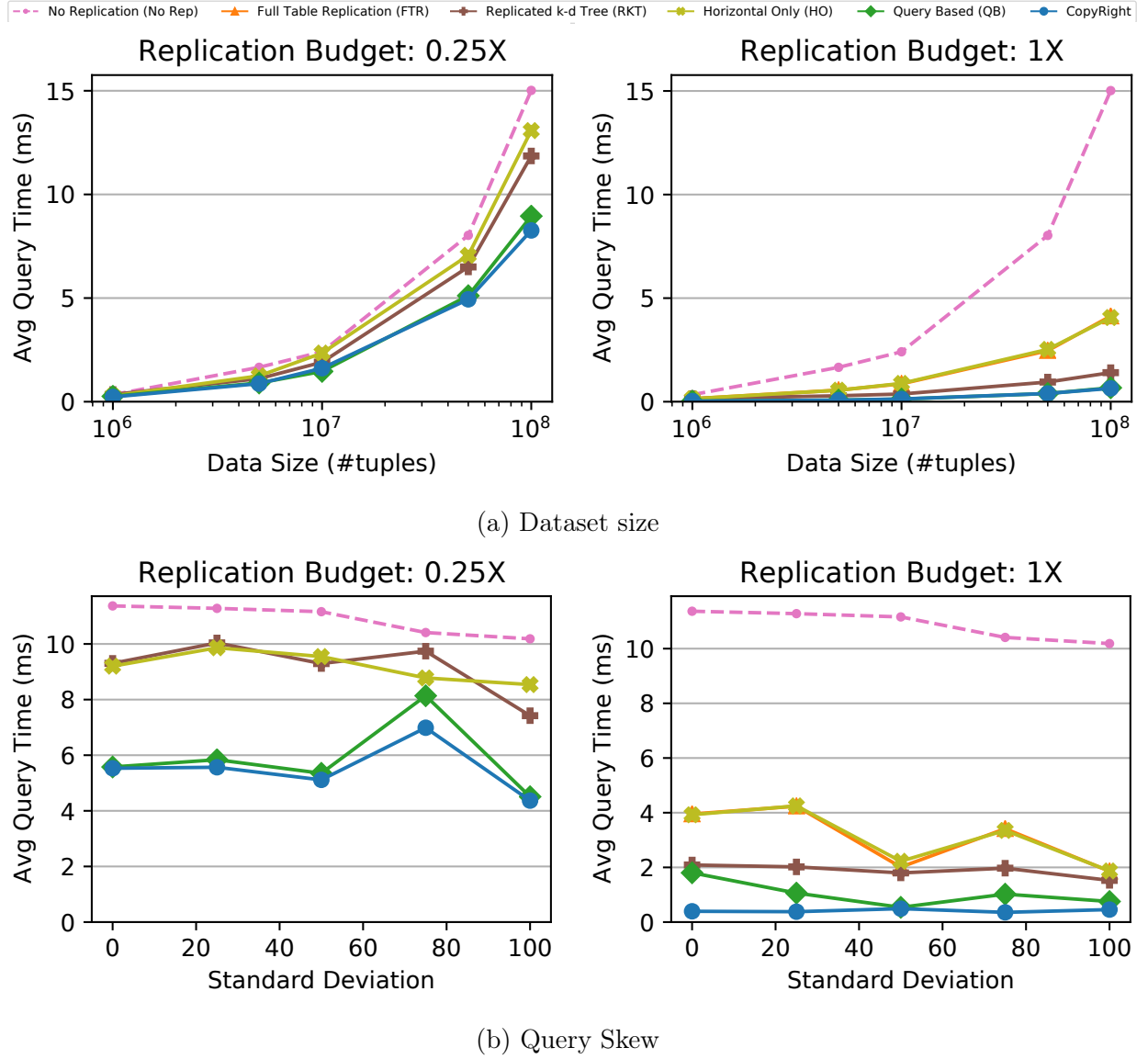
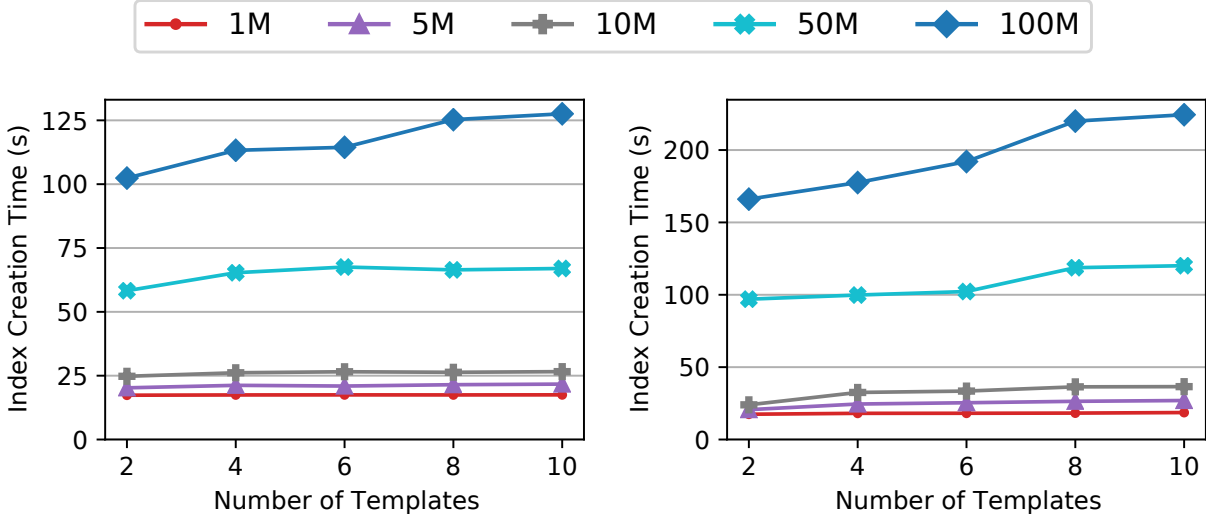


Figure 3.14: Copyright's performance across workloads with varying dataset and workload parameters.

Number of templates: We vary the number of templates from 2 to 20. The average selectivity of the workload is 0.1%. The benefit from replication initially increases, as we can create replicas optimized for each template. Further increase in the number of templates eventually decreases the relative benefit, as more templates get assigned to each replica. Figure 3.13b shows that Copyright is able to delay the curse of dimensionality by replicating data.

Dataset size: We vary the number of tuples in the table from 1M to 100M. The average selectivity of the workload is 0.01%. Figure 3.14a shows that Copyright maintains its performance benefit across datasets with different sizes.

Query skew: We vary the distribution of templates in the workload. The number of queries of each type is sampled from a normal distribution with mean 200. We vary the standard deviation from 0 to 100. The average selectivity of the workload is 0.01%. Each



(a) Replication Budget: 0.25X

(b) Replication Budget: 1X

Figure 3.15: Index creation time with workload complexity.

point on the x-axis in Figure 3.14b corresponds to a workload with a skewed distribution of queries. The performance gain depends on the similarity of query templates and their relative distributions. **CopyRight** maintains its performance advantage across workloads with varying query skew.

Optimization Time: We study the impact of varying data and workload complexity on the index creation time. We vary the number of templates in the workload from 2 - 10 and use datasets with 1M to 100M tuples. The average selectivity of the workload is 0.1%. Figure 3.15 shows that the index creation time increases slowly with number of query templates and fast with dataset size as the data sorting time dominates layout optimization time.

3.8 Related Work

Layouts: DBMSs often use horizontal, vertical and hybrid partitioning to reduce the amount of data accessed during query processing [13, 15, 24, 83, 73, 33, 40, 16]. To accelerate the performance of multi-dimensional queries, data is often laid out as multi-dimensional clustered indexes [62, 17, 39, 81] or using specialized sort order [81, 63]. See [66, 79] for a survey. Kraska et al. introduced the idea of learned indexes [48] and recent work has extended it to multi-dimensional layouts. The most relevant to our work is Flood [60] and Tsunami [32], layouts optimized for in-memory databases. Tsunami, described in this paper, builds on top of Flood. [53, 95, 38] learn layouts for disk-based systems. [28, 71, 92] adapt layout based on data distribution but not query distribution. Array DBMSs organize large arrays by partitioning them into sub-arrays called chunks or tiles. Prior work has studied various strategies for chunking [68, 75, 82, 97]. Chunking is also studied in multi-dimensional databases for computing data cubes for accelerating aggregation queries [98, 43]. **CopyRight** and these works share the idea of optimizing layouts by learning from data and query distributions. None of these works consider partial or full replication of data while optimizing

their layouts. [76] proposes maintaining copies of multi-dimensional arrays, each chunked differently, but does not consider partial replication. Qd-tree [95] constructs a decision tree to partition data to reduce disk-based block accesses. They briefly discuss the possibility of incorporating table-level replication by simultaneously learning 2 trees, but does not give any details.

Replication: Joint optimization of partitioning and replication has primarily been studied in the context of distributed databases. These approaches focus on minimizing the number of cross-partition queries and are agnostic of the exact layout of data in each partition. [41, 55] only supports horizontally splitting a table into a fixed number of shards (number of nodes in the cluster) based on one attribute and fully replicating a table at nodes. Schism [26] takes a graph partitioning approach and NashDB[57] uses an economic model based approach to handle fine-grained replication, but their methods are tightly coupled to the cost model (number of cross-partition queries) and cannot be easily extended to a layout-dependent cost model. These systems assume that all replicas are equally good for answering a query. [51] supports partitioning each replica by a different key but only at table level. The complexity of layouts and performance that can be gained is limited by the fault tolerance and recovery requirements in distributed databases [80]. Our work is for in-memory single node setup and focuses only on performance. Ramamurthy et al. makes the case for employing different storage models (column-store/row-store) in different full replicas of the table [72]. Our work goes much beyond this in replicating parts of the data and considering more complex layouts for each replica.

Automated Physical Database Design: There is a rich corpus of work on automatically tuning a database and its physical design that focuses on selecting structures like indexes and materialized views for optimizing query performance [10, 22, 69, 12, 21]. Arguably, each partial replica of a table can be expressed as a materialized view. However, materialized view selection algorithms explore the space of syntactically relevant views, focusing on creating logical views that cover queries, whereas we focus on replicating tuples themselves and on optimizing the physical layout of replicated regions [99, 12, 20]. CopyRight can be used to partially replicate materialized views. The most relevant to our work is integrating horizontal and vertical partitioning with index and materialized view selection [13] which explores a large space of layouts like CopyRight. They consider only single-column range and hash partitioning.

3.9 Conclusion

We designed and implemented CopyRight, a layout-aware partial replication engine that replicates parts of the data that matters to maximize overall query performance. CopyRight achieves 1.1X to 7.9X speedup over the best non-replicated layout with 0.25X overhead across a range of real-world workloads. At 25% space overhead CopyRight attains the same or up to 5.2X better performance than full replication of the table that has 100% space overhead.

Chapter 4

Self-Organizing Data Containers

4.1 Introduction

Cloud object stores combined with log-structured tables (LSTs) have become the de facto storage standard for modern cloud-based analytical data processing. In particular, columnar storage formats like Apache Parquet [6] and ORC [5] have been extensively adopted for efficient read-heavy analysis workloads. These formats design data files to be immutable once created, optimized specifically for efficient columnar scans. However, modern analytical workloads frequently require incremental updates to tabular data to quickly incorporate new information and derive timely insights. LSTs effectively meet this requirement while retaining the advantages of immutable columnar storage by layering metadata on top of immutable columnar files to maintain multiple concurrent table versions with support for ACID transactions. Several implementations of LSTs have emerged, notably Delta Lake [3], Apache Iceberg [2], and Apache Hudi [1].

Despite their broad adoption, existing Log-Structured Tables (LSTs) often exhibit sub-optimal query performance due to ineffective data layouts. Cloud storage disaggregation, while highly scalable and cost-effective, exacerbates performance challenges because cloud storage—such as Amazon S3—has lower bandwidth and higher latency compared to local storage. For example, disaggregated query engines like Amazon Redshift Spectrum running TPC-H benchmarks load data directly from S3 files and can be significantly slower (at least $8\times$) than tightly integrated storage-query execution systems such as Redshift [88]. Effective data layouts that reduce data movement by indexing, and physically organizing data is needed to mitigate these issues.

Recent studies have proposed *instance-optimized layouts* [95, 31, 32, 85], which leverage richer metadata to achieve significant performance improvements over basic layouts like range partitioning or Z-order. By collecting more metadata and employing expressive partitioning schemes tailored specifically to datasets and workloads, these layouts efficiently meet specific analytical requirements. Although such instance-optimized strategies deliver strong initial performance benefits, they currently assume static workloads and stable data distributions. However, real-world analytical workloads are continually evolving, with data frequently being appended, updated, or deleted, and query patterns shifting due to changing business needs and user interests. As a result, a data layout that was initially optimal often becomes

suboptimal over time, causing performance degradation. To maintain consistent query performance, instance-optimized layouts must adapt incrementally in response to changing data and workloads. Unfortunately, existing instance-optimized techniques lack support for such incremental, ongoing reorganization, significantly limiting their effectiveness in practical, long-running analytical deployments.

Consider dashboards analyzing New York City taxi data, where query patterns significantly vary by taxi type. Queries analyzing green taxis (`taxi_type = 'Green'`) often filter by borough (e.g., `pickup_borough = 'Queens'`), while those analyzing yellow taxis (`taxi_type = 'Yellow'`) typically filter based on airport pickups such as `pickup_location = 'LaGuardia'` or `'JFK'`. Organizing data into expressive layouts optimized specifically for these recurring predicates, along with maintaining indexes and metadata, significantly accelerates query execution. If electric taxis (`vehicle_type = 'electric'`) are subsequently introduced, query workloads shift toward filtering explicitly for electric cabs. Rather than fully reorganizing the entire dataset—which is costly and unnecessary—data layouts should adapt incrementally, selectively reorganizing recently inserted data and partitions impacted by the emerging query filters, ensuring efficient queries throughout workload changes.

Existing LSTs, such as Databricks Delta Lake [3], Apache Iceberg [2], Apache Hudi [1], and S3 Tables [9], support only basic data layouts, such as range partitioning and Z-order sorting, to enable data skipping during query processing. These systems neither support more sophisticated data layouts nor collect the rich statistics and metadata needed to adopt them. Moreover, users must explicitly specify sorting and partitioning criteria, as these systems do not automatically optimize data layouts based on actual data distributions or workload patterns.

To this end, we propose a cloud-optimized storage layer called Self-Organizing Data Containers (SDCs) [56]. By *self-organizing*, we mean that the container has the flexibility to take on many possible layouts and that it adapts to the clients’ workload as they interact with it. Specifically, by tightly coupling data with rich metadata, including distributional statistics, indexes, and workload access patterns, SDCs naturally contain the information required to support efficient data access. Most cloud storage systems view data objects as immutable; however, they confound the immutability of the logical contents of blocks (i.e., the set of records stored in each file) with the physical layout of data (i.e., whether records are column-oriented, sorted, etc.). In contrast, SDCs support multiple physical representations within a single data object, and those layouts can be transformed and adapted over time as the access patterns shift. Thus, even though the underlying logical data may remain immutable, SDCs physically reorganize over time, continually “self-organizing” into optimal layouts.

We propose a new self-organizing, self-optimizing, metadata-rich table storage layer over data stored in cloud object stores, called a self-organizing data container (SDC), that enables order-of-magnitude performance improvements in data-intensive applications through instance-optimization, i.e., the adaptation of data representation to exploit both the distribution of the data and the workload operating on it. SDCs capture both data and metadata, like access histories and distributional statistics, and are designed to be flexible enough to encompass a variety of modern high-performance representations for data analytics, including partitioning, and indexing.

We implemented SDCs as an extension of Delta Lake, enhancing it to support more

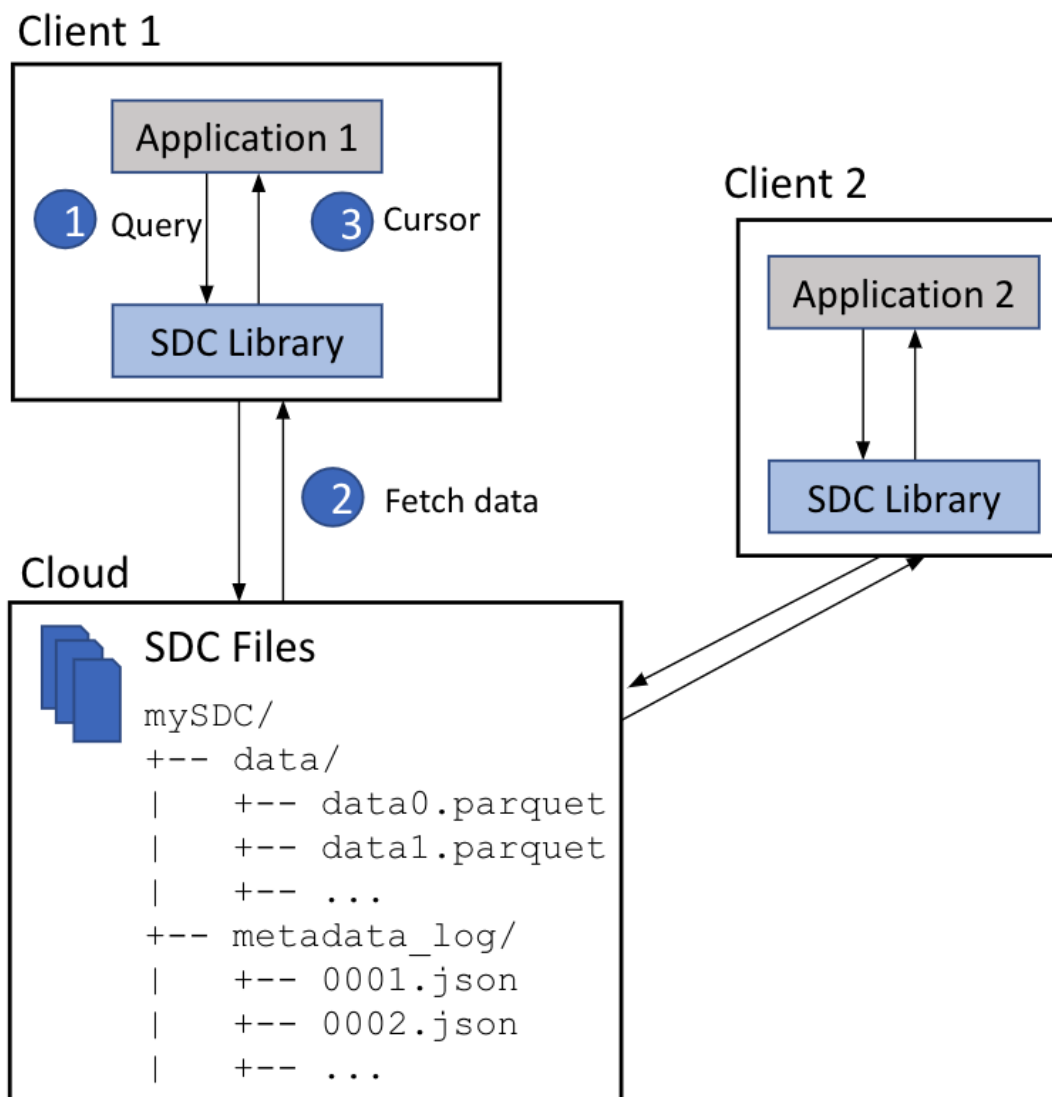


Figure 4.1: SDC organizes data and metadata as immutable files on cloud object stores. Clients access data through the SDC library, which is integrated into their application code and exposes a standard table API. This API supports scans with column projections and filter pushdowns, inserts, updates, and deletes. SDC library guarantees ACID semantics, enabling multiple clients to safely interact with the same data concurrently.

advanced multi-dimensional data layouts, specifically leveraging **Pando**. Delta Lake provides an **OPTIMIZE** method for compacting small files and sorting data using user-defined columns. We extend this to automatically compact and organizes data into optimized **Pando** layouts guided by observed data distributions and workload characteristics. Furthermore, our implementation incrementally adapts layouts as these distributions and query patterns shift over time.

4.2 Background

In this section, we provide background on Delta Lake, as SDCs are built on top of it. Delta Lake is an open-source ACID table storage layer designed over cloud object stores. It stores data as immutable Parquet files on cloud object stores such as Amazon S3. Delta Lake maintains information about which files belong to a table in an ACID manner, using a write-ahead log that itself resides in the cloud object store. This design enables clients to update multiple objects simultaneously, replace subsets of objects, and achieve serializable transactions, while maintaining high parallel read and write performance for the data objects themselves. All of the metadata is stored within the underlying object store, and transactions are achieved using optimistic concurrency control protocols against the object store. No persistent servers are required to maintain the state for a Delta table. Users only need to launch compute resources when running queries, enabling flexible and independent scaling of compute and storage.

Each write operation atomically appends to its transaction log by creating a new file for the operation corresponding to a new *version* of the table. When inserting data, clients write new Parquet files and append corresponding metadata to the log. Update and delete operations use a copy-on-write mechanism, transactionally adding new modified data objects and marking previous objects as removed, enabling full ACID semantics. Concurrent write transactions leverage optimistic concurrency control protocols to ensure serializability.

The log files are periodically compacted into checkpoints, enabling metadata searches that are orders of magnitude faster than directly listing files in the object store. The transaction log also stores metadata, including min/max statistics for each data file, in addition to the list of files. To perform reads, Delta Lake clients first read the log to quickly identify relevant data files in the latest version based on metadata such as min/max statistics. Queries then selectively access these Parquet files, efficiently skipping files whose data ranges do not intersect the query filters.

To achieve high data skipping for queries, Delta Lake supports clustering data using techniques such as multi-dimensional Z-ordering and Hilbert curves. It provides a **OPTIMIZE** command that clusters and compacts data files for improved query performance. By default, this command merges numerous small Parquet files, clusters the data based on the specified columns, and writes it into fewer larger files, enhancing read performance. Delta Lake executes these clustering and compaction operations transactionally, without impacting concurrent reads or writes. Applying **OPTIMIZE** with Z-order requires rewriting the entire dataset. Managed version of Delta Lake also supports liquid clustering that incrementally reclusters newly inserted data using the Hilbert curve. Currently, users must manually specify the clustering columns for all supported data layouts. The system does not automatically

select or evolve data layouts based on query workloads or data distribution changes. Further, manually changing the clustering columns requires rewriting the complete dataset.

4.3 SDC Overview

SDC is designed as a library that can be integrated as the storage layer into other applications. It provides a standard table API over data that is managed as log-structured tables (LSTs) on top of cloud object stores, while preserving full ACID guarantees. It exposes the same read and write API as popular LST frameworks, including Delta Lake, Iceberg, and Hudi. Specifically, our current implementation is built on top of Delta Lake. Unlike existing systems, SDC supports more sophisticated data layouts, specifically the **Pando** layout. Data layout optimization is done independently for each table. SDC does not support join-induced predicates for partitioning data using **Pando**. This is done because existing LSTs only support pushing down filters on the table that is being scanned. Join-level information is not available at the storage layer. Users need not specify clustering columns; the data layout is automatically selected based on the data distribution and workload characteristics.

We extend the **OPTIMIZE** operation to enable compaction and incremental clustering of both new and existing data, updating layouts as workload or data distributions evolve. SDC reorganizes data layout incrementally at the granularity of logical units called *slices* to minimize data reorganization overhead. Each slice contains the files generated as the output of a data layout optimization of **Pando**. Each slice has a separate partition-tree index. We will discuss how to decide which files belong to which slice in Section 4.4. Slice-level metadata, including file information and indexes, is stored in the transaction log. Slices are maintained at a target size that strikes a balance: sufficiently small to minimize reclustering overhead, but large enough that indexing remains effective and overhead from managing slices is controlled. As new data is appended, an incremental **OPTIMIZE** reorganizes it into new slice(s) based on **Pando**. Slices that become too small from DML operations are merged to maintain the target size. SDC maintains workload statistics at the slice level. When the workload distribution changes significantly, the layout is reoptimized at slice granularity using these statistics.

In SDCs, slices are managed independently to ensure scalability. This design localizes layout decisions and metadata management, enabling incremental reorganization without coordination across the entire dataset. In contrast, **CopyRight**'s replication strategy requires a global view to allocate replication space effectively. For example, if one slice's workload shifts toward queries with little replication benefit, its budget could be reallocated to another slice where replication offers greater gains, even if that slice's workload is unchanged. A Fixed per-slice budget cannot capture such cross-slice trade-offs, making it suboptimal for dynamic workloads. Implementing replication effectively in this setting would require a global policy for allocating storage across slices that scales to large datasets. We leave this integration of **CopyRight** into SDCs as future work.

In this work, we do not address the problem of deciding when to invoke the **OPTIMIZE** operation. This is because **OPTIMIZE** is resource-intensive. As SDC is designed as a library integrated into other applications, we make the design decision to allow applications full control over scheduling to ensure predictable and manageable performance characteristics. Applications may choose to run **OPTIMIZE** during downtime, periodically, or based on specific

business logic. Alternative approaches, such as performing reorganization implicitly as side effects of reads or writes, would introduce unpredictability and complicate application design. Managed versions of LSTs typically schedule optimization operations either periodically or based on automated scheduling frameworks such as AutoComp [37], which can easily integrate our enhanced OPTIMIZE method. Next, we describe how SDC supports dynamic layout reorganization in more detail.

4.4 Self-Organization

4.4.1 Static Data and Workload

First, we discuss how SDC works when the data is static and the query workload is fixed. SDC uses **Pando** as its data layout. The number of partitioning trees is set to 1 by default. Other instance-optimized layouts that can partition data based on a given dataset and workload can also be used instead of **Pando**. To support instance-optimized layouts, we need access to a sample of the workload that is run over the data. We assume that the recent history of workload is representative of the future workload that will be run on the data. We extend Delta Lake to periodically log filters pushed down from read queries. In , we assume queries are logged at a predefined path in cloud object storage. We will discuss more details on collecting workload statistics in later sections.

We consider the scenario when the data is static and the user explicitly executes **OPTIMIZE** to reorganize the entire dataset. Upon executing **OPTIMIZE**, we apply the **Pando** algorithm to create partitioning trees and repartition data into new Parquet files. The resulting tree structures are serialized and stored in a new index file at a specific cloud object store path tied to the commit version of the **OPTIMIZE** command. A corresponding commit entry is added to the Delta Lake transaction log recording the new files, their metadata, and the index file path.

At query time, the relevant index file for the version is retrieved from the transaction log. It is used to get the list of files to scan while skipping ones that do not intersect with the query filter according to the partition-tree index. We also note that the existing min/max metadata stored in transaction logs remains available and can be used for skipping. It will be effective at skipping for simpler workloads that do not involve complex predicates.

4.4.2 Dynamic Data

We now discuss how to optimize data layouts when the underlying data is dynamically changed. When data is being inserted or appended, it would be significantly expensive to read and rewrite all data files whenever new files are added. Instead, we incrementally reorganize only a subset of the data at-a-time, thereby minimizing the overhead.

At a high level, data files are grouped into logical units called *slices*. Each slice maintains its own partition-tree index, and the files in a slice are the result of one **Pando** data layout optimization. We maintain slices close to a predefined target size, ensuring that each slice remains above a configurable minimum threshold. We use 150 GB as the target slice size and

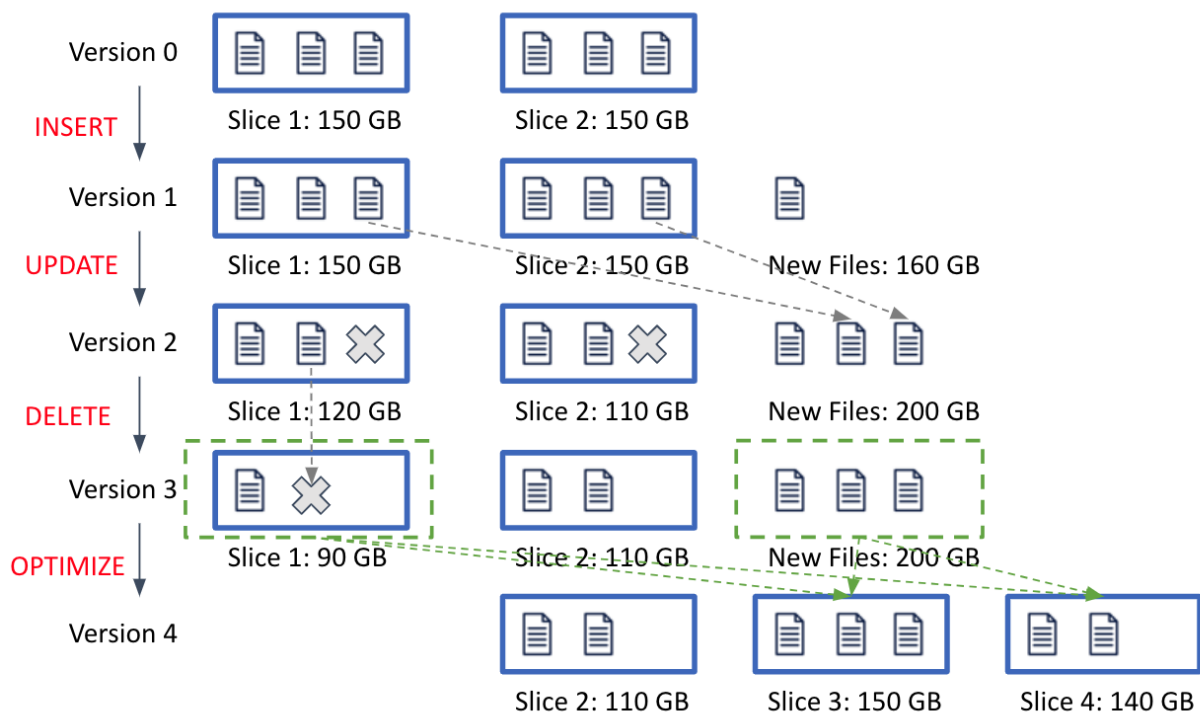


Figure 4.2: An illustrative example of how SDC handles dynamic data and incremental re-optimization.

100 GB as the minimum slice size. These are the values used for a similar mechanism for bin-packing compaction in Delta Lake.

Over time, some slices may shrink below this threshold due to deletions. They will subsequently be merged with other slices and reorganized. When users perform an insert, they write new Parquet files into the table. When **OPTIMIZE** is triggered in SDC, these new files and any slices that are below the threshold size are reorganized. First, we partition the data into slices of the target size. We use the Qd-tree construction algorithm to do this splitting. We choose this over simple bin-packing so that data accesses together are in the same slices. Then, within each slice, we independently train and apply **Pando**. Each newly created slice is assigned a unique identifier and is stored with an individual index file containing its partition-tree structures. All slices and their indexes are transactionally committed together in a single new version.

At query time, we retrieve the set of slices in the current version along with their indexes from the log. These partition-tree indexes are used to identify data files that need to be scanned after skipping blocks that don't intersect the query predicates. **Pando** indexes are in the order of 100s of KBs in space. To avoid accessing too many small index files, index files for various slices are compacted and checkpointed, similar to how transaction logs are checkpointed in Delta Lake. SDC collects workload info for each slice. If a query intersects any file in a slice, then it's logged in the slice's workload. Clients buffer the queries and periodically flush them to storage at a specified path based on the slice's UUID.

We illustrate how SDC handles dynamic data using the example shown in Figure 4.2. The minimum slice size is 100 GB and the target slice size is 150 GB. Initially, the table contains 300 GB of data split evenly into two slices (UUIDs 1 and 2), each partitioned into files using Pando. Let’s use version 0 to denote this state of the table. The first operation is an `INSERT`. It appends 160 GB of data to the table as new file(s). They are initially not assigned to any slice. The new file(s) are added to the transaction log, and the operation is committed.

The next operation is an `UPDATE`. Since Delta Lake employs copy-on-write, files in slices 1 and 2 affected by the update are replaced by tombstone markers, and new files are added to the table. A subsequent `DELETE` operation similarly replaces a file in slice 1 with a tombstone marker. Each of these operations is sequentially committed, updating the set of live files and tombstones. Some files referenced in the original partition-tree indexes of slices 1 and 2 now point to files marked with tombstones because of the deletes. At query time, files that are scanned are validated against the list of active files in the version being read.

After the deletions, the size of slice 1 shrinks to 90 GB, falling below the minimum size threshold of 100 GB. When the next `OPTIMIZE` command is executed, it reorganizes the newly added files along with the small slice 1. This totals 290 GB, which is repartitioned into two slices, one with 150 GB and one with 140 GB, to closely match the target slice size. For each new slice created, a partition-tree layout is trained based on recent workload history. Finally, the new slices (UUIDs 3 and 4) with their new layouts are committed as version 4.

4.4.3 Dynamic Workloads

We now discuss how SDC adapts data layouts to dynamic workloads, where query patterns evolve over time. Real-world analytics often experience workload shifts, such as changes in the queried columns or distributions of query predicates. To maintain efficiency, SDC continuously monitors queries and incrementally optimizes layouts based on observed workload changes.

Our goal is to efficiently detect whether the current data layout remains optimal under changing workloads. One approach is to reoptimize using recent workloads on a data sample, then compare expected performance against the current layout. However, this method is expensive, as it requires sampling potentially large tables and rerunning optimization. Instead, we use a lightweight mechanism by directly monitoring workloads and comparing them to historical templates used at layout creation time. When a significant drift in workload is detected, we trigger reorganization of the data layout.

In this work, we assume a *steady-state dynamic workload*, where the query distribution changes at discrete points in time, after which it remains stable for a period. Common examples of such workload shifts occur when new dashboards or reporting templates are introduced, existing query templates are retired, or query filters shift periodically, such as daily, weekly, or monthly reporting tasks targeting different subsets of data. Our lightweight monitoring and comparison mechanism works well for efficiently detecting and adapting to such steady-state dynamic workloads.

We perform reoptimization at the granularity of slices. Each slice stores the workload distribution used to derive its current data layout along with its partition tree indexes. During query execution, if a slice is accessed, we log the query’s predicate patterns against that slice. These queries are recorded periodically in cloud object storage. To detect workload shifts, we compute the statistical distance between the current workload distribution and the originally

stored distribution for each slice. In this work, we specifically utilize Total Variation Distance to quantify this difference, but the general framework supports other distance metrics, which we leave for future work. When the distance between the recent workload and the current workload exceeds a predefined threshold, we trigger reoptimization of the slice.

Reoptimization occurs incrementally at slice granularity during scheduled OPTIMIZE operations. When invoked, we evaluate workload distances for slices queried since the previous optimization. If a significant workload shift is identified, we reorganize that slice. Incremental optimization of target-sized slices ensures predictable overhead, reducing interference with concurrent workloads. This facilitates simpler and more predictable application scheduling. It also saves compute and storage resources by avoiding unnecessary optimization of unchanged or lightly queried slices. By collecting slice-level workload statistics, SDC efficiently adapts data layouts to dynamic workloads, providing robust and predictable performance.

4.4.4 Parallelism

We now discuss how each incremental layout reorganization is implemented within SDC. Each slice selected for reoptimization undergoes a complete reclustering of its corresponding data files, based on the newly data layout. The reorganization operation is similar to a parallel repartitioning operation by data shuffling in frameworks such as Apache Spark. It follows a standard two-phase map-reduce execution strategy. In the map phase, multiple parallel tasks independently read subsets of files from the slice, shuffle records, and assign each record to a partition determined by the new partition-tree layout. In the reduce phase, records belonging to each partition are collected from all map tasks. Each reduce task combines these records into one or more final data files, ensuring target file sizes of the instance.

4.5 Evaluation

In this section, we present an experimental evaluation of SDC. Our experiments demonstrate the importance of adapting data layouts as data and workload conditions evolve. We show that SDC effectively adapts to these changes, resulting in significant performance improvements. Additionally, the overhead associated with incremental optimization remains modest, highlighting the practicality of our approach.

4.5.1 Experimental Setup

Our experiments were conducted using Apache Spark version 3.3.1. All experiments were run on Amazon Web Services (AWS) clusters consisting of one head node and four worker nodes. Each node was provisioned using AWS **r5.2xlarge** instances, which offer 8 virtual cores and 64GB of RAM. The evaluation data was stored in Amazon S3.

We used the LST-Bench benchmark suite to systematically evaluate SDC. We selected the TPC-DS dataset at scale factor 100 (SF100), and created a single-user workload by permuting the benchmark’s 99 standard queries. Specifically, to investigate the impact of dynamic data scenarios, we chose LST-Bench’s resiliency workload (WP2), which explicitly stresses the impact of the OPTIMIZE phase. To evaluate dynamic workload conditions, we modified the

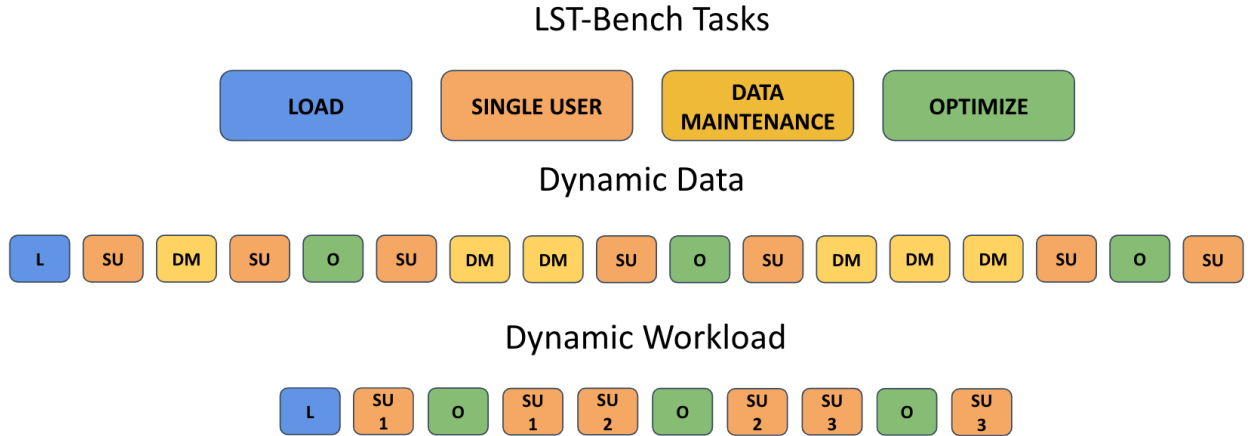


Figure 4.3: LST-Bench workload extensions to evaluate SDC. There are four different types corresponding to loading data, running TPC-DS queries, running TPC-DS data maintenance, and running OPTIMIZE. The sequence of operations for evaluating SDC under dynamic data and dynamic workload is shown above.

workload to introduce evolving query patterns. Specifically, we divided the templates into three sets of templates. Within each template, we generated 6 queries per template. We executed all the queries from one set once, then executed OPTIMIZE, and then the same set of queries from the set again. We repeat this for the other two sequentially. The workloads are shown in Figure 4.3.

To evaluate the SDC layout optimization with **Pando**, we trained the layout algorithm on a representative sample corresponding to 0.1% of the total dataset. The number of trees was set to 1. The minimum number of records in each block was set such that each file has 128 MB of uncompressed data. The minimum size for a slice is 10 GB, and the target size for a slice is 15 GB. We conducted performance tests comparing SDC against a Delta Lake deployment configured with Z-ordering. For Delta Lake with Z-order, we used columns from the most frequently executed queries as the manually specified layout configurations. In each scenario, we recorded the runtime for individual queries, as well as the overhead and invocation times of the OPTIMIZE operations quantify query performance and optimization costs explicitly.

4.5.2 Dynamic Data

Figure 4.4 shows the results for dynamic data scenarios. Each workload batch is labeled as SU- i , and the corresponding optimized version after invoking the OPTIMIZE command is labeled as SU- i O. Initially, layouts are not optimized, and both Z-order and SDC exhibit similar performance during SU-1. For SU-2, performance significantly degrades (up to 2.7X slower) for both layouts due to accumulating small files from frequent data modifications. After optimization (SU-2O), we observe a substantial improvement: the Z-order layout is up

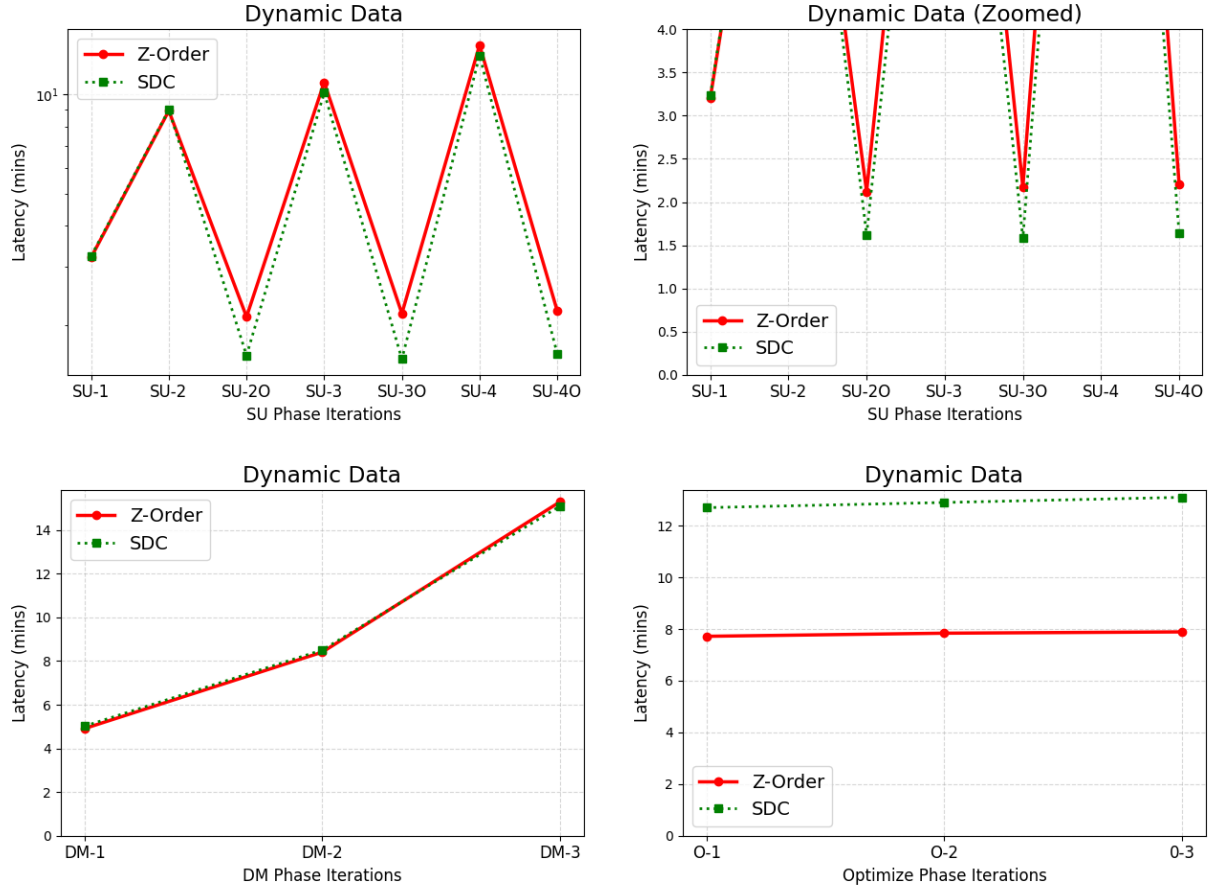


Figure 4.4: Performance of SDC with dynamic data.

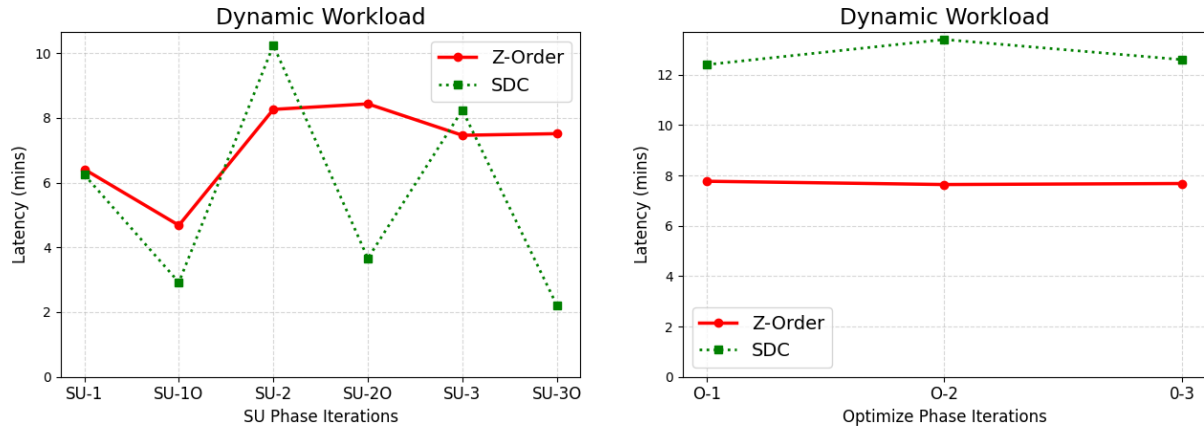


Figure 4.5: Performance of SDC with dynamic workloads.

to 1.5X faster, and SDC delivers up to 2X speedup compared to the non-optimized scenario and 1.3X compared to the Z-order layout. Subsequent batches, SU-3 and SU-4, progressively worsen due to further data maintenance operations. After optimization, however, SU-3O and SU-4O achieve similar performance to SU-2O because the data operations in the TPC-DS workload have uniform random access patterns.

SDC does not alter the write path, so its data maintenance performance closely matches that of Z-order. As expected, the maintenance overhead grows incrementally with successive operations.

Regarding optimization cost, both layouts involve sorting, an inherently expensive operation. However, the more sophisticated multi-dimensional layout used by SDC involves additional predicate evaluations compared to Z-order. Consequently, SDC optimization is up to 1.6X slower than Z-order. Optimization phases O-1, O-2, and O-3 take roughly the same time, as the total data size remains relatively stable due to the random update patterns in TPC-DS. Although later optimization steps require reading more files, sorting and writing data partitions dominate the total runtime rather than reading operations.

4.5.3 Dynamic Workloads

Figure 4.5 presents the results for dynamic workload scenarios. The clustering columns for the Z-order layout are set based on templates from SU-1. Initially, SDC and Z-order exhibit similar performance in SU-1, as their layouts are identical and not yet optimized. After optimization (SU-1O), SDC outperforms Z-order by 1.6X by better skipping data enabled by richer metadata. When the workload template changes (SU-2), the performance of both layouts deteriorates. SDC performs worse than Z-order initially because it has overfitted to the previous workload distribution. However, after applying OPTIMIZE, SDC adapts its layout to regain its performance, whereas Z-order remains unchanged due to fixed clustering columns. This adaptive optimization is also evident in SU-3 and SU-3O, where SDC achieves up to a 3.3X performance improvement over Z-order by dynamically reorganizing its layout.

The Z-order implementation in Delta Lake is neither idempotent nor incremental. As a result, optimization operations O-2 and O-3 incur the same overhead as O-1, despite no data changes. SDC, on the other hand, reorganizes all slices of the table when workloads change across the dataset, resulting in up to 1.7X slower optimization performance compared to Z-order.

4.6 Related Work

Data Partitioning: Carefully organizing data on storage for accelerating the performance of queries is a well-studied area. Most production systems often partition the data either based on ingestion time, or using range, hash, or round-robin distribution schemes [50]. Several techniques have been proposed to tune the data layout to reduce the amount of data that is scanned using horizontal, vertical, and hybrid partitioning [73, 24, 13, 83, 16, 33]. To prune the set of the blocks that are scanned, these systems maintain additional statistics along with these blocks such as minimum and maximum values for each attributes called Small Materialized Aggregates (SMAs) [59], and zone maps [36]. Slalom [64] logically partitions the

data and build lightweight partition-specific indexes, but they do not consider optimizing the physical layout of the data. [34] combines physical and virtual partitioning to fragment and dynamically tune partition sizes for efficient intra-query parallelism, but not data skipping. Distributed database systems employ various strategies to partition data between nodes to minimize the number of cross-partition queries [41, 55, 26]. Recent work has shown that significant gains in query performance can be achieved by more expressive partitioning schemes that collect rich metadata and can adapt to datasets and workloads [56, 47, 32, 95, 31, 84, 85]. Qd-tree [95], MTO [31], and Pando [85] are learned layouts that maximize data skipping in cloud-based DBMSs (discussed in this thesis).

Automated Physical Database Design: There is a rich corpus of work on automatically tuning a database and its physical design that focuses on selecting structures like indexes and materialized views for optimizing query performance [10, 22, 69, 12, 21, 13, 22, 10, 11, 100, 19, 70]. However, they only work for simple schemes like single or multi-column range and hash partitioning, b-tree indexes, etc. Materialized view selection algorithms explore the space of syntactically relevant views, focusing on creating logical views that cover queries, whereas we focus on optimizing the physical layout of the table [12, 99, 20]. Our layout schemes can be used to partition materialized views.

Recent work has explored techniques for adapting learned layouts such as QD-Tree and MTO. MTO reorganizes subtrees based on workload changes, balancing estimated benefit against reorganization cost, but assumes a fixed initial layout [31]. PAW embeds workload variance into partitioning to improve robustness to workload drift [54] but does not change the layout. SAT triggers full reorganization when the ratio of query selectivity to data-skipping falls below a threshold, offering no incremental adaptation [94]. OREO uses online algorithms without workload priors but incurs high overhead [74]. In contrast, SDCs take a lightweight, practical approach that works well in many real-world deployments where workload changes are triggered by specific events and can be easily detected through workload analysis. These systems adapt only to workload shifts and ignore data updates, inserts, and deletes which is a key focus of SDCs’ incremental maintenance.

4.7 Conclusion

We presented Self-Organizing Data Containers (SDCs), a practical cloud-native table storage layer designed to automatically adapt sophisticated data layouts as underlying workloads and data distributions evolve. By extending Delta Lake with expressive multi-dimensional partitioning schemes (Pando), our approach continuously reorganizes data through incremental optimization steps at carefully managed scales, guided by rich workload and data metadata. As demonstrated by our experimental evaluation, SDCs effectively address performance degradation caused by dynamic data and changing analytical queries, all while incurring modest optimization overhead. Through incremental, workload-aware layout optimization, SDCs unlock substantial and sustained performance benefits compared to existing systems that rely on simpler or manually-specified layouts.

Chapter 5

Conclusion and Future Work

This thesis explored the optimization of data layouts for evolving workloads in cloud table storage systems. Our key contribution has been to demonstrate how adaptive, metadata-driven data layout optimization can substantially improve the performance of analytical workloads running on disaggregated cloud architectures. We showed that incremental and adaptive data layout strategies, leveraging richer metadata about datasets, workloads, and queries, can enable higher efficiency and better responsiveness in practical data analytics scenarios than static or manually specified layouts.

We first introduced **Pando**, a correlation-aware data layout mechanism designed to leverage metadata about correlation among query predicates. **Pando** efficiently reduces data movement by significantly improving data skipping performance. By organizing data blocks based on explicitly collected correlations between query predicates, it delivers substantial performance improvements over simpler partitioning and sorting methods.

Next, we proposed **CopyRight**, a partial replication strategy designed to optimize layouts for workloads with heterogeneous access patterns. By selectively replicating subsets of data and separately optimizing each replica’s layout, **CopyRight** achieves substantial performance improvements for workloads involving multiple distinct analytical queries. This approach provides significant speed-up with minimal additional storage overhead.

Finally, we introduced Self-Organizing Data Containers (SDCs), a practical cloud-native table storage layer built on top of Delta Lake, designed to incrementally adapt **Pando** layout as data distributions and query workloads evolve over time. SDCs collect detailed metadata about queries and data distributions and incrementally reorganize underlying data layouts at fine-grained, manageable granularities. Thus, SDCs provide sustainable and consistently high performance as real-world data analytics workloads change, making them practical for long-running deployments.

5.1 Future Work

While this thesis makes several important advances, there remain exciting directions and open questions, which we discuss next.

Partial Replication for SDCs. Currently, our implementation of SDCs does not support replication. A natural extension is to integrate partial replication mechanisms proposed by CopyRight into SDCs. This would allow selective replication of data partitions or subsets that frequently exhibit competing access patterns. By combining incremental layout optimization with partial replication, SDCs could more efficiently handle dynamic real-world workloads characterized by multiple simultaneous analytical access patterns.

Fully Incremental Reorganization. Our current reorganization mechanism within each slice in SDCs involves a complete (though incremental at slice granularity) reorganization. Future work could explore making it truly incremental at the granularity of files. Rather than reorganizing entire slices, we could selectively reorganize only the small number of data files requiring optimization, substantially reducing compute resources and resulting overhead.

Automated Invocation of Incremental Optimization. Another important question is to decide systematically *when* to invoke the expensive optimization operations in practice. While managed frameworks such as AutoComp provide a viable strategy, future work could further automate invocation policies based on explicit performance thresholds or cost constraints. These automated policy frameworks would be highly beneficial in managed cloud environments to minimize user overhead, reduce costs, and further improve the efficiency and usability of SDCs.

Managing Resource Contention for Continuous Optimization. In the current library version, explicitly scheduling optimizations is critical due to potential resource contention with ongoing analytics operations. An interesting future direction is to develop a resource-aware optimization scheduler: a continuously running incremental variant that opportunistically runs small reorganization tasks only during idle compute periods or lower utilization cycles. Such a solution would ensure incremental data layout optimization occurs transparently, efficiently, and without detrimental impact on foreground workloads.

Extending to Complex and Richer Use Cases. Finally, future work can extend our adaptive storage layer approach to even richer analytics use cases containing complex, expensive-to-process data types. Examples include managing vector embeddings in machine learning workloads, organizing videos and images in large-scale multimedia analytics platforms, or efficiently supporting AI-driven predicate filtering from Large Language Models (LLMs). As these diverse data types increasingly populate data lakes, developing unified frameworks to efficiently organize and optimize them would significantly simplify data management and accelerate cloud analytics.

Bibliography

- [1] 2021. Apache Hudi. <https://hudi.apache.org/>.
- [2] 2021. Apache Iceberg. <https://iceberg.apache.org/>.
- [3] 2021. Delta Lake. <https://delta.io/>.
- [4] 2025. Amazon S3 - Cloud Object Storage - AWS. <https://aws.amazon.com/s3/>
- [5] 2025. Apache ORC. <https://orc.apache.org/>.
- [6] 2025. Apache Parquet. <https://parquet.apache.org/>.
- [7] 2025. Data Lake Storage for Big data Analytics | Microsoft Azure. <https://azure.microsoft.com/en-us/products/storage/data-lake-storage>
- [8] 2025. Google Cloud storage. <https://cloud.google.com/storage>
- [9] 2025. Store tabular data at scale – S3 Tables – AWS. <https://aws.amazon.com/s3/features/tables/>
- [10] Sanjay Agrawal, Nicolas Bruno, Surajit Chaudhuri, and Vivek Narasayya. 2006. AutoAdmin: Self-Tuning Database Systems Technology. *IEEE Data Engineering Bulletin* (2006), 7–15.
- [11] Sanjay Agrawal, Surajit Chaudhuri, Lubor Kollar, Arun Marathe, Vivek Narasayya, and Manoj Syamala. 2004. Database Tuning Advisor for Microsoft SQL Server 2005. In *VLDB* (vldb ed.). Very Large Data Bases Endowment Inc. <https://www.microsoft.com/en-us/research/publication/database-tuning-advisor-for-microsoft-sql-server-2005/>
- [12] Sanjay Agrawal, Surajit Chaudhuri, and Vivek R. Narasayya. 2000. Automated Selection of Materialized Views and Indexes in SQL Databases. In *Proceedings of the 26th International Conference on Very Large Data Bases (VLDB '00)*. Morgan Kaufmann Publishers Inc., San Francisco, CA, USA, 496–505.
- [13] Sanjay Agrawal, Vivek Narasayya, and Beverly Yang. 2004. Integrating Vertical and Horizontal Partitioning into Automated Physical Database Design. In *Proceedings of the 2004 ACM SIGMOD International Conference on Management of Data* (Paris, France) (*SIGMOD '04*). Association for Computing Machinery, New York, NY, USA, 359–370. <https://doi.org/10.1145/1007568.1007609>

- [14] Amazon [n.d.]. Managing the volume of merged rows. <https://docs.aws.amazon.com/redshift/latest/dg/vacuum-managing-volume-of-unmerged-rows.html>. Accessed: 2023-03-15.
- [15] Joy Arulraj, Andrew Pavlo, and Prashanth Menon. 2016. Bridging the archipelago between row-stores and column-stores for hybrid workloads. In *Proceedings of the 2016 International Conference on Management of Data*. 583–598.
- [16] Manos Athanassoulis, Kenneth S. Bøgh, and Stratos Idreos. 2019. Optimal Column Layout for Hybrid Workloads. *Proc. VLDB Endow.* 12, 13 (Sept. 2019), 2393–2407. <https://doi.org/10.14778/3358701.3358707>
- [17] Jon Louis Bentley. 1975. Multidimensional Binary Search Trees Used for Associative Searching. *Commun. ACM* 18, 9 (Sept. 1975), 509–517. <https://doi.org/10.1145/361002.361007>
- [18] Paul G. Brown and Peter J. Hass. 2003. BHUNT: Automatic Discovery of Fuzzy Algebraic Constraints in Relational Data. In *Proceedings of the 29th International Conference on Very Large Data Bases - Volume 29* (Berlin, Germany) (*VLDB '03*). VLDB Endowment, 668–679.
- [19] Nicolas Bruno and Surajit Chaudhuri. 2007. An Online Approach to Physical Design Tuning. In *2007 IEEE 23rd International Conference on Data Engineering*. 826–835. <https://doi.org/10.1109/ICDE.2007.367928>
- [20] Nicolas Bruno and Surajit Chaudhuri. 2007. Physical Design Refinement: The ‘Merge-Reduce’ Approach. *ACM Trans. Database Syst.* 32, 4 (Nov. 2007), 28–es. <https://doi.org/10.1145/1292609.1292618>
- [21] Surajit Chaudhuri and Vivek Narasayya. 2007. Self-Tuning Database Systems: A Decade of Progress. In *Proceedings of the 33rd International Conference on Very Large Data Bases* (Vienna, Austria) (*VLDB '07*). VLDB Endowment, 3–14.
- [22] Surajit Chaudhuri and Vivek R. Narasayya. 1997. An Efficient Cost-Driven Index Selection Tool for Microsoft SQL Server. In *Proceedings of the 23rd International Conference on Very Large Data Bases (VLDB '97)*. Morgan Kaufmann Publishers Inc., San Francisco, CA, USA, 146–155.
- [23] Zach Christopherson. 2016. Amazon Redshift Engineering’s Advanced Table Design Playbook: Compound and Interleaved Sort Keys. (2016). <https://aws.amazon.com/blogs/big-data/amazon-redshift-engineerings-advanced-table-design-playbook-compound-and-interleaved-sort-keys/>
- [24] Douglas W. Cornell and Philip S. Yu. 1990. An Effective Approach to Vertical Partitioning for Physical Design of Relational Databases. *IEEE Trans. Softw. Eng.* 16, 2 (Feb. 1990), 248–258. <https://doi.org/10.1109/32.44388>

- [25] Philippe Cudré-Mauroux, Eugene Wu, and Samuel Madden. 2009. The Case for RodentStore, an Adaptive, Declarative Storage System. *CoRR* abs/0909.1779 (2009). arXiv:0909.1779 <http://arxiv.org/abs/0909.1779>
- [26] Carlo Curino, Evan Philip Charles Jones, Yang Zhang, and Samuel R Madden. 2010. Schism: a workload-driven approach to database replication and partitioning. (2010).
- [27] Databricks [n.d.]. Data skipping index. <https://docs.databricks.com/delta/data-skipping.html/>. Accessed: 2022-12-01.
- [28] Angjela Davitkova, Evica Milchevski, and Sebastian Michel. [n.d.]. The ML-Index: A Multidimensional, Learned Index for Point, Range, and Nearest-Neighbor Queries.
- [29] K. Deb, A. Pratap, S. Agarwal, and T. Meyarivan. 2002. A fast and elitist multiobjective genetic algorithm: NSGA-II. *IEEE Transactions on Evolutionary Computation* 6, 2 (2002), 182–197. <https://doi.org/10.1109/4235.996017>
- [30] Jialin Ding, Matt Abrams, Sanghita Bandyopadhyay, Luciano Di Palma, Yanzhu Ji, Davide Pagano, Gopal Paliwal, Panos Parchas, Pascal Pfeil, Orestis Polychroniou, Gaurav Saxena, Aamer Shah, Amina Voloder, Sherry Xiao, Davis Zhang, and Tim Kraska. 2024. Automated Multidimensional Data Layouts in Amazon Redshift. In *Companion of the 2024 International Conference on Management of Data* (Santiago AA, Chile) (*SIGMOD '24*). Association for Computing Machinery, New York, NY, USA, 55–67. <https://doi.org/10.1145/3626246.3653379>
- [31] Jialin Ding, Umar Farooq Minhas, Badrish Chandramouli, Chi Wang, Yinan Li, Ying Li, Donald Kossmann, Johannes Gehrke, and Tim Kraska. 2021. Instance-Optimized Data Layouts for Cloud Analytics Workloads. In *Proceedings of the 2021 International Conference on Management of Data* (Virtual Event, China) (*SIGMOD '21*). Association for Computing Machinery, New York, NY, USA, 418–431. <https://doi.org/10.1145/3448016.3457270>
- [32] Jialin Ding, Vikram Nathan, Mohammad Alizadeh, and Tim Kraska. 2020. Tsunami: A Learned Multi-Dimensional Index for Correlated Data and Skewed Workloads. *Proc. VLDB Endow.* 14, 2 (nov 2020), 74–86. <https://doi.org/10.14778/3425879.3425880>
- [33] Markus Dreseler, Jan Kossmann, Martin Boissier, Stefan Klauck, Matthias Uflacker, and Hasso Plattner. [n.d.]. Hyrise Re-engineered: An Extensible Database System for Research in Relational In-Memory Data Management.
- [34] Camille Furtado, Alexandre A. B. Lima, Esther Pacitti, Patrick Valduriez, and Marta Mattoso. 2005. Physical and Virtual Partitioning in OLAP Database Clusters. In *Proceedings of the 17th International Symposium on Computer Architecture on High Performance Computing (SBAC-PAD '05)*. IEEE Computer Society, USA, 143–150. <https://doi.org/10.1109/CAHPC.2005.32>
- [35] GlobalData. 2024. Global data analytics market to reach \$190 billion in 2028, forecasts GlobalData. (12 2024). <https://www.globaldata.com/media/thematic-research/global-data-analytics-market-to-reach-190-billion-in-2028-forecasts-globaldata/>

- [36] Goetz Graefe. 2009. Fast Loads and Fast Queries. In *Proceedings of the 11th International Conference on Data Warehousing and Knowledge Discovery* (Linz, Austria) (*DaWaK '09*). Springer-Verlag, Berlin, Heidelberg, 111–124. https://doi.org/10.1007/978-3-642-03730-6_10
- [37] Anja Gruenheid, Jesús Camacho-Rodríguez, Carlo Curino, Raghu Ramakrishnan, Stanislav Pak, Sumedh Sakdeo, Lenisha Gandhi, Sandeep K. Singhal, Pooja Nilangekar, and Daniel J. Abadi. 2025. AutoComp: Automated Data Compaction for Log-Structured Tables in Data Lakes. In *Companion of the 2025 International Conference on Management of Data, SIGMOD/PODS 2025, Berlin, Germany, June 22-27, 2025*, Volker Markl, Joseph M. Hellerstein, and Azza Abouzied (Eds.). ACM, 404–417. <https://doi.org/10.1145/3722212.3724430>
- [38] Tu Gu, Kaiyu Feng, Gao Cong, Cheng Long, Zheng Wang, and Sheng Wang. 2021. The RLR-Tree: A Reinforcement Learning Based R-Tree for Spatial Data. arXiv:2103.04541 [cs.DB]
- [39] Antonin Guttman. 1984. R-Trees: A Dynamic Index Structure for Spatial Searching. In *Proceedings of the 1984 ACM SIGMOD International Conference on Management of Data* (Boston, Massachusetts) (*SIGMOD '84*). Association for Computing Machinery, New York, NY, USA, 47–57. <https://doi.org/10.1145/602259.602266>
- [40] Richard A. Hankins and Jignesh M. Patel. 2003. Data Morphing: An Adaptive, Cache-Conscious Storage Technique. In *Proceedings of the 29th International Conference on Very Large Data Bases - Volume 29* (Berlin, Germany) (*VLDB '03*). VLDB Endowment, 417–428.
- [41] Benjamin Hilprecht, Carsten Binnig, and Uwe Röhm. 2020. Learning a partitioning advisor for cloud databases. In *Proceedings of the 2020 ACM SIGMOD International Conference on Management of Data*. 143–157.
- [42] Srikanth Kandula, Laurel Orr, and Surajit Chaudhuri. 2019. Pushing Data-Induced Predicates through Joins in Big-Data Clusters. *Proc. VLDB Endow.* 13, 3 (nov 2019), 252–265. <https://doi.org/10.14778/3368289.3368292>
- [43] Nikos Karayannidis and Timos Sellis. 2003. SISYPHUS: The implementation of a chunk-based storage manager for OLAP data cubes. *Data & Knowledge Engineering* 45 (05 2003), 155–180. [https://doi.org/10.1016/S0169-023X\(02\)00178-7](https://doi.org/10.1016/S0169-023X(02)00178-7)
- [44] Michael S. Kester, Manos Athanassoulis, and Stratos Idreos. 2017. Access Path Selection in Main-Memory Optimized Data Systems: Should I Scan or Should I Probe?. In *Proceedings of the 2017 ACM International Conference on Management of Data* (Chicago, Illinois, USA) (*SIGMOD '17*). Association for Computing Machinery, New York, NY, USA, 715–730. <https://doi.org/10.1145/3035918.3064049>
- [45] Hideaki Kimura, George Huo, Alexander Rasin, Samuel Madden, and Stanley B. Zdonik. 2009. Correlation Maps: A Compressed Access Method for Exploiting Soft

- Functional Dependencies. *Proc. VLDB Endow.* 2, 1 (aug 2009), 1222–1233. <https://doi.org/10.14778/1687627.1687765>
- [46] Hideaki Kimura, George Huo, Alexander Rasin, Samuel Madden, and Stanley B. Zdonik. 2010. CORADD: Correlation Aware Database Designer for Materialized Views and Indexes. *Proc. VLDB Endow.* 3, 1–2 (sep 2010), 1103–1113. <https://doi.org/10.14778/1920841.1920979>
 - [47] Tim Kraska, M. Alizadeh, Alex Beutel, Ed H. Chi, Ani Kristo, Guillaume Leclerc, S. Madden, Hongzi Mao, and V. Nathan. 2019. SageDB: A Learned Database System. In *CIDR*.
 - [48] Tim Kraska, Alex Beutel, Ed H. Chi, Jeffrey Dean, and Neoklis Polyzotis. 2018. The Case for Learned Index Structures. In *Proceedings of the 2018 International Conference on Management of Data* (Houston, TX, USA) (*SIGMOD '18*). Association for Computing Machinery, New York, NY, USA, 489–504. <https://doi.org/10.1145/3183713.3196909>
 - [49] Sebastian Kruse and Felix Naumann. 2018. Efficient Discovery of Approximate Dependencies. *Proc. VLDB Endow.* 11, 7 (mar 2018), 759–772. <https://doi.org/10.14778/3192965.3192968>
 - [50] Per-Ake Larson, Cipri Clinciu, Campbell Fraser, Eric N. Hanson, Mostafa Mokhtar, Michal Nowakiewicz, Vassilis Papadimos, Susan L. Price, Srikumar Rangarajan, Remus Rusanu, and Mayukh Saubhasik. 2013. Enhancements to SQL Server Column Stores. In *Proceedings of the 2013 ACM SIGMOD International Conference on Management of Data* (New York, New York, USA) (*SIGMOD '13*). Association for Computing Machinery, New York, NY, USA, 1159–1168. <https://doi.org/10.1145/2463676.2463708>
 - [51] Juchang Lee, Kyu Hwan Kim, Hyejeong Lee, Mihnea Andrei, Seongyun Ko, Friedrich Keller, and Wook-Shin Han. 2020. Asymmetric-Partition Replication for Highly Scalable Distributed Transaction Processing in Practice. *Proc. VLDB Endow.* 13, 12 (Aug. 2020), 3112–3124. <https://doi.org/10.14778/3415478.3415538>
 - [52] Viktor Leis, Andrey Gubichev, Atanas Mirchev, Peter Boncz, Alfons Kemper, and Thomas Neumann. 2015. How Good Are Query Optimizers, Really? *Proc. VLDB Endow.* 9, 3 (nov 2015), 204–215. <https://doi.org/10.14778/2850583.2850594>
 - [53] Pengfei Li, Hua Lu, Qian Zheng, Long Yang, and Gang Pan. 2020. LISA: A Learned Index Structure for Spatial Data. In *Proceedings of the 2020 ACM SIGMOD International Conference on Management of Data* (Portland, OR, USA) (*SIGMOD '20*). Association for Computing Machinery, New York, NY, USA, 2119–2133. <https://doi.org/10.1145/3318464.3389703>
 - [54] Zhe Li, Man Lung Yiu, and Tsz Nam Chan. 2022. PAW: Data Partitioning Meets Workload Variance. In *2022 IEEE 38th International Conference on Data Engineering (ICDE)*. 123–135. <https://doi.org/10.1109/ICDE53745.2022.00014>

- [55] Yi Lu, Xiangyao Yu, and Samuel Madden. 2019. STAR: Scaling Transactions through Asymmetric Replication. *Proc. VLDB Endow.* 12, 11 (July 2019), 1316–1329. <https://doi.org/10.14778/3342263.3342270>
- [56] Samuel Madden, Jialin Ding, Tim Kraska, Sivaprasad Sudhir, David Cohen, Timothy Mattson, and Nesime Tatbul. 2022. Self-Organizing Data Containers. (2022).
- [57] Ryan Marcus, Olga Papaemmanouil, Sofiya Semenova, and Solomon Garber. 2018. NashDB: an end-to-end economic method for elastic database fragmentation, replication, and provisioning. In *Proceedings of the 2018 International Conference on Management of Data*. 1253–1267.
- [58] Microsoft [n.d.]. Columnstore Indexes Query performance. <https://learn.microsoft.com/en-us/sql/relational-databases/indexes/columnstore-indexes-query-performance?view=sql-server-ver16>. Accessed: 2022-12-01.
- [59] Guido Moerkotte. 1998. Small Materialized Aggregates: A Light Weight Index Structure for Data Warehousing. In *Proceedings of the 24rd International Conference on Very Large Data Bases (VLDB '98)*. Morgan Kaufmann Publishers Inc., San Francisco, CA, USA, 476–487.
- [60] Vikram Nathan, Jialin Ding, Mohammad Alizadeh, and Tim Kraska. 2020. Learning multi-dimensional indexes. In *Proceedings of the 2020 ACM SIGMOD International Conference on Management of Data*. 985–1000.
- [61] Vikram Nathan, Jialin Ding, Tim Kraska, and Mohammad Alizadeh. 2020. Cortex: Harnessing Correlations to Boost Query Performance. <https://doi.org/10.48550/ARXIV.2012.06683>
- [62] J. Nievergelt, Hans Hinterberger, and Kenneth C. Sevcik. 1984. The Grid File: An Adaptable, Symmetric Multikey File Structure. *ACM Trans. Database Syst.* 9, 1 (March 1984), 38–71. <https://doi.org/10.1145/348.318586>
- [63] Shoji Nishimura and Haruo Yokota. 2017. QUILTS: Multidimensional Data Partitioning Framework Based on Query-Aware and Skew-Tolerant Space-Filling Curves. In *Proceedings of the 2017 ACM International Conference on Management of Data (Chicago, Illinois, USA) (SIGMOD '17)*. Association for Computing Machinery, New York, NY, USA, 1525–1537. <https://doi.org/10.1145/3035918.3035934>
- [64] Matthaios Olma, Manos Karpapthiotakis, Ioannis Alagiannis, Manos Athanassoulis, and Anastasia Ailamaki. 2017. Slalom: Coasting through Raw Data via Adaptive Partitioning and Indexing. *Proc. VLDB Endow.* 10, 10 (jun 2017), 1106–1117. <https://doi.org/10.14778/3115404.3115415>
- [65] OmniSci [n.d.]. OmniSci. <https://www.omnisci.com/>. Accessed: 2021-03-24.
- [66] Beng Chin Ooi, Ron Sacks-davis, and Jiawei Han. [n.d.]. Indexing in Spatial Databases.

- [67] Oracle [n.d.]. Database Data Warehousing Guide. https://docs.oracle.com/database/121/DWHSG/zone_maps.htm#DWHSG-GUID-BEA5ACA1-6718-4948-AB38-1F2C0335FDE4. Accessed: 2022-12-01.
- [68] E. J. Otoo, Doron Rotem, and Sridhar Seshadri. 2007. Optimal Chunking of Large Multidimensional Arrays for Data Warehousing. In *Proceedings of the ACM Tenth International Workshop on Data Warehousing and OLAP* (Lisbon, Portugal) (*DOLAP '07*). Association for Computing Machinery, New York, NY, USA, 25–32. <https://doi.org/10.1145/1317331.1317337>
- [69] Stefano Paraboschi, Giuseppe Sindoni, Elena Baralis, and Ernest Teniente. 2003. *Materialized Views in Multidimensional Databases*. IGI Global, USA, 222–251.
- [70] Andrew Pavlo, Matthew Butrovich, Lin Ma, Prashanth Menon, Wan Shen Lim, Dana Van Aken, and William Zhang. 2021. Make your database system dream of electric sheep: towards self-driving operation. *Proc. VLDB Endow.* 14, 12 (July 2021), 3211–3221. <https://doi.org/10.14778/3476311.3476411>
- [71] Jianzhong Qi, Guanli Liu, Christian S. Jensen, and Lars Kulik. 2020. Effectively Learning Spatial Indices. *Proc. VLDB Endow.* 13, 12 (July 2020), 2341–2354. <https://doi.org/10.14778/3407790.3407829>
- [72] Ravishankar Ramamurthy, David J. DeWitt, and Qi Su. 2002. A Case for Fractured Mirrors. In *Proceedings of the 28th International Conference on Very Large Data Bases* (Hong Kong, China) (*VLDB '02*). VLDB Endowment, 430–441.
- [73] Jun Rao, Chun Zhang, Nimrod Megiddo, and Guy Lohman. 2002. Automating Physical Database Design in a Parallel Database. In *Proceedings of the 2002 ACM SIGMOD International Conference on Management of Data* (Madison, Wisconsin) (*SIGMOD '02*). Association for Computing Machinery, New York, NY, USA, 558–569. <https://doi.org/10.1145/564691.564757>
- [74] Kexin Rong, Paul Liu, Sarah Ashok Sonje, and Moses Charikar. 2024. Dynamic Data Layout Optimization with Worst-Case Guarantees . In *2024 IEEE 40th International Conference on Data Engineering (ICDE)*. IEEE Computer Society, Los Alamitos, CA, USA, 4288–4301. <https://doi.org/10.1109/ICDE60146.2024.00327>
- [75] Florin Rusu and Yu Cheng. 2013. A Survey on Array Storage, Query Languages, and Systems. arXiv:1302.0103 [cs.DB]
- [76] S. Sarawagi and M. Stonebraker. 1994. Efficient organization of large multidimensional arrays. In *Proceedings of 1994 IEEE 10th International Conference on Data Engineering*. 328–336. <https://doi.org/10.1109/ICDE.1994.283048>
- [77] P. Griffiths Selinger, M. M. Astrahan, D. D. Chamberlin, R. A. Lorie, and T. G. Price. 1979. Access Path Selection in a Relational Database Management System. In *Proceedings of the 1979 ACM SIGMOD International Conference on Management of Data* (Boston, Massachusetts) (*SIGMOD '79*). Association for Computing Machinery, New York, NY, USA, 23–34. <https://doi.org/10.1145/582095.582099>

- [78] Raghav Sethi, Martin Traverso, Dain Sundstrom, David Phillips, Wenlei Xie, Yutian Sun, Nezih Yegitbasi, Haozhun Jin, Eric Hwang, Nileema Shingte, and Christopher Berner. 2019. Presto: SQL on Everything. In *35th IEEE International Conference on Data Engineering, ICDE 2019, Macao, China, April 8-11, 2019*. IEEE, 1802–1813. <https://doi.org/10.1109/ICDE.2019.00196>
- [79] Hari Singh and Seema Bawa. 2017. A Survey of Traditional and MapReduceBased Spatial Query Processing Approaches. *SIGMOD Rec.* 46, 2 (Sept. 2017), 18–29. <https://doi.org/10.1145/3137586.3137590>
- [80] Muthian Sivathanu, Midhul Vuppalapati, Bhargav S. Gulavani, Kaushik Rajan, Jyoti Leeka, Jayashree Mohan, and Piyus Kedia. 2020. INSTalytics: Cluster Filesystem Co-Design for Big-Data Analytics. *ACM Trans. Storage* 15, 4, Article 23 (Jan. 2020), 30 pages. <https://doi.org/10.1145/3369738>
- [81] Zack Slayton. 2017. Z-Order Indexing for Multifaceted Queries in Amazon DynamoDB. <https://aws.amazon.com/blogs/database/z-order-indexing-for-multifaceted-queries-in-amazon-dynamodb-part-1/>. Accessed: 2021-05-31.
- [82] Emad Soroush, Magdalena Balazinska, and Daniel Wang. 2011. ArrayStore: A Storage Manager for Complex Parallel Array Processing. In *Proceedings of the 2011 ACM SIGMOD International Conference on Management of Data (Athens, Greece) (SIGMOD '11)*. Association for Computing Machinery, New York, NY, USA, 253–264. <https://doi.org/10.1145/1989323.1989351>
- [83] Mike Stonebraker, Daniel J. Abadi, Adam Batkin, Xuedong Chen, Mitch Cherniack, Miguel Ferreira, Edmond Lau, Amerson Lin, Sam Madden, Elizabeth O’Neil, Pat O’Neil, Alex Rasin, Nga Tran, and Stan Zdonik. 2005. C-Store: A Column-Oriented DBMS. In *Proceedings of the 31st International Conference on Very Large Data Bases (Trondheim, Norway) (VLDB '05)*. VLDB Endowment, 553–564.
- [84] Sivaprasad Sudhir, Michael Cafarella, and Samuel Madden. 2022. Replicated Layout for In-Memory Database Systems. *Proc. VLDB Endow.* 15, 4 (apr 2022), 984–997. <https://doi.org/10.14778/3503585.3503606>
- [85] Sivaprasad Sudhir, Wenbo Tao, Nikolay Laptev, Cyrille Habis, Michael Cafarella, and Samuel Madden. 2023. Pando: Enhanced Data Skipping with Logical Data Partitioning. *Proc. VLDB Endow.* 16, 9 (May 2023), 2316–2329. <https://doi.org/10.14778/3598581.3598601>
- [86] Liwen Sun, Michael J. Franklin, Sanjay Krishnan, and Reynold S. Xin. 2014. Fine-Grained Partitioning for Aggressive Data Skipping. In *Proceedings of the 2014 ACM SIGMOD International Conference on Management of Data (Snowbird, Utah, USA) (SIGMOD '14)*. Association for Computing Machinery, New York, NY, USA, 1115–1126. <https://doi.org/10.1145/2588555.2610515>

- [87] Liwen Sun, Michael J. Franklin, Jiannan Wang, and Eugene Wu. 2016. Skipping-Oriented Partitioning for Columnar Layouts. *Proc. VLDB Endow.* 10, 4 (nov 2016), 421–432. <https://doi.org/10.14778/3025111.3025123>
- [88] Junjay Tan, Thanaa Ghanem, Matthew Perron, Xiangyao Yu, Michael Stonebraker, David DeWitt, Marco Serafini, Ashraf Aboulmaga, and Tim Kraska. 2019. Choosing a Cloud DBMS: Architectures and Tradeoffs. *Proc. VLDB Endow.* 12, 12 (Aug. 2019), 2170–2182.
- [89] Ashish Thusoo, Joydeep Sarma, Namit Jain, Zheng Shao, Prasad Chakka, Ning Zhang, Suresh Anthony, Hao Liu, and Raghotham Murthy. 2010. Hive - A Petabyte Scale Data Warehouse Using Hadoop. *Proceedings - International Conference on Data Engineering*, 996–1005. <https://doi.org/10.1109/ICDE.2010.5447738>
- [90] TPC-DS [n.d.]. TPC-DS. <https://www.tpc.org/tpcds/>. Accessed: 2022-12-01.
- [91] TPC-H [n.d.]. TPC-H. <https://www.tpc.org/tpch/>. Accessed: 2022-12-01.
- [92] Haixin Wang, Xiaoyi Fu, Jianliang Xu, and Hua Lu. 2019. Learned Index for Spatial Queries. In *20th IEEE International Conference on Mobile Data Management (MDM)*. IEEE, United States, 569–574. <https://doi.org/10.1109/MDM.2019.00121> 20th IEEE International Conference on Mobile Data Management (MDM), MDM 2019 ; Conference date: 10-06-2019 Through 13-06-2019.
- [93] Yingjun Wu, Jia Yu, Yuanyuan Tian, Richard Sidle, and Ronald Barber. 2019. Designing Succinct Secondary Indexing Mechanism by Exploiting Column Correlations. In *Proceedings of the 2019 International Conference on Management of Data (Amsterdam, Netherlands) (SIGMOD '19)*. Association for Computing Machinery, New York, NY, USA, 1223–1240. <https://doi.org/10.1145/3299869.3319861>
- [94] Xiaoxiao Xie, Shengfei Shi, Hongzhi Wang, and Mohan Li. 2023. SAT: sampling acceleration tree for adaptive database repartition. *World Wide Web* 26, 5 (Aug. 2023), 3503–3533. <https://doi.org/10.1007/s11280-023-01199-3>
- [95] Zongheng Yang, Badrish Chandramouli, Chi Wang, Johannes Gehrke, Yinan Li, Umar Farooq Minhas, Per-Åke Larson, Donald Kossmann, and Rajeev Acharya. 2020. Qd-Tree: Learning Data Layouts for Big Data Analytics. In *Proceedings of the 2020 ACM SIGMOD International Conference on Management of Data (Portland, OR, USA) (SIGMOD '20)*. Association for Computing Machinery, New York, NY, USA, 193–208. <https://doi.org/10.1145/3318464.3389770>
- [96] Matei Zaharia, Mosharaf Chowdhury, Michael J. Franklin, Scott Shenker, and Ion Stoica. 2010. Spark: Cluster Computing with Working Sets. In *Proc. SOCC*. 10.
- [97] Ramon Antonio Rodrigues Zalipynis. 2018. ChronosDB: Distributed, File Based, Geospatial Array DBMS. *Proc. VLDB Endow.* 11, 10 (June 2018), 1247–1261. <https://doi.org/10.14778/3231751.3231754>

- [98] Yihong Zhao, Prasad M. Deshpande, and Jeffrey F. Naughton. 1997. An Array-Based Algorithm for Simultaneous Multidimensional Aggregates. *SIGMOD Rec.* 26, 2 (June 1997), 159–170. <https://doi.org/10.1145/253262.253288>
- [99] D. Zilio, C. Zuzarte, S. Lightstone, Wenbin Ma, G. Lohman, R. Cochrane, H. Pirahesh, L. Colby, Jarek Gryz, E. Alton, Dongming Liang, and G. Valentin. 2004. Recommending materialized views and indexes with the IBM DB2 design advisor. *International Conference on Autonomic Computing, 2004. Proceedings.* (2004), 180–187.
- [100] Daniel C. Zilio, Jun Rao, Sam Lightstone, Guy Lohman, Adam Storm, Christian Garcia-Arellano, and Scott Fadden. 2004. DB2 design advisor: integrated automatic physical database design. In *Proceedings of the Thirtieth International Conference on Very Large Data Bases - Volume 30* (Toronto, Canada) (*VLDB '04*). VLDB Endowment, 1087–1097.